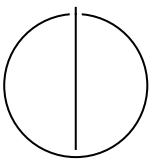
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Evaluation of Live Migration Strategies at the Edge

Ivo Raimondi



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

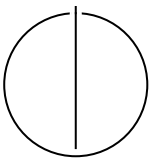
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Evaluation of Live Migration Strategies at the Edge

Bewertung von Live-Migrationsstrategien am Edge

Author:	Ivo Raimondi
Examiner:	Prof. Dr.-Ing. Jörg Ott
Supervisor:	M.Sc. Giovanni Bartolomeo
Submission Date:	16 July 2026



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 16 July 2026 Ivo Raimondi

Acknowledgments

If I were to list every person to whom I am grateful, these acknowledgments would never end. I would first like to thank my parents, my sister, and my whole family in Spain and Argentina for their constant support, encouragement, and trust throughout my studies and in every project I decided to pursue.

I am also deeply grateful to my friends, who have made me feel supported and loved both in the good moments and in the difficult ones. Their presence has made this journey much easier and much more meaningful.

I would like to thank the University of Seville and all the people who have been part of my education during these four years. I am equally grateful to the Technical University of Munich for giving me the opportunity to complete the final year of my Bachelor's degree as an Erasmus+ student in such an inspiring academic environment. This has been an experience that I will never forget.

I would also like to thank all members of the Chair of Connected Mobility. In particular, I thank Prof. Dr.-Ing. Jörg Ott for hosting this thesis, and M.Sc. Giovanni Bartolomeo for his guidance, patience, and support throughout this work and during my time in the wonderful Oakestra team.

Finally, I would like to thank the members of NATO IST-240. Their work and discussions helped shape the topic of this thesis, and I am grateful for their help, feedback, and for giving me the opportunity to attend one of their meetings and present part of my results. It was a unique experience that helped me learn a lot.

To all of you:

Gracias de corazón.

Abstract

Edge deployments often need to move services when users move, nodes become overloaded, links degrade, or operational requirements change. Live migration can preserve execution state and reduce service disruption, but its cost depends on the application state, the network path, and the mechanism used to move the service. This thesis evaluates live migration strategies at the edge under explicit network constraints, rather than assuming a local virtual bridge or a data-center link.

The work compares native Linux process migration with Checkpoint/Restore In Userspace (CRIU) and application-aware WebAssembly (Wasm) migration. The CRIU experiments cover cold migration, pre-copy, and post-copy. The Wasm experiments use a runtime-level checkpointing mechanism for a transformed WebAssembly module. The benchmark framework runs these experiments across WiFi 6, 5G, Long-Term Evolution (LTE), Starlink, and three progressively constrained tactical edge profiles. Four workloads expose different forms of migration state: a Counter process, a Transmission Control Protocol (TCP) client, a memory-heavy Game of Life simulation, and a Wasm matrix multiplication workload. The evaluation measures downtime, phase breakdowns, transfer cost, archive size, resource usage, and workload-level success after restore.

The results show that network conditions become decisive once the migrated state is large enough. For small workloads, fixed orchestration and transfer setup costs remain visible, and migration is often feasible even under constrained links. For Game of Life, the larger heap makes memory movement dominate. Cold migration is simple, but places the full state-transfer cost inside downtime. Pre-copy reduces the final unavailable window for memory-heavy workloads, but increases total migration time and depends on the workload not dirtying memory faster than the pre-copy iterations can progress. Post-copy can create very small initial archives and low measured downtime. It still depends on lazy page delivery after restore, which makes it fragile under lossy or slow profiles. The Wasm path remains successful through the most constrained profile while capturing application-level state instead of a full Linux process image, although this specialization reduces generality. Overall, the experiments show that edge migration cannot be evaluated with one workload or one network assumption. The migration approach must match workload behavior, link quality, transfer topology, and the required validation criterion.

Contents

1. Introduction

Edge computing is a distributed computing model that moves computation and data storage closer to users. It reduces the need to send all data to a distant cloud. It can also reduce latency for applications that interact with the physical world. This is especially relevant for Tactical Edge Environment (TEE), which are the initial motivation for this thesis. In a TEE, compute nodes, sensors, and users operate under difficult conditions. Links may have low bandwidth, high latency, packet loss, or intermittent connectivity. Nodes may be mobile, with limited power and hardware resources. In federated tactical cloud environments, centralized cloud assumptions do not hold and adaptive workload distribution is needed to keep services useful under changing mission and network conditions [1].

Live migration is one possible mechanism for adaptive placement. It moves a running computation from one node to another while preserving execution state. Survey work on container live migration frames this mechanism as a tool for availability, mobility, resilience, and resource management in future networks [2]. A migration can help a service follow a user, leave a failing node, avoid a congested link, or use a better execution location. The idea is simple: move the service without starting it again from the beginning. Estimating the cost is harder. The system has to capture state, move it through the network, restore it on the destination, and check that execution continues correctly. Some of this cost appears as downtime. Other work happens before or after the unavailable window, but it still determines whether migration is practical.

This thesis evaluates these costs. It compares native Linux process migration based on CRIU with Wasm-based checkpoint/restore migration. The evaluation is performed under controlled network profiles. The profiles include WiFi 6, 5G, LTE, Starlink-like, and tactical edge conditions. The measured metrics include downtime, checkpoint time, transfer time, restore time, archive size, transfer subphases, and resource usage.

The goal is not to propose a new migration algorithm. The goal is to build a reproducible measurement framework and use it to see where migration time is spent. A low-level migration mechanism can look promising in isolation. Once transfer setup, archive creation, restore overhead, and network conditions are included, the same mechanism may no longer be useful.

1.1. Motivation

Edge systems are heterogeneous. They include cloud servers, regional edge nodes, small local machines, and resource-constrained devices. Their networks also differ in bandwidth, delay, jitter, and loss. A placement decision can be good at one point in time and poor later.

Static placement is therefore limited. A service may need to move because a node becomes overloaded, a link degrades, a user moves, or a mission changes. Restarting the service elsewhere is often simple, but it can lose state and interrupt users. Live migration offers a stronger abstraction because it tries to preserve execution state while reducing service disruption. The practical question is whether that abstraction is suitable. A migration mechanism is useful only when its overhead fits the application and the network.

This matters at the tactical edge. Tactical networks are described as disadvantaged networks with limited bandwidth, intermittent connectivity, and variable latency [1]. A migration technique that works on a local virtual bridge may not work under those constraints. Much state-of-the-art migration work also has a different target. It often focuses on data-center availability, mobility in edge networks, moving-target defense, or cold-start avoidance in serverless systems [2]. This thesis asks a narrower question: how expensive is migration when the state must cross explicit constrained links?

The amount and type of state also change by workload. A counter has little dynamic state. A Game of Life process stresses memory transfer. A TCP client adds connection continuity. A Wasm workload moves runtime-level state instead of a Linux process tree.

There is also a technology question. Native Linux migration with CRIU can preserve rich process state, including memory and, in some cases, TCP sockets. It is close to the operating system and powerful. It also depends on kernel features, compatible execution environments, file-system availability, and careful network handling. Wasm takes a different path. It offers a portable sandbox and can represent execution state at the runtime or bytecode level [3]. That makes it attractive for heterogeneous edge systems, but it shifts overhead into runtime support and checkpoint/restore logic. This thesis examines both paths in the same experimental framework.

This thesis analyzes these costs through measurement. It asks how much downtime each approach introduces, which phase dominates that downtime, and how network profiles change the result for each strategy.

1.2. Contributions

This thesis makes the following contributions:

- An automated experimental setup for repeated live migration measurements using Terraform [4], Multipass virtual machines [5], Python orchestrators [6], traffic-control profiles, and plot generation.
- A heterogeneous benchmark suite with four workloads: a minimal counter, a memory-intensive Game of Life process, a TCP client with connection continuity requirements, and a Wasm workload.
- A downtime model that separates checkpointing, transfer, and restore. The transfer phase is also decomposed into archive creation, setup, copy, unpacking, and cleanup costs.
- An evaluation of the migration mechanisms under seven network profiles. The profiles represent WiFi, 5G, LTE, Starlink-like, and three tactical edge conditions.
- Resource-usage measurements based on node exporter snapshots, with CPU, memory, and disk summaries during migration.

The implementation is designed for reproducibility. Network conditions are stored as structured profile data. Benchmark suites are registered in a common configuration file. Run identifiers are written to log files and later used for plot filtering. As a result, plots can be regenerated for a specific batch even when the Comma-Separated Values (CSV) files contain older experiments.

1.3. Thesis Structure

Chapter 2 introduces edge computing, TEE, live migration, checkpoint/restore with CRIU, and Wasm migration. Chapter 3 describes the benchmark framework, workloads, transfer modes, network profiles, and data processing. Chapter 4 presents the experiment overview, downtime results, phase breakdowns, transfer analysis, and resource usage. Chapter 5 summarizes the findings and outlines future work.

2. Background

2.1. Edge Computing and the Compute Continuum

Cloud computing centralizes compute and storage in large data centers. This model provides scale, but it can place computation far from users and data sources. Edge computing moves part of that computation closer to the network edge [2]. Its goal is to reduce latency, reduce uplink traffic, improve locality, and support applications that interact with nearby devices.

The compute continuum extends this idea. It treats cloud, edge, and device resources as parts of one execution environment. This view is useful because applications do not always fit one fixed layer [3]. Some work belongs in the cloud. Some work belongs near the user. Some work must run on local devices. The right placement can change over time.

The main difficulty is heterogeneity. Nodes can differ in CPU capacity, memory, storage, energy, architecture, operating system support, and network access.

The network varies too. A link can be high-bandwidth and stable, or low-bandwidth and lossy. Placement and migration therefore become runtime problems, not only deployment decisions.

Live migration fits this setting because it can move stateful computation between nodes. It is different from stateless redeployment, where a new instance starts and depends on external storage or application-level recovery [2]. Live migration attempts to preserve execution state directly. That can reduce application disruption, but it also creates system overhead.

2.2. Tactical Edge Environments

A TEE is an edge environment deployed close to an operational mission [1, 7]. It differs from a stable data center. Hardware may be limited. Nodes may move. Power may be constrained. Communication may rely on tactical networks with limited bandwidth, intermittent connectivity, high latency, and packet loss. These conditions make centralized cloud assumptions weak [1].

Prior tactical edge work motivates adaptive computing for this reason. Tactical edge

computing can process sensor data close to the source, reduce bandwidth use, and avoid sending unneeded raw data across constrained links [7]. Federated tactical cloud work also argues for adaptive workload distribution across mission clouds, with services for resource discovery, service deployment, trust management, and adaptive orchestration [1]. Tactical edge systems are therefore a stress case for migration. Adaptation is useful only when its benefit is larger than its cost. In tactical edge work, this cost includes bandwidth impact, planning time, and CPU and memory overhead of the adaptive services [7]. For live migration, the cost also includes the time to capture state, the bytes moved through the constrained link, the restore work at the destination, and the period in which the service is frozen. A method that improves placement can still be impractical if the migration consumes the same scarce bandwidth needed by the mission workload.

A service may need to move because a node goes offline, because a link becomes worse, or because a better processing location appears. A migration that takes seconds on a local virtual network may take much longer under a tactical profile. The tactical context also justifies explicit network profiles. A single default virtual network does not represent the range of links that may appear in the field. Bandwidth, delay, and loss must be controlled. In this thesis, those conditions are profiled, applied automatically, and assessed consistently across workloads. This makes experiments repeatable and makes the link assumptions visible.

2.3. Live Migration

Live migration moves a service instance from a source node to a destination node while the system tries to keep the service available [2]. It is used to improve availability, resilience, mobility, load balancing, and resource management in edge-to-cloud systems. Unlike a restart on a new node, live migration tries to preserve the state of the running service.

The difficulty depends on whether the service is stateless or stateful. A stateless service can often be recreated on another node because its dynamic state is external or disposable. A stateful service has execution state that changes during operation. That state may include memory pages, open files, runtime data structures, and network connections. For this reason, stateful live migration usually follows a checkpoint, transfer, restore, and validation sequence. The checkpoint captures process or runtime state. The transfer phase moves that state to the destination. The restore phase recreates the service. Validation checks whether execution continues correctly.

The most important user-facing metric is downtime. Downtime is the period in which the service is unavailable or frozen. For a basic stateful migration, downtime

includes the checkpoint, the transfer of the checkpoint, and the restore. It is not always equal to the total migration time. A strategy can perform some work before or after the downtime window.

This thesis considers three native migration strategies. They differ in where they place checkpointing, transfer, and restore work relative to the downtime window.

2.3.1. Cold Migration

Cold migration is the baseline strategy. The service is stopped first. Then the source node checkpoints all required state, transfers that state to the destination, and restores the service there. This makes the control flow simple and robust. It also means that the full checkpoint, transfer, and restore path contributes to downtime. Cold migration is therefore useful as a lower-complexity reference point, but it can be expensive when the checkpoint is large or the network is slow. Figure 2.1 summarizes the sequence.

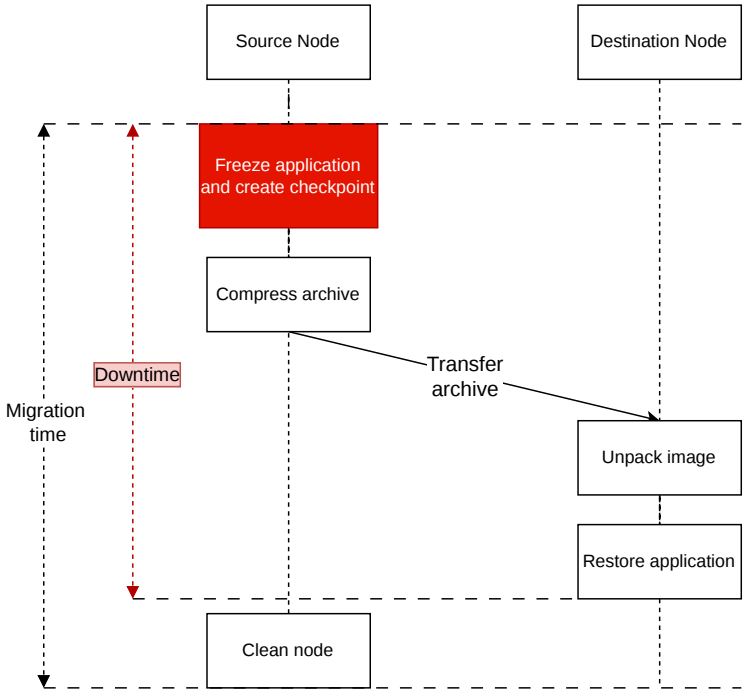


Figure 2.1.: Cold migration workflow.

2.3.2. Pre-copy Migration

Pre-copy migration moves part of the state before the final stop. The source performs one or more pre-dumps while the service keeps running. Each pre-dump captures memory state, and later iterations can focus on memory that changed after the previous pre-dump. The service is stopped only for the final dump, final transfer, and restore. This can reduce downtime because less work remains in the frozen window. The trade-off is that the service can update memory while pre-dumps are running. Later pre-dumps must then transfer newer versions of those dirty pages. If pages are dirtied repeatedly, pre-copy can transfer more data and spend longer in preparation without reducing the final delta as much. Figure 2.2 shows this separation between preparation and downtime.

2.3.3. Post-copy Migration

Post-copy migration moves the service to the destination earlier. The source creates an initial checkpoint that is sufficient to restore execution on the destination. After restore, missing memory pages are fetched from the source when the restored service accesses them. This can reduce the initial transfer before execution resumes, because only the state needed to start the process must arrive before restore. It also introduces a dependency after restore: the source must remain available as a page server until the destination has received the missing state. Figure 2.3 shows the page-fetch phase after restore.

2.3.4. Strategy Comparison

The preceding strategies expose different timing boundaries. Downtime is treated as the interval in which the workload cannot make progress for the user. Cold migration places checkpoint creation, archive transfer, unpacking, and restore inside this interval. Pre-copy moves repeated pre-dumps before downtime while the workload keeps running; downtime starts with the final checkpoint and ends after restore. Post-copy restores from a smaller initial checkpoint and lets execution resume earlier, but it continues to fetch missing memory pages after restore.

The abstract strategy describes when state is moved. The concrete archive path is an experimental variable in this work and is described in Chapter 3.

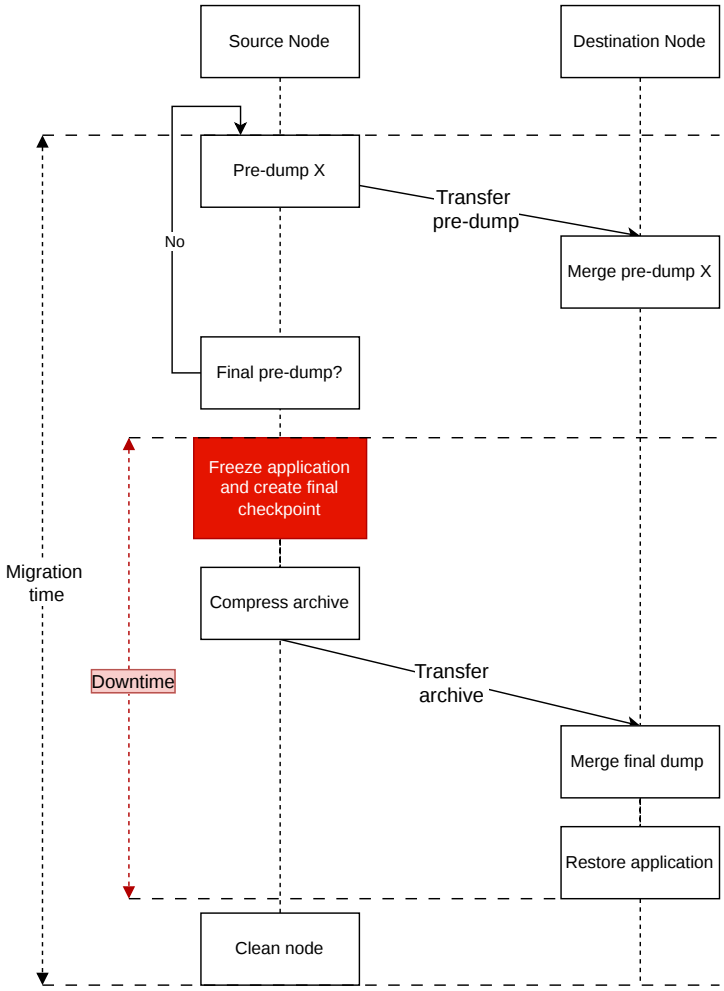


Figure 2.2.: Pre-copy migration workflow.

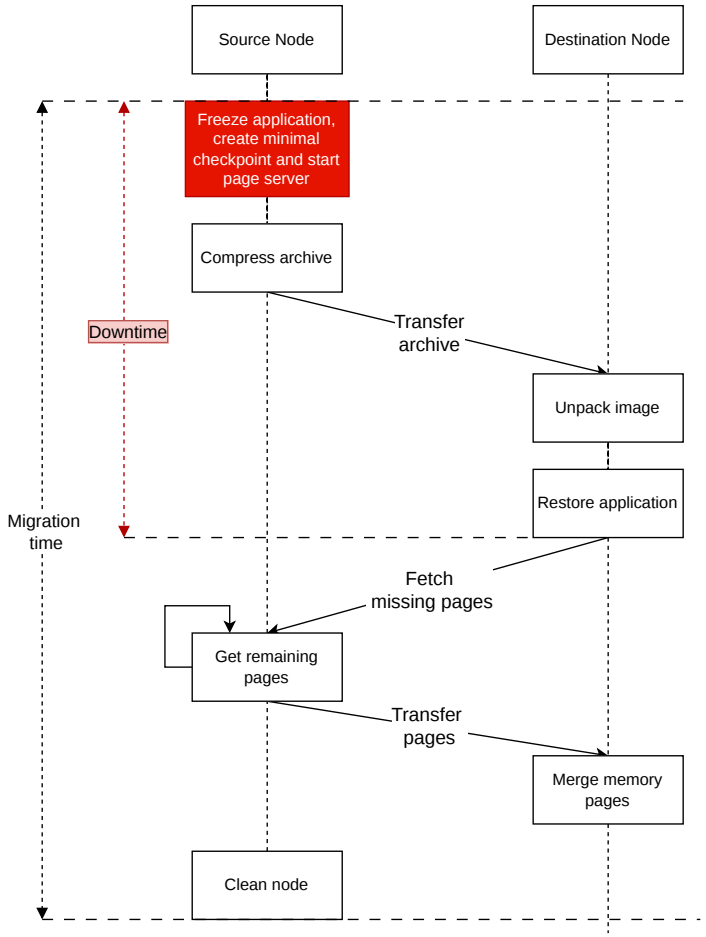


Figure 2.3.: Post-copy migration workflow.

Table 2.1.: Timing boundaries of the native CRIU migration strategies.

Strategy	Work inside downtime	Work outside downtime and trade-off
Cold	Freeze the workload, create the complete checkpoint, transfer and unpack the archive, then restore.	Cleanup happens after a successful restore. The method is simple and robust, but it usually has the longest unavailable window.
Pre-copy	Freeze the workload for the final checkpoint, transfer and merge the final image state, then restore.	Pre-dumps and pre-dump transfers run while the workload continues. Downtime can shrink, but image-chain handling and total transferred data increase.
Post-copy	Create the initial lazy checkpoint, transfer and unpack the initial archive, then restore with lazy-pages support.	Missing memory pages are fetched after the workload resumes. The initial transfer can be smaller, but the restored process depends on the source page server.

2.4. CRIU

CRIU is a Linux checkpoint/restore tool. It saves a running process tree into image files and later recreates that process tree from those images. The captured state can include memory pages, process hierarchy, file descriptors, signal state, namespaces, and other kernel-visible execution state. The project presents checkpoint/restore as a general primitive for several scenarios, including container and application live migration, faster startup of slow services, kernel-upgrade workflows, high-performance computing checkpointing, process snapshots, debugging, and fault-tolerant recovery [8]. Live migration is the scenario used in this work. The official CRIU live migration sequence is dump, copy images, restore, and kill the stopped source process after a successful restore [9].

The native benchmarks use CRIU directly on Linux processes. This keeps the experiment close to the checkpoint/restore mechanism itself. It also exposes the practical responsibilities that a higher-level orchestrator would normally hide. The destination must have compatible binaries and file paths. Output files may need to exist before restore. The source process must stay stopped until the destination has been validated. Network state must also make sense on the destination. In the TCP benchmark, this requires moving a Virtual IP address (VIP) so the restored client can keep the same connection tuple.

CRIU supports iterative migration through pre-dump. A pre-dump captures memory while the process continues to run. Later pre-dumps can reference earlier image directories. The final dump references the last pre-dump and uses memory tracking to capture the remaining changes [10]. This mechanism is the basis for the pre-copy strategy. It can reduce the final freeze time, but it creates an image chain that must be preserved and transferred correctly.

CRIU also supports lazy migration, which is the basis for post-copy migration. With lazy migration, the initial image archive does not contain all memory pages. The source-side dump process remains alive as a page server. The destination starts a lazy-pages daemon, restores the process with lazy-pages support, and fetches missing pages from the source when the restored process accesses them [11, 12]. This lowers the amount of state required before restore. The restored process, however, remains dependent on source-to-destination connectivity until all needed pages have arrived.

These mechanisms make CRIU useful for fine-grained migration experiments, but they are not a complete orchestration system by themselves. The benchmark framework around CRIU prepares image directories, preserves relative references between pre-dumps, selects the archive transfer path, manages page-server ports, captures phase timings, and validates that the workload continues after restore.

2.5. WebAssembly Migration

Wasm is a portable bytecode format with sandboxed execution. It is attractive for the compute continuum because it reduces dependence on a specific operating system or CPU architecture. Tinto et al. [3] present a Wasm live migration mechanism that injects checkpoint and restore procedures into compiled Wasm modules. The approach moves migration logic into the Wasm execution model instead of relying on Linux process checkpointing.

Conceptually, this approach treats migration as a runtime-level transformation. The compiled module is partitioned into regions or code blocks. Static analysis identifies where checkpoint logic can be inserted. The injected logic has two roles. During normal execution it lets the program continue with little interference. When migration is requested, execution proceeds to the next checkpoint block, serializes the runtime-visible state, stores the resume point, and terminates the current execution. On the destination, the same instrumented module is loaded with the serialized state. The restore logic skips blocks that were already executed and resumes from the saved point. The migrated state is therefore Wasm state, not a Linux process tree.

Figure 2.4 summarizes this flow. The red region marks the period in which the running Wasm application is unavailable while checkpoint state is produced, transferred,

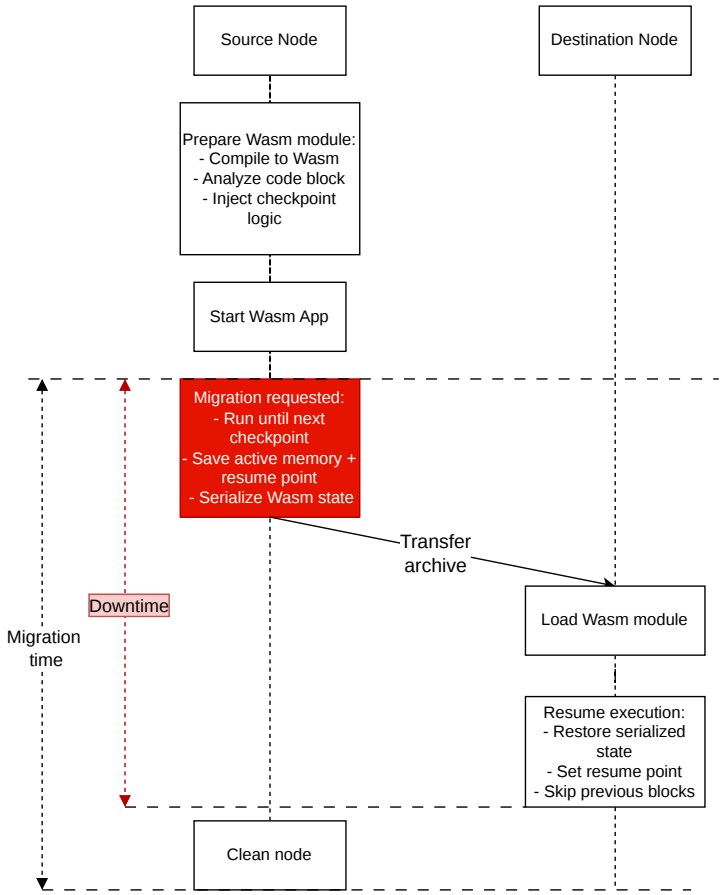


Figure 2.4.: Wasm migration workflow.

and restored. This is not a pre-copy or post-copy optimization. It is a single Wasm checkpoint/restore path.

This distinction matters for the comparison. Native CRIU captures operating-system state such as processes, memory mappings, file descriptors, and, when supported, network state. Wasm migration captures state through the Wasm execution model and the supporting runtime. This can improve portability across heterogeneous nodes, but it depends on migration support in the toolchain and runtime. The experimental setup chapter explains how this existing Wasm migration prototype is wrapped and measured.

3. Experimental Setup

This chapter describes the experimental methodology. The implementation is not a single migration script. It is a benchmark framework around several workloads, several migration mechanisms, controlled network profiles, and a shared plotting pipeline. The design goal is repeatability. A run should identify the workload, the migration strategy, the transfer mode, the network profile, and the recorded metric data from which each plot is generated.

The development process follows three steps. First, each mechanism is tested with a minimal same-host migration. This removes network and orchestration issues from the first proof of concept. Second, the same workload is migrated between two Multipass nodes [5]. This adds file transfer, destination preparation, and validation. Third, the manual flow is wrapped in repeatable orchestrators that write CSV metrics and plot-ready artifacts.

The setup, experiment code, orchestration scripts, plotting utilities, and results are available in the public experiment repository [13].

The dataset combines four workloads, two transfer modes, and seven network profiles: WiFi 6, 5G, LTE, Starlink, TEE Best, TEE Avg, and TEE Worst. The native CRIU workloads are evaluated with cold, pre-copy, and post-copy migration. The Wasm workload is evaluated with its application-aware checkpoint/restore mechanism. The evaluation reports timing distributions for successful migrations and records failed attempts separately, so that reliability and latency remain distinct results.

3.1. Environment

The experiments run on three Multipass Virtual Machines (VMs) that are provisioned by Terraform [4]. The source node is `edge-node-1`. The destination node is `edge-node-2`. The third node, `edge-host-1`, is used either as a relay for archive transfer or as the TCP server for the networked workload. All benchmark commands assume that the nodes have completed the repository bootstrap step and that CRIU, Python [6], Secure Shell (SSH)/Secure Copy Protocol (SCP), and the workload dependencies are available.

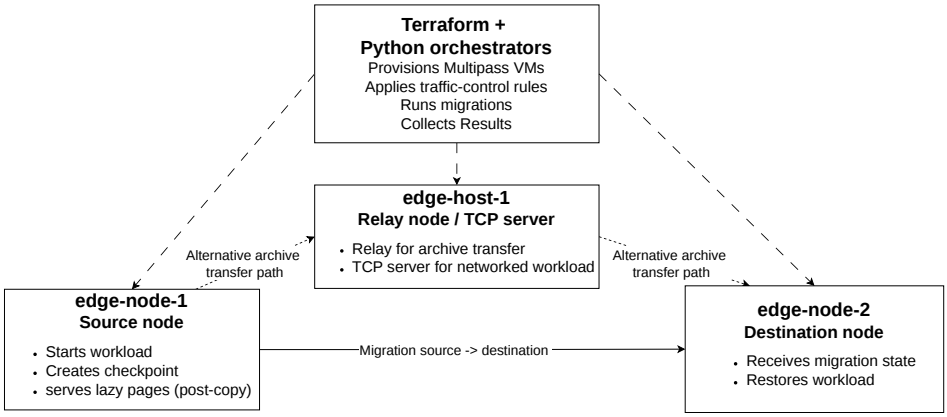


Figure 3.1.: Simplified test environment used by the benchmark framework.

Table 3.1 lists the resource configuration used by the Terraform deployment. Each node uses the Ubuntu 24.04 LTS Multipass image, identified as `noble` in the Terraform configuration. The cloud-init bootstrap installs the runtime and migration dependencies in the same way on all nodes, including CRIU, Python support packages, traffic-control tools, and SSH/SCP support. This keeps the source, destination, and relay comparable; their experimental roles differ, but their base resources do not.

Table 3.1.: Virtual machine characteristics used in the experiments.

Node	Image	vCPU	Memory	Disk
`edge-node-1`	Ubuntu 24.04 LTS (`noble`)	2	2 GB	10 GB
`edge-node-2`	Ubuntu 24.04 LTS (`noble`)	2	2 GB	10 GB
`edge-host-1`	Ubuntu 24.04 LTS (`noble`)	2	2 GB	10 GB

Table 3.2 complements the resource description by showing how the same three-node deployment is used during the experiments. The source and destination nodes are the migration endpoints. The third node is not a larger machine; it is a separate edge host used to model relay-based transfer paths and to host the server side of the TCP workload.

Table 3.2.: Roles of the virtual machines used in the experiments.

Node	Main role	Use in the benchmarks
`edge-node-1`	Source node	Starts the workload, creates checkpoints, and serves lazy pages for post-copy runs.
`edge-node-2`	Destination node	Receives migration state and restores the workload.
`edge-host-1`	Relay node and TCP server	Relays archives in host/relay transfer mode and runs the TCP server in the TCP benchmark.

The framework is intentionally VM-based. Multipass gives each node an independent Linux environment while keeping setup lightweight. Terraform makes the VM creation repeatable. Python orchestrators then execute commands through Multipass, apply traffic-control rules, run migrations, and collect results. This structure is useful for edge experiments because the same source and destination roles can be reused under different network profiles.

3.2. Workloads

The benchmark suite uses four workloads. Three use native Linux process migration with CRIU: a counter process, a Game of Life process, and a TCP client. The fourth uses a Wasm application. Table 3.3 summarizes their purpose.

Table 3.3.: Benchmark workloads.

Workload	Technology	Purpose
Counter	Native CRIU	Minimal stateful process. It isolates fixed checkpoint, archive, transfer, and restore overhead.
Game of Life	Native CRIU	Heap-backed C workload. It introduces a larger memory footprint and visible application progress.
TCP client	Native CRIU	Networked process with an established TCP connection. It tests process migration together with socket identity.
Wasm	Wasm checkpoint/restore	Runtime-level checkpoint/restore path based on an existing injected Wasm file.

3.2.1. Counter

The counter workload is the smallest native benchmark. It is a C program that prints an increasing counter value. The local proof of concept first checkpoints and restores a simple loop on the same host. This validates the basic CRIU sequence: compile the process, start it, identify the Process Identifier (PID), dump it into an image directory, and restore from that directory [9].

The inter-node version keeps the same idea but adds the migration steps that are needed in the benchmark. A launcher compiles the counter on the source node, starts it in the background, redirects output to `/home/ubuntu/counter.out`, and writes PID files used by the orchestrator. The migration script then dumps the process, creates an archive, transfers it, prepares the destination output file, restores the process, and checks that the counter continues after restore.

This workload is useful because the application state is small. If downtime is high here, application memory size is usually not the cause. The more likely causes are fixed system costs, such as CRIU metadata handling, archive creation, SSH setup, transfer setup, destination unpacking, or restore preparation.

3.2.2. Game of Life

The Game of Life workload is a C implementation of Conway's Game of Life [14]. The local proof of concept uses a small board and restores it on the same host. This checks that a non-trivial application with heap state can be checkpointed and restored before the benchmark adds node-to-node transfer.

The benchmark version allocates two grids on the heap. The default configuration uses a `2048x2048` grid. With two `uint32_t` grids, this produces about 32 MiB of heap state. The size is large enough for checkpointing and memory transfer to appear in the measurements, while still fitting comfortably in the Multipass nodes.

The workload has two output modes. Summary mode prints compact heartbeat lines containing generation, live-cell count, and checksum. It is the benchmark mode because it validates progress without making stdout dominate the run. Grid mode draws the board and is used only for small demonstrations. The workload also supports different initial patterns. Random initialization is used in benchmarks. A cannon pattern is available for visual demonstrations.

The restore check watches `/home/ubuntu/gol.out`. A successful run shows that the file continues to update after migration. This workload stresses memory migration more than the counter. It is useful for comparing cold, pre-copy, and post-copy behavior when the process has a larger changing heap.

3.2.3. TCP Client

The TCP benchmark migrates a client process that already has an established socket to a server. The same-host proof of concept validates that CRIU can dump and restore a TCP-connected process when the networking assumptions required by the live migration procedure are satisfied [9]. The multi-node benchmark adds a harder requirement: after moving to a different VM, the restored client must keep the same network identity.

An established TCP connection is identified by the client IP, client port, server IP, and server port. If the client is restored on another node with a different local IP, the server sees a different peer. The restored socket state no longer matches the peer's TCP tuple. Editing CRIU images alone is not enough because the peer and the network still know the old address.

The benchmark solves this with a client VIP address. The client binds to the VIP on `edge-node-1`. During migration, the orchestrator removes the VIP from the source and assigns it to `edge-node-2`. It then sends gratuitous ARP so the server updates its neighbor entry. The server remains on `edge-host-1`. A successful migration keeps the client socket alive without forcing a new server-side connection.

This workload is stricter than the counter and Game of Life workloads. It tests process state, file state, network state, and address continuity together. It is therefore useful for identifying the practical limits of native process migration in edge settings.

3.2.4. WebAssembly

The Wasm benchmark uses Edoardo Tinto's existing injected Wasm migration prototype and upstream command repository [15]. The experiments use a fork of that repository with the local integration changes needed by the benchmark framework [16]. The default module is `3mm_with_cr.wasm` from the `wasm_test_computation` directory. The module already contains the checkpoint and restore logic described in Section 2.5. This work does not change the static analysis, checkpoint placement, injected save/skip code, or runtime migration mechanism. It integrates that mechanism into the same benchmark framework used for the native CRIU workloads.

The integration wraps the prototype's three command binaries: `create_command`, `start_command`, and `migrate_command`. The wrapper deploys the binaries and the selected module to both nodes, starts the source request server, triggers the computation, requests migration, packages the produced memory files, transfers them with the shared direct or host/relay helpers, and starts the destination request server. A run is valid only when the destination log confirms that memory was restored and that execution reaches the end of the call.

The main changes are measurement and repeatability changes. The wrappers add

stricter argument checks. They also accept - for the cgroup argument, which lets the benchmark run outside Oakestra [17]. Finally, they add stable log markers for checkpoint and restore timing. The resulting rows use the shared CSV schema: checkpoint, archive creation, transfer, restore, downtime, archive size, transfer mode, success status, and profile name. Only one Wasm migration mechanism is evaluated. Unlike native CRIU, it does not expose separate cold, pre-copy, or post-copy strategies; checkpointing and resumption are part of the prototype's application-aware migration mechanism. The command-level flow and metric mapping are documented in Appendix A.8.

3.3. Native CRIU Migration Implementation

The native CRIU modules share the same high-level implementation. A setup script cleans the source and destination, starts the workload on `edge-node-1`, and records the PID. A strategy implementation then performs the migration. After restore, a validation step checks that the workload continues correctly on `edge-node-2`. Each run appends a row to `migration_metrics.csv` and writes raw logs under `metrics/run_logs/`.

Cold migration creates a complete checkpoint while the process is stopped. The benchmark follows the dump, copy, restore, and source-cleanup sequence described by the CRIU live migration documentation [9]. It keeps the dumped process stopped on the source until validation completes. The image directory is compressed into an archive, copied to the destination, unpacked, and restored. The measured downtime includes checkpoint creation, archive transfer, and restore.

Pre-copy migration adds preparation work before the final freeze. The orchestrator performs one or more pre-dumps while the workload keeps running. Each pre-dump creates an image directory. Later image directories reference earlier ones through CRIU's previous-image mechanism [10]. The destination receives the pre-dump state before the final freeze. The final dump captures the remaining delta, transfers that final state, and restores from the complete image chain. In the plotted downtime, pre-dump time is excluded because the workload is still running during that phase.

Post-copy migration uses CRIU lazy pages. The source starts a lazy dump and remains available as a page server. The initial image archive is transferred to the destination. The destination starts the lazy-pages daemon and restores the process with lazy-pages support. Missing pages are fetched from the source when the restored process accesses them [11, 12]. The initial unavailable window can be shorter, but the restored process still depends on the source page server.

The same strategy pattern is reused across the counter, Game of Life, and TCP modules. The TCP module adds server management and VIP handoff. The Game of Life module adds workload-specific launch options. The counter module stays minimal

and serves as the baseline.

3.4. Transfer Modes

The framework separates the migration coordinator from the archive data path. The coordinator starts commands, applies network conditions, and collects logs. The archive path decides how checkpoint images move between nodes. This is an experimental design choice. CRIU examples commonly show a direct `scp` copy of image directories between source and destination, but edge deployments may not always allow direct source-to-destination transfer [9, 10].

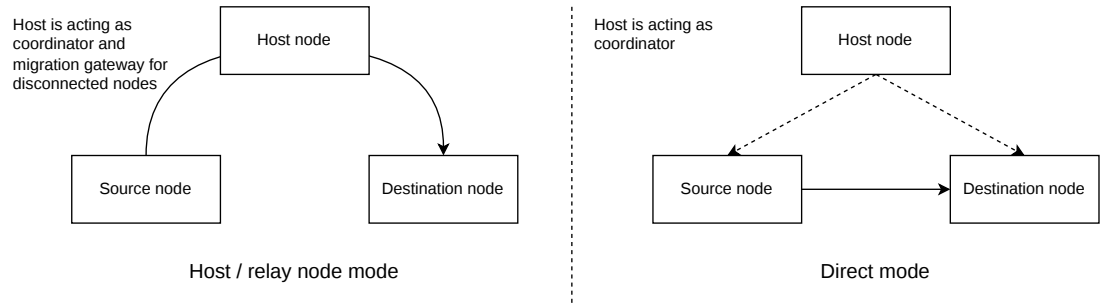


Figure 3.2.: Archive transfer paths implemented in the benchmark framework.

Two modes are implemented. In direct mode, the source VM sends the archive directly to the destination VM with SSH/SCP. The host still coordinates the experiment, but it is not part of the archive data path. This mode represents the case where both migration endpoints can reach each other.

In host or relay mode, the archive is staged through an intermediate node. If `edge-host-1` is configured as relay, the orchestrator sets up SSH trust and copies the archive source-to-relay and relay-to-destination. If no relay is used, the physical host performs the same staging role through Multipass file transfer. This mode represents disconnected or restricted topologies where source and destination cannot exchange archives directly.

For post-copy, the transfer mode applies only to the initial image archive. Lazy memory pages are not relayed by `edge-host-1`. They move over the CRIU page-server path between source and destination after restore [11, 12]. This distinction matters because the archive transfer can be shaped or staged differently from the post-copy page traffic.

The transfer helpers install SSH keys before copying an archive. Connectivity is validated by the actual SCP transfer, which has retries and records the transfer result.

A separate pre-transfer `ssh echo` probe is not used in the migration window, because high-loss profiles can make that probe fail independently from the archive transfer itself.

3.5. Benchmark Framework

The repeatable benchmark layer is built around Python orchestrators. Each workload has a single-run benchmark command and a repeat runner. The repeat runner executes the selected migration strategies, transfer modes, and repetitions. It can continue after failed runs so that one transient failure does not discard a whole batch.

The top-level runner, `run_all.py`, executes full experiment sweeps. It uses two JavaScript Object Notation (JSON) inputs. `network_profiles.json` defines link conditions. `benchmarks.json` defines suite commands. For each profile, the runner applies traffic-control rules, runs the selected benchmark suites, and clears the rules at the end. The benchmark registry contains four suites: Counter Migration, TCP Client Migration, Game of Life Migration, and Wasm Migration.

Large experiment batches can accumulate state in several places: CRIU processes, application processes, node exporter snapshots, plotting memory, and Multipass VM state. To reduce this effect, `run_all.py` supports chunked execution. The option `--run-chunk-size` splits a requested run count into smaller groups. The option `--restart-between-run-chunks` restarts the source and destination VMs between groups and reapplies the active network profile before continuing. The relay VM is not restarted by default because it is mostly a relay/server role and Multipass can occasionally leave it stuck in a restarting state during long unattended batches.

The runner also supports deferred plotting. With `--defer-suite-plots`, each suite writes run identifiers during execution but skips plot rendering. After the suite finishes, `run_all.py` combines the run identifiers and generates one plot directory for the corresponding benchmark and profile. This is necessary because the CSV files are append-only. Run-id filtering prevents old rows from being mixed into new figures.

3.6. Network Profiles

Network profiles are stored in `network_profiles.json`. Each profile defines bandwidth, latency, and packet loss used during the emulation. The runner applies these values with Linux traffic control and netem. Rules are applied to VM-to-VM traffic. Physical host-to-VM control traffic remains mostly unshaped so that `multipass exec`, monitoring, and recovery commands remain usable.

Table 3.4.: Network profiles used by the benchmark runner.

Profile	Bandwidth (Mbit/s)	Latency (ms)	Packet loss (%)
`1_WiFi_6`	800	5	0.00001
`2_5G`	400	10	0.01
`3_LTE`	50	40	0.08
`4_Starlink`	100	45	1.0
`5_TEE_Best`	4	200	5.0
`6_TEE_Avg`	2	400	10.0
`7_TEE_Worst`	0.65	3000	15.0

The profiles serve two purposes. First, they make the experiments repeatable. Second, they make the link assumptions explicit. The WiFi, 5G, and LTE values are derived from representative network values at the edge [18]. The Starlink-like profile uses representative Starlink service values [19]. The three tactical edge profiles represent progressively more constrained tactical-network links based on the emulated tactical-network conditions reported by Kudla et al. [20].

3.7. Plotting and Data Processing

Each benchmark writes raw rows to a module-specific `migration_metrics.csv`. The schema is shared across modules where possible. Important fields include run identifier, technology, migration method, transfer mode, checkpoint time, archive size, transfer time, restore time, downtime, success status, timestamp, and profile name. Newer rows also include transfer subphase columns: archive creation, setup, copy leg one, copy leg two (host mode), cleanup, and unpacking.

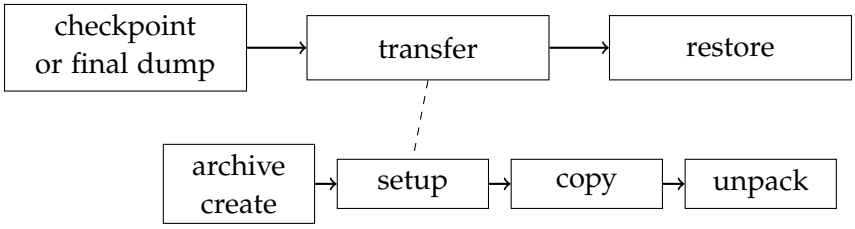


Figure 3.3.: Timing model used by the plots. Downtime is decomposed into checkpoint, transfer, and restore. Transfer analysis further separates archive creation, setup, copy, and unpacking costs.

The primary metric is downtime. It is computed as checkpoint time plus transfer

time plus restore time. For pre-copy rows, checkpoint time refers to the final dump only. Earlier pre-dumps are recorded as preparation work because the workload continues to run during them. For post-copy rows, downtime ends after restore, even though lazy page fetching may continue afterward.

The generated plots use a setup-adjusted timing convention. `transfer_setup_ms` is subtracted from plotted transfer and downtime values. The raw CSV remains unchanged. This convention treats SSH trust setup, IP lookup, source-file validation, and similar setup as deployment overhead that would normally be prepared before a production migration.

The downtime comparison plot shows the distribution of downtime for each strategy and transfer mode. The phase breakdown plot shows mean checkpoint, setup-adjusted transfer, and restore time, with standard-deviation annotations for each phase. The transfer analysis plot relates archive size to transfer time. The transfer phase breakdown plot decomposes archive creation, copy legs, cleanup, and destination unpacking. Direct mode has one copy leg. Host/relay mode has two copy legs.

Node exporter snapshots are optional. When enabled, the runner stores before and after snapshots from the source and destination nodes. These snapshots are summarized into CPU, memory, and disk metrics. They help show whether a migration strategy shifts overhead to a specific node or resource.

4. Evaluation

This chapter evaluates the migration measurements collected with the benchmark framework. The analysis covers the four workloads introduced in Chapter 3: Counter, TCP client, Game of Life, and Wasm. It also covers the full network profile set used by the experiments: WiFi 6, 5G, LTE, Starlink, TEE Best, TEE Avg, and TEE Worst.

All timing plots use successful runs only. A failed run is not a valid latency sample because it does not represent a completed migration. Failure is still part of the result, so success counts are tracked separately and shown in every plot annotation. This is important for post-copy. Several Game of Life post-copy runs restore the process but fail validation because the output does not continue to update. Those runs are not used for downtime medians, but their failures are discussed as evidence about the limits of lazy page migration under constrained links.

4.1. Experiment Overview

The evaluation varies the workload, migration mechanism, and archive transfer mode. Counter is the smallest native CRIU process. The TCP client adds network connection continuity. Game of Life has a larger heap and exposes memory migration effects. Wasm uses the `3mm_with_cr.wasm` matrix multiplication module.

The native workloads use cold migration, pre-copy migration, and post-copy migration. Wasm provides one application-aware checkpoint/restore path. Direct transfer copies the archive from source to destination, while host/relay transfer stages it through `edge-host-1`. Host/relay mode is useful when nodes cannot communicate directly, but it adds a second serialized copy leg.

Table 4.1 summarizes the interpretation used throughout the chapter. The point is not to rank the mechanisms globally. Each mechanism optimizes a different part of the migration problem and therefore exposes a different failure mode.

Table 4.1.: Migration mechanism trade-offs used to interpret the results.

Mechanism	Main advantage	Main cost or assumption	Best suited case
Cold migration	Simple boundary: dump, transfer, restore. No dependency on source after restore.	Full archive transfer is inside downtime, so large state and slow links directly increase service interruption.	Small state, reliable transfer, or cases where simplicity matters more than downtime.
Pre-copy	Moves part of the memory state before the final freeze, reducing the final downtime window.	More total migration work, longer preparation time, and repeated disk/archive activity. Effect depends on how much memory changes between iterations.	Memory-heavy workloads when a longer migration process is acceptable to reduce final downtime.
Post-copy	Very small initial archive and potentially the shortest measured downtime.	Restored execution depends on lazy page delivery from the source; poor links can stall progress after restore.	Workloads with small or predictable working sets, or links reliable enough for lazy page faults.
Wasm checkpointing	Captures application/runtime state rather than a full Linux process image, reducing transferred state.	Requires application/runtime support and is not a transparent native-process mechanism.	Portable or application-aware services where checkpoint points can be engineered.

Success is evaluated at the workload level, not only at the command-exit level. Table 4.2 summarizes the validation condition used before a row is treated as a successful migration. A run can therefore produce partial timing data and still be marked as failed if the restored workload does not make the expected progress.

Table 4.2.: Application-level success criteria used by the benchmark scripts.

Workload	Validation signal	Success condition
Counter	Last value written to `/home/ubuntu/counter.out`.	After restore, the script waits briefly and reads the last counter value on the destination. The run succeeds when the value is numeric and at least the expected next value after the frozen source state. If the exact numeric check is unavailable, the fallback is restored-process liveness, with a note in the CSV.
TCP client	Destination socket state and server connection count.	After restore, the script checks that the destination still has an established TCP socket to the server and that the server did not accept a new connection. A reconnect is counted as failure because the original socket was not preserved.
Game of Life	Modification time of `/home/ubuntu/gol.out`.	After restore, the script samples the output file modification time, waits, and samples it again. The run succeeds only if the file is updated on the destination, which indicates that the simulation is still running.
Wasm	Runtime log messages from the injected checkpoint/restore runtime.	The run succeeds after the destination runtime reports that memory restore completed and the application reaches the end-of-call marker. Transfer and restore errors prevent the success flag from being set.

Table 4.3.: Evaluation coverage. Counts show successful runs over attempted runs in the final plot sets.

Profile	Counter	TCP client	Game of Life	Wasm	Status
WiFi 6	239/240	237/240	240/240	80/80	Complete
5G	240/240	239/240	239/240	80/80	Complete
LTE	239/240	240/240	159/240	80/80	Complete
Starlink	240/240	236/240	159/240	80/80	Complete
TEE Best	239/240	187/240	160/240	80/80	Complete
TEE Avg	238/240	200/240	158/240	79/80	Complete
TEE Worst	178/180	110/180	no complete plot set	60/60	Partial; Game of Life transfer failure

For Counter, the TCP client, and Game of Life, a complete profile contains 40 cold, 40 pre-copy, and 40 post-copy runs in host/relay mode, plus the same 120 runs in direct mode, for 240 attempts. The Wasm column contains one migration mechanism measured 40 times in host/relay mode and 40 times in direct mode, for 80 attempts. For TEE Worst, the planned run count was reduced to 30 per transfer as the execution time would otherwise become too long, so the complete denominators become 180 for the CRIU workloads and 60 for Wasm. Run identifiers that were launched but did not produce a metrics row are counted as unsuccessful or missing attempts.

Table 4.3 shows the first cross-workload result: the same network profile does not affect every workload in the same way. Counter loses only a small number of runs across the full profile ladder. Wasm loses one run across the completed profiles and transfers a median archive of about 2 KiB. The TCP client is stable on the access-network profiles, but its success rate decreases under tactical edge conditions because preserving an established socket is more sensitive than restoring a local counter. Game of Life is the least stable under post-copy because its larger heap makes the restored process depend on lazy page delivery for observable progress.

Table 4.4 gives a compact latency view of the same result. It uses direct mode so the table compares one-hop transfers; host/relay results are kept in the appendix and follow the same trends with larger transfer costs. The table shows the order of magnitude of each workload before the chapter focuses on representative plots.

Table 4.4.: Direct-mode setup-excluded downtime medians for all profiles. Cells show median downtime in milliseconds and successful runs over attempted runs.

Workload	Profile	Cold	Pre-copy	Post-copy	Wasm
Counter	WiFi 6	2 500 ms (40/40)	2 500 ms (39/40)	3 000 ms (40/40)	–
Counter	5G	2 700 ms (40/40)	2 700 ms (40/40)	3 200 ms (40/40)	–
Counter	LTE	3 900 ms (39/40)	3 900 ms (40/40)	4 500 ms (40/40)	–
Counter	Starlink	4 400 ms (40/40)	4 300 ms (40/40)	4 900 ms (40/40)	–
Counter	TEE Best	12 700 ms (40/40)	13 100 ms (40/40)	14 300 ms (40/40)	–
Counter	TEE Avg	28 800 ms (40/40)	28 900 ms (39/40)	30 900 ms (40/40)	–
Counter	TEE Worst	236 900 ms (29/30)	262 300 ms (30/30)	244 200 ms (30/30)	–
TCP client	WiFi 6	2 200 ms (40/40)	2 200 ms (39/40)	2 700 ms (38/40)	–
TCP client	5G	2 400 ms (40/40)	2 400 ms (39/40)	2 900 ms (40/40)	–
TCP client	LTE	3 700 ms (40/40)	3 600 ms (40/40)	4 300 ms (40/40)	–
TCP client	Starlink	4 000 ms (40/40)	3 900 ms (39/40)	4 600 ms (40/40)	–
TCP client	TEE Best	12 900 ms (28/40)	13 200 ms (37/40)	14 100 ms (34/40)	–
TCP client	TEE Avg	29 800 ms (38/40)	29 000 ms (28/40)	30 900 ms (32/40)	–
TCP client	TEE Worst	258 900 ms (27/30)	242 200 ms (17/30)	328 200 ms (14/30)	–
Game of Life	WiFi 6	4 000 ms (40/40)	3 400 ms (40/40)	3 000 ms (40/40)	–
Game of Life	5G	4 400 ms (40/40)	3 700 ms (40/40)	3 200 ms (40/40)	–
Game of Life	LTE	7 700 ms (40/40)	6 000 ms (39/40)	– (0/40)	–
Game of Life	Starlink	51 400 ms (40/40)	18 000 ms (40/40)	– (0/40)	–
Game of Life	TEE Best	472 100 ms (40/40)	107 100 ms (40/40)	– (0/40)	–
Game of Life	TEE Avg	1 198 600 ms (40/40)	218 500 ms (40/40)	– (0/40)	–
Game of Life	TEE Worst	–	–	–	–
Wasm	WiFi 6	–	–	–	3 500 ms (40/40)
Wasm	5G	–	–	–	3 700 ms (40/40)
Wasm	LTE	–	–	–	4 800 ms (40/40)
Wasm	Starlink	–	–	–	5 100 ms (40/40)
Wasm	TEE Best	–	–	–	13 400 ms (40/40)
Wasm	TEE Avg	–	–	–	27 500 ms (40/40)
Wasm	TEE Worst	–	–	–	201 700 ms (30/30)

Counter is the fixed-overhead baseline. Its direct-mode medians remain close across cold, pre-copy, and post-copy because the archive is tiny and memory optimization has little room to help. Even under TEE Worst, the direct-mode Counter runs remain successful in the final plot set because the state being transferred is small enough that the migration path can survive it.

The TCP client has a similar memory footprint, but it adds network-state correctness. Its direct-mode medians are close to Counter from WiFi 6 to Starlink. Under the TEE profiles, the latency remains in the same order of magnitude as Counter, but success drops sharply. Under TEE Worst, direct post-copy is successful in only 14 of the 30 attempts, maintaining the established connection without the need to reconnect.

Game of Life is the memory-pressure case. It is the workload where pre-copy becomes visibly valuable, especially under the constrained networks. It is also where post-copy shows both its strength and its risk. When the lazy-pages path can keep up, post-copy

reduces the initial checkpoint archive and the downtime substantially because the destination resumes from a small image and fetches missing pages later. The drawback is that those later page faults still depend on the page-server connection. With a larger heap, lazy-page delivery becomes part of application progress, so post-copy can return from restore and still fail the workload-level validation if the connection is too slow or unreliable.

Wasm provides the application-aware case. It checkpoints the `3mm_with_cr.wasm` runtime state rather than a full Linux process image. Although this is not a same-workload comparison with Game of Life, the measurements isolate the behavior of targeted application-state capture. Its downtime also grows with the network profile, but it remains successful through TEE Worst.

Table 4.5 isolates the Game of Life post-copy result. Game of Life post-copy succeeds completely on WiFi 6 and 5G, but it collapses after that: LTE and Starlink have no successful post-copy runs, TEE Best has only one successful host-mode run, and TEE Avg again has none. Since Game of Life did not complete under TEE Worst for any CRIU strategy, that profile is excluded from the timing comparison and is discussed only as a failed migration case.

Table 4.5.: Game of Life post-copy success by profile and transfer mode.

Profile	Direct	Host/relay	Interpretation
WiFi 6	40/40	40/40	Reliable lazy-page path.
5G	40/40	40/40	Reliable lazy-page path.
LTE	0/40	0/40	Restored process did not make observable progress.
Starlink	0/40	0/40	Restored process did not make observable progress.
TEE Best	0/40	1/40	Near-total failure.
TEE Avg	0/40	0/40	No successful post-copy migrations.
TEE Worst	–	–	Bulk archive transfer failed before a complete plot set was produced.

The raw CSV files preserve all rows. The generated timing plots remove failed rows from distributions and add success information separately. This separation is intentional, since timing a failed migration would make the latency distribution look better or worse for the wrong reason. Success rate and successful-run latency answer different questions.

4.1.1. Transfer Modes and Timing Convention

The transfer mode changes the data path, not the checkpoint algorithm. In direct mode, the source transfers the archive to the destination. In host/relay mode, the source first transfers the archive to `edge-host-1`; the relay then transfers it to the destination. This is closer to a migration gateway model, but it also doubles the number of copy legs.

The timing convention follows the implementation described in Section 3.7. The raw CSV rows include setup work. The plotted downtime, transfer, and migration-time values exclude transfer setup. Setup includes tasks such as IP lookup, source-file checks, and SSH key installation. These operations are recorded for reproducibility. A production-ready migration system, however, should prepare them before the service is asked to move. Archive creation, copy legs, cleanup, unpacking, checkpointing, and restore remain in the plotted migration cost. With this convention, the downtime comparison stays consistent with the phase-breakdown plots: plotted downtime is the sum of checkpoint or final dump, setup-excluded transfer, and restore.

This convention matters most in the worst profiles. Under TEE Worst, even an SSH handshake can fail or take a long time. The benchmark therefore treats the actual SCP operation as the transfer validation step. That avoids counting a fragile setup probe as migration behavior, while still allowing real transfer failures to appear in the results.

The native CRIU results measure process migration below the container layer. A container migration would add runtime metadata, namespaces, filesystem state, image layers, volumes, and orchestration decisions. The results therefore explain the checkpoint/transfer/restore core, not the full overhead of a Kubernetes or Podman migration.

4.2. Downtime Results

Downtime is the period in which the workload cannot make progress. In cold migration, downtime includes the full checkpoint, archive transfer, and restore. In pre-copy, earlier pre-dumps run while the workload continues, so downtime starts at the final dump. In post-copy, downtime ends after restore, although missing pages may still be fetched afterward.

The generated Cumulative Distribution Functions (CDFs) use the same success-only convention for all four workloads. The appendix includes the Counter, TCP client, Game of Life, and Wasm CDFs for both transfer modes. The main text shows direct-mode CDFs for Game of Life and Wasm. Game of Life is the native workload where the strategy differences are most visible. Wasm is shown next to it because it provides the contrasting application-aware migration mechanism. Figures 4.1 and 4.2 show the Game of Life downtime CDFs. Figures 4.3 and 4.4 show the corresponding Wasm

CDFs. In both cases, the access-network figure compares WiFi 6, 5G, and LTE. Starlink is placed in the tactical-edge figure, where it is compared with the three TEE profiles. This keeps the access-network plots readable while preserving Starlink as the bridge into the disadvantaged-link comparison.

The CDF x-axes are linear and use millisecond tick labels. Their distributions shift right as the network becomes more constrained, but timing alone is not enough. The run-status notes show where fewer successful runs are available, especially for Game of Life post-copy. Because each CDF is split by transfer mode, the failed-run counts in a figure refer to that specific transfer path. The coverage tables aggregate host and direct mode.

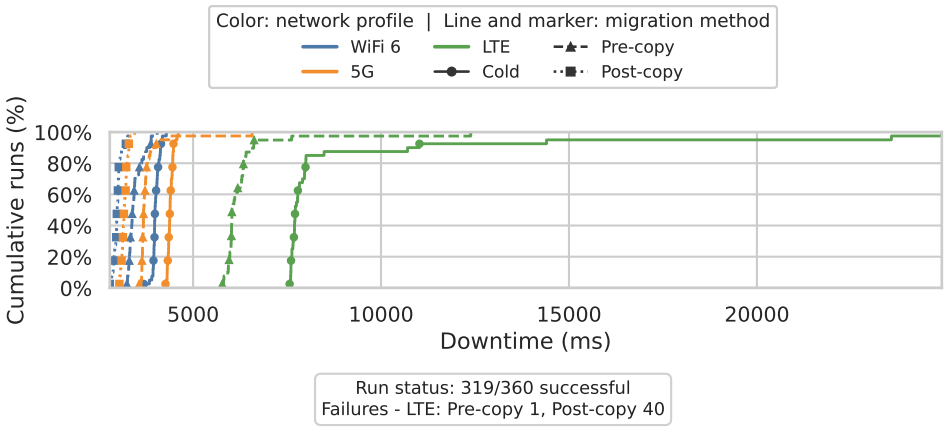


Figure 4.1.: Game of Life downtime CDF for direct transfer mode across WiFi 6, 5G, and LTE. The x-axis is linear milliseconds.

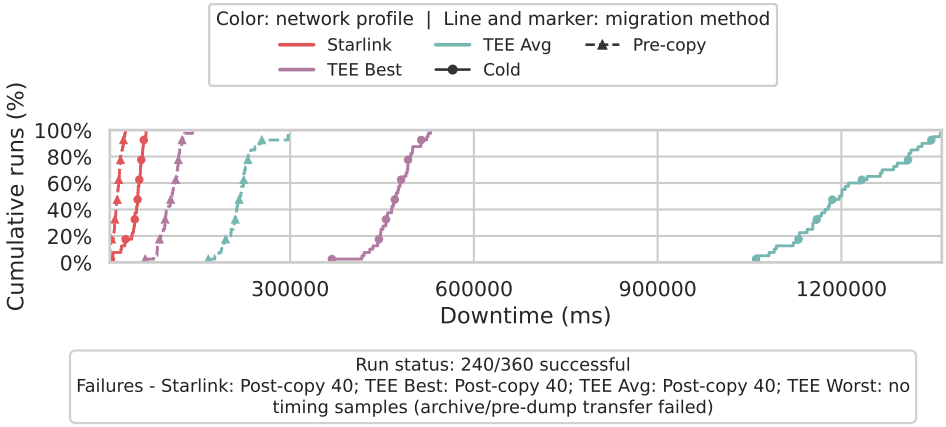


Figure 4.2.: Game of Life downtime CDF for direct transfer mode across Starlink and the tactical edge profiles. The x-axis is linear milliseconds; TEE Worst has no successful Game of Life timing samples and is reported in the run-status note rather than as a curve.

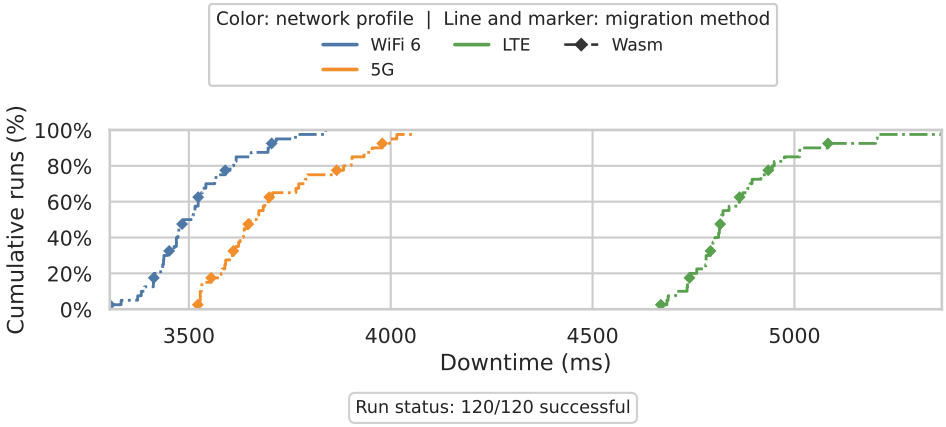


Figure 4.3.: Wasm downtime CDF for direct transfer mode across WiFi 6, 5G, and LTE. The x-axis is linear milliseconds.

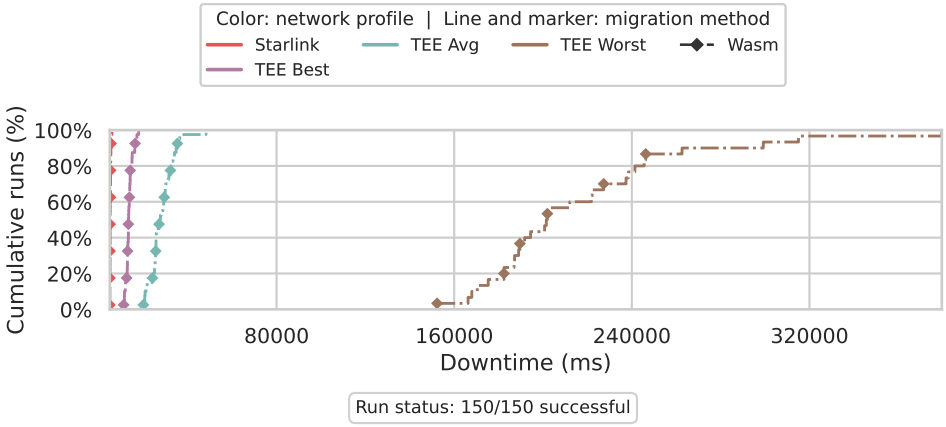


Figure 4.4.: Wasm downtime CDF for direct transfer mode across Starlink and the tactical edge profiles. The x-axis is linear milliseconds.

Figure 4.5 shows a representative per-profile Game of Life downtime plot. Counter and the TCP client are important for coverage, but their state is so small that fixed orchestration and transfer overhead can hide differences between migration strategies. Game of Life has a larger heap, so the benefit and risk of pre-copy and post-copy become visible.

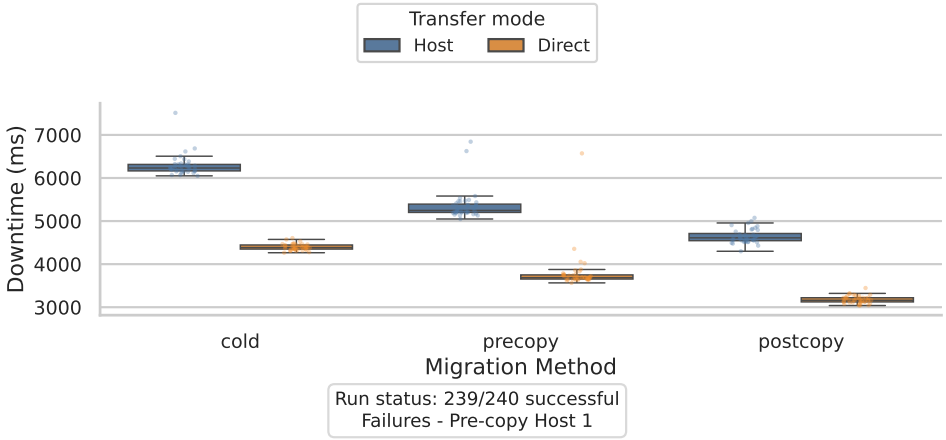


Figure 4.5.: Representative Game of Life downtime comparison under the 5G profile.

Figure 4.6 shows the Wasm result for the same 5G profile, so it can be compared with the native CRIU Game of Life result in Figure 4.5.

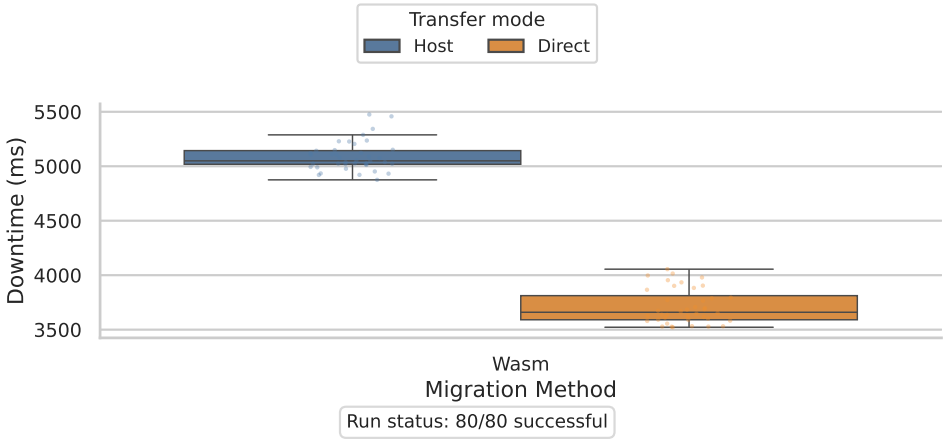


Figure 4.6.: Representative Wasm downtime comparison under the 5G profile.

The first completed profiles show the expected baseline behavior. WiFi 6 and 5G keep transfer costs small enough that all three native strategies can complete for all native workloads. For Counter and the TCP client, cold and pre-copy are nearly indistinguishable in median direct-mode downtime because the transferred state is small. For Game of Life, the larger heap already makes the strategy boundary visible: direct-mode median downtime on WiFi 6 is 4 000 ms for cold migration, 3 400 ms for pre-copy, and 3 000 ms for post-copy. On 5G, the same medians are 4 400 ms, 3 700 ms, and 3 200 ms. This ranking is not absolute. On fast profiles the strategies are close enough that fixed orchestration and restore effects still matter. The behavior matches the trade-offs described by Soussi et al. [2]: cold, pre-copy, and post-copy do not form a fixed ranking. They move work to different places.

As the network becomes worse, downtime is increasingly determined by transfer. LTE already separates the strategies more strongly. For Game of Life in direct mode, LTE cold migration has a median downtime of 7 700 ms, while pre-copy reduces it to 6 000 ms, a reduction of about 22%. Starlink makes the reduction much larger: direct cold migration has a median downtime of 51 400 ms, while direct pre-copy has a median downtime of 18 000 ms, about 65% lower. Under TEE Best, the same comparison is 472 100 ms versus 107 100 ms, a reduction of about 77%, and under TEE Avg it is 1 198 600 ms versus 218 500 ms, a reduction of about 82%. Host/relay mode follows the same pattern with larger values because it serializes two copy legs: under TEE Avg, host cold migration reaches a median downtime of 2 436 000 ms, while host pre-copy reduces the final downtime window to 380 000 ms, about 84% lower.

The price of pre-copy is total migration time. For Game of Life, pre-copy under TEE Avg direct mode reduces median downtime by about 82%, from 1 198 600 ms

to 218 500 ms. The median setup-excluded migration time, however, increases by about 63%, from 1 219 200 ms for cold migration to 1 988 700 ms for pre-copy. In host/relay mode, the same profile reduces downtime by about 84%, from 2 436 000 ms to 380 000 ms, while migration time increases by about 53%, from 2 444 100 ms for cold migration to 3 735 800 ms for pre-copy. This is the central pre-copy trade-off: it can protect availability by moving most memory transfer before the final freeze, but the migration as a whole occupies the system for longer.

Under TEE Worst, Game of Life did not reach a stable timing-distribution stage. The archive for cold migration and the first pre-dump archive for pre-copy repeatedly failed during SCP transfer. The cold Game of Life archive is approximately 12.1 MiB. On the tested TEE Worst profile, that size is large enough, and the configured link slow and lossy enough, that the current SCP-based migration path could not reliably deliver it before the transport failed. This is why no TEE Worst Game of Life curve appears in the CDF.

Post-copy has the lowest theoretical initial archive cost and can minimize measured downtime. It also has the strongest dependency on source-to-destination page delivery after restore. On WiFi 6 and 5G, the successful Game of Life post-copy archive is only about 25 KiB, which explains the low initial downtime. The restored process still depends on the source page server. If the process touches missing pages and the page-server path is slow, lossy, or unavailable, the process may not make observable progress. This is why Game of Life post-copy fails validation in the slower profiles while Counter can still complete. Counter touches little memory and has a tiny working set. Game of Life touches a much larger heap, so lazy page fetching becomes part of the application's progress path.

Wasm is useful here because it gives a different answer to the same edge question: reduce the state before it enters the network. The 5G Wasm plot shows one mechanism rather than three strategies, but its medians sit near the small native workloads rather than near memory-heavy Game of Life. Under TEE Worst, Wasm still completes with 60/60 successful runs, while Game of Life cannot produce a complete CRIU plot set. This contrast does not claim that Wasm is universally faster. It shows the reliability advantage of application-aware state capture when the runtime can save only what must be resumed.

4.3. Phase Breakdown

The phase breakdown separates downtime into checkpoint, transfer, and restore. Figures 4.7 and 4.8 show the same representative 5G profile for Game of Life and Wasm. For Game of Life, the main visual feature is transfer. Once the workload is large enough,

checkpoint and restore remain measurable, but the copy path becomes the main driver when the archive must cross the link.

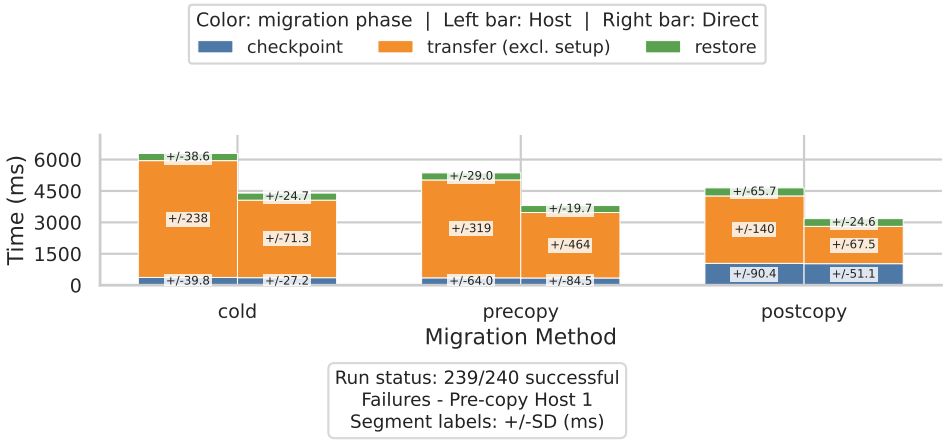


Figure 4.7.: Game of Life phase breakdown under the 5G profile.

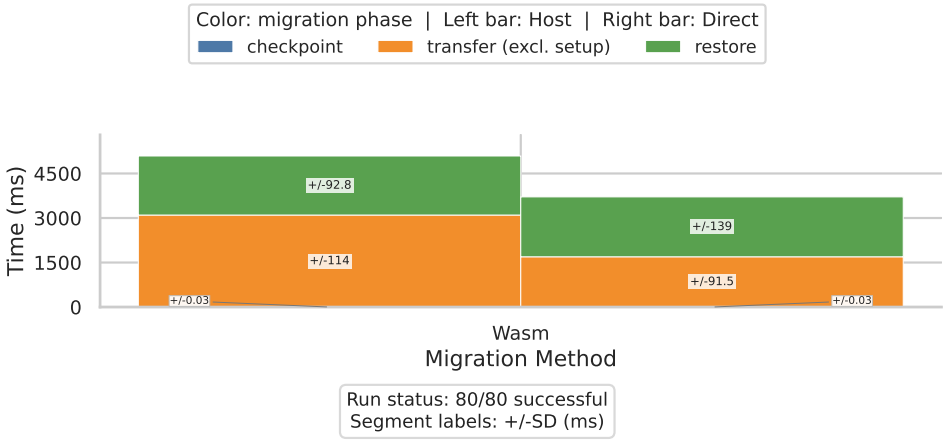


Figure 4.8.: Wasm phase breakdown under the 5G profile.

Cold migration places all three phases in the downtime window. The method is simple and robust, but it is vulnerable to slow links. When the archive is large, the transfer phase dominates the unavailable window.

Pre-copy changes the timing boundary. The pre-dump iterations run before downtime while the source process continues. The final dump is then smaller because much of the memory state has already been transferred. Measured downtime can therefore be much lower than cold migration in Starlink and tactical edge profiles. The cost does

not disappear; it moves into migration preparation. This is why migration time and downtime must be interpreted separately.

Post-copy changes the boundary in the opposite direction. It transfers a small initial archive and restores early. Missing pages are fetched after restore. In the measurements, the restore phase counts only until `criu restore -lazy-pages` returns. Lazy-page serving after the process resumes is not added to downtime, although it can still affect whether the application makes progress. This distinction matters because post-copy can have a short measured downtime and still be unusable if the process immediately stalls on missing pages.

The Game of Life validation makes this distinction explicit. After a post-copy restore, the benchmark keeps the lazy-pages daemon alive for up to 15 s, waits 3 s more, records the modification time of `gol.out`, waits another 2 s, and requires the file to be updated. Since the workload writes once per second, this is a prompt-progress test rather than an immediate one-second test. A much longer wait might convert some failed runs into eventual completions. That would answer a different question: whether the process eventually recovers after a long page-fault stall. This thesis uses the stricter criterion because service migration should restore observable application progress within a bounded time. Under LTE, Starlink, and the TEE profiles, the failed Game of Life post-copy rows are dominated by `gol_out_not_updating` and process-validation failures, not by the absence of a restore command. They show that post-copy's source dependency is a practical limitation under disadvantaged links.

The small workloads provide useful contrast. Counter is mostly fixed overhead plus transfer. The TCP client adds socket continuity, but its memory footprint is still small. For both workloads, the phase breakdown is less about memory strategy and more about whether the archive path and restore validation survive the network profile. Game of Life is the native workload where the phase decomposition is most informative because the heap is large enough for pre-copy and post-copy to change the behavior of the application.

Wasm makes the checkpoint part of the phase breakdown almost disappear at millisecond scale. Figure 4.9 shows the same checkpoint values in microseconds. The median 5G checkpoint across both transfer modes is about 160 μs. The visible Wasm downtime is therefore dominated by transfer and destination-side restore, not by state capture. This is the practical effect of inserting checkpoint logic at application/runtime level: the benchmark does not ask a general-purpose kernel checkpoint tool to discover and dump the whole process state.

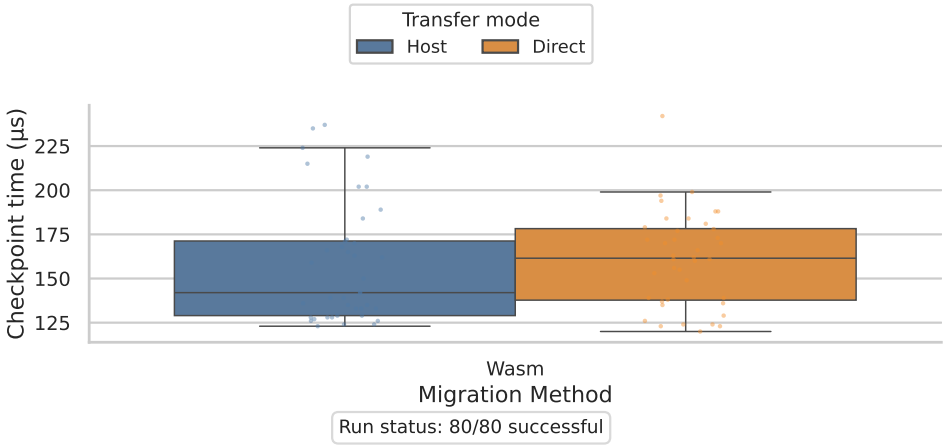


Figure 4.9.: Wasm checkpoint precision under the 5G profile. Values are shown in microseconds because the checkpoint phase is below millisecond scale.

4.4. Transfer Analysis and Breakdown

Transfer analysis explains most of the profile-to-profile change. Figures 4.10 and 4.11 relate archive size to transfer time for the representative 5G profile. The relationship is not perfectly linear because the plotted transfer window includes more than network payload delivery. For CRIU, it includes archive creation and destination unpacking. For Wasm, it includes archive creation, while destination seeding remains inside restore. Still, the main trend is clear: larger archives cost more, and the cost grows when the link is constrained. In the Game of Life plot, post-copy points lie near the origin because their initial image is very small compared with cold and pre-copy archives; these points are successful tiny transfers, not failed zero-byte samples.

Table 4.6 summarizes the state sizes behind that behavior. Direct mode is shown because the archive content is produced by the migration mechanism; host/relay mode changes how the archive travels, not what is inside it. The values are ranges of per-profile medians over successful runs. For pre-copy, the table reports the final archive transferred inside the downtime window. Earlier pre-dump archives are transferred before downtime and are not included in the shown value. For post-copy, the table reports only the initial image archive; pages served later by lazy paging are not part of this archive size. The wide Game of Life pre-copy range is itself a result: CRIU tracks memory changes across pre-dumps, so the final archive shrinks when few pages are dirtied after the previous iteration and grows when the workload keeps changing memory.

Table 4.6.: Direct-mode median archive sizes by workload and migration mechanism. Values are ranges of per-profile medians over successful runs.

Workload	Cold archive	Pre-copy final archive	Post-copy initial archive	Wasm state archive
Counter	24.6–24.7 KiB	24.8–25.0 KiB	22.0 KiB	–
TCP client	26.7–26.8 KiB	26.9 KiB	24.1–24.2 KiB	–
Game of Life	12.09 MiB	1.95–7.71 MiB	25.3 KiB	–
Wasm	–	–	–	2.0 KiB

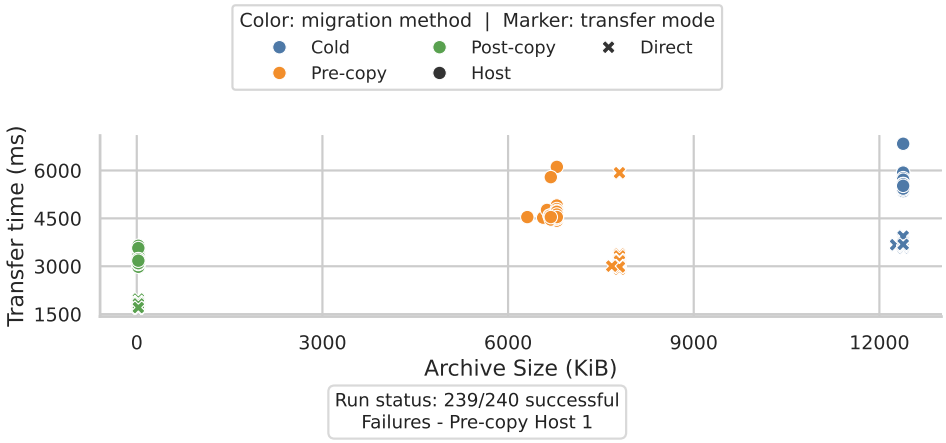


Figure 4.10.: Game of Life transfer size and transfer time under the 5G profile.

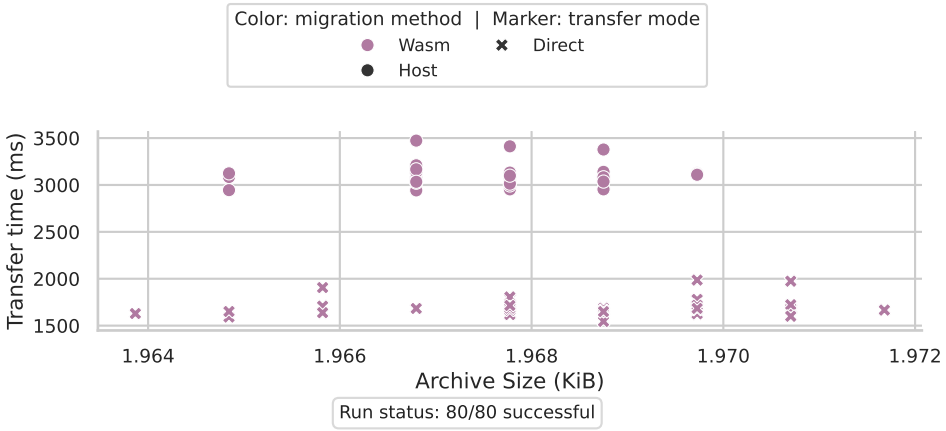


Figure 4.11.: Wasm transfer size and transfer time under the 5G profile.

Figures 4.12 and 4.13 show why host/relay mode is slower than direct mode. Direct mode has one copy leg. Host/relay mode has two copy legs and temporary relay storage. This extra flexibility has a measurable cost. It matters little for very small archives on fast links, but it becomes visible when the profile is slow or lossy.

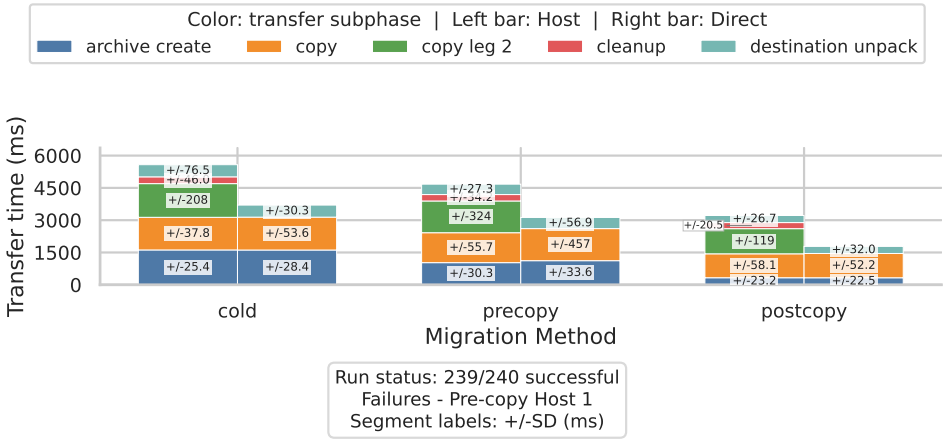


Figure 4.12.: Game of Life transfer phase breakdown under the 5G profile.

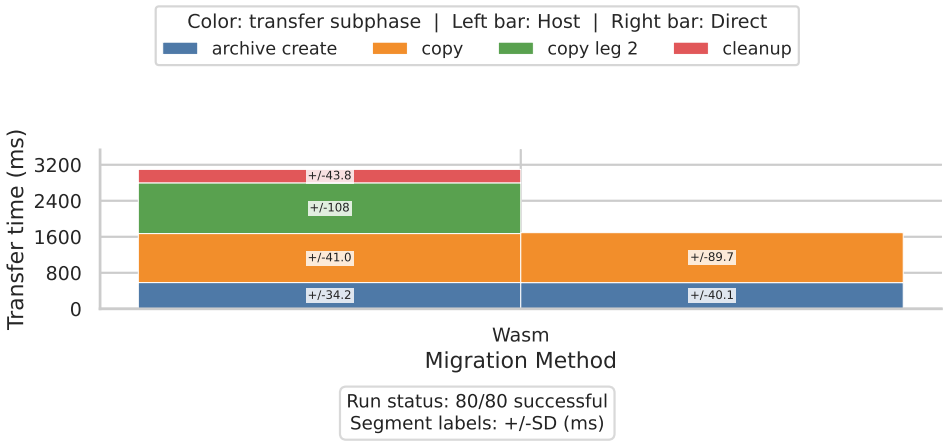


Figure 4.13.: Wasm transfer phase breakdown under the 5G profile.

The strategy-level archive sizes explain the rest. Cold migration moves one complete checkpoint archive. For Game of Life, that archive is consistently about 12.1 MiB. Pre-copy is different: each pre-dump writes an image of the process while it is still running, and the final dump is linked to those previous images. The final archive

therefore does not need to contain every memory page again. It mainly contains the process state that still has to be fixed at the final freeze, especially pages that changed after the previous pre-dump.

The median final archive for Game of Life ranges from about 7.7 MiB on WiFi 6 to about 4.8 MiB on Starlink, 2.5 MiB on TEE Best, and 2.0 MiB on TEE Avg. This should not be read as a general rule that worse networks make checkpoints smaller. It is an observed interaction between this workload and the pre-copy schedule. In slower profiles, each pre-dump transfer lasts much longer, so the final dump is taken after the Game of Life process has evolved for more time. Because the final dump is relative to the previous pre-dump images, it only needs to store what is still different at the final freeze. In addition, the random Game of Life board may become more regular or more compressible over time, so the remaining image can shrink even if the original application size has not changed. The measurements do not include dirty-page counts or board population counts, so they cannot isolate which effect dominates. Counter and the TCP client do not show this effect clearly because their image sets are only tens of KiB. The cost is visible in pre-dump time: under TEE Avg, the median Game of Life pre-dump phase is about 1 740 000 ms in direct mode and about 3 361 000 ms in host/relay mode.

Post-copy moves a very small initial archive when it succeeds. For Game of Life on WiFi 6 and 5G, the median initial archive is roughly 25 KiB. That explains why post-copy can look attractive in a downtime-only plot. The missing pages still have to move later through the lazy-pages path. In a fast profile this can be acceptable. In a disadvantaged profile it creates a hidden service-progress dependency: the process is restored, but execution can stall on remote page faults. These differences are small for Counter and the TCP client because the process state is small. They become central for Game of Life.

The Wasm transfer archive contains serialized application/runtime state rather than a CRIU image directory. Its median size is about 2 KiB across profiles, an order of magnitude below Counter and the TCP client and far below Game of Life cold migration. Although this is not a same-workload comparison, it demonstrates how runtime-aware checkpointing can save a compact state representation instead of moving kernel-visible process state.

4.5. Resource Usage

Resource usage reinforces the phase analysis. Figure 4.14 shows the representative Game of Life profile split by source and destination. Cold migration is comparatively balanced because the source dumps one image set and the destination restores it. In

direct mode under 5G, median CPU utilization is 12.9% on the source and 13.5% on the destination. Disk Input/Output (I/O) is also present on both nodes, with direct-mode medians of about 3.4 MB/s on the source and 2.7 MB/s on the destination, because the archive is created, transferred, unpacked, and restored.

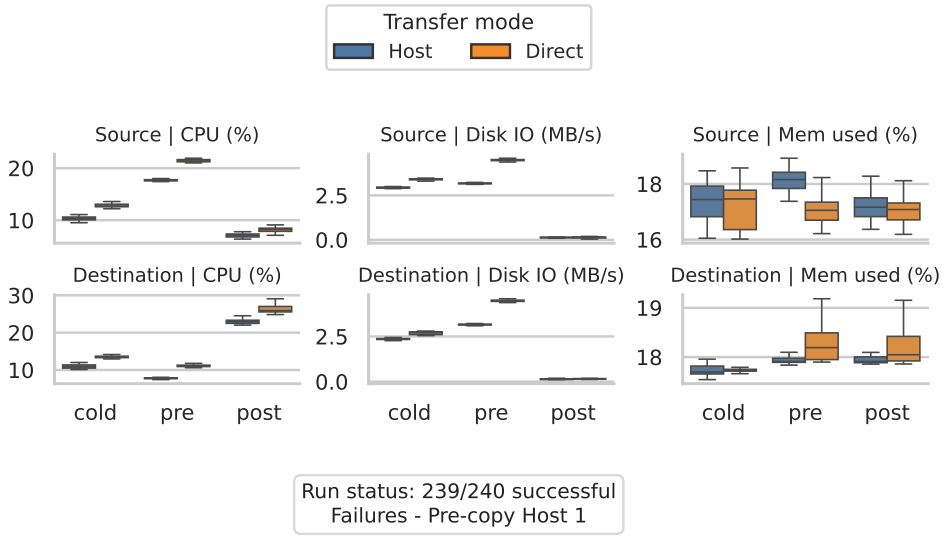


Figure 4.14.: Game of Life per-node resource usage under the 5G profile.

Pre-copy shifts CPU toward the source. In the same 5G direct-mode plot, median source CPU rises to 21.4%, while destination CPU is 11.1%. This matches the implementation: the source repeatedly pre-dumps while the workload keeps running and tracks memory that changes between iterations. Pre-copy also produces high disk activity on both nodes, about 4.5 MB/s each in the 5G direct median, because several image directories are archived, transferred, unpacked, and later used as an image chain.

Post-copy has a different shape. Source CPU is lower, about 8.2% in the 5G direct median, while destination CPU rises to 25.9%. That is consistent with the destination restoring early and then resolving missing pages through the lazy-pages path. Disk I/O is much lower for post-copy, around 0.14–0.16 MB/s, because the initial archive is tiny. The cost has moved from disk/archive handling into remote page serving and destination-side execution progress.

Figure 4.15 shows the same view for Wasm. The source and destination CPU values are close to each other, with direct-mode medians of 12.9% and 12.3%. That is more balanced than Game of Life pre-copy and post-copy, and similar in shape to a small cold migration. The important difference is disk I/O: Wasm direct-mode medians are about 0.19 MB/s on the source and 0.16 MB/s on the destination, far below all

Game of Life strategies except post-copy's tiny initial archive. Memory percentage after migration does not show a stable strategy pattern in these plots. It is useful as a sanity check that the nodes remain in a comparable range, but the clearer resource signals are CPU placement and disk I/O.

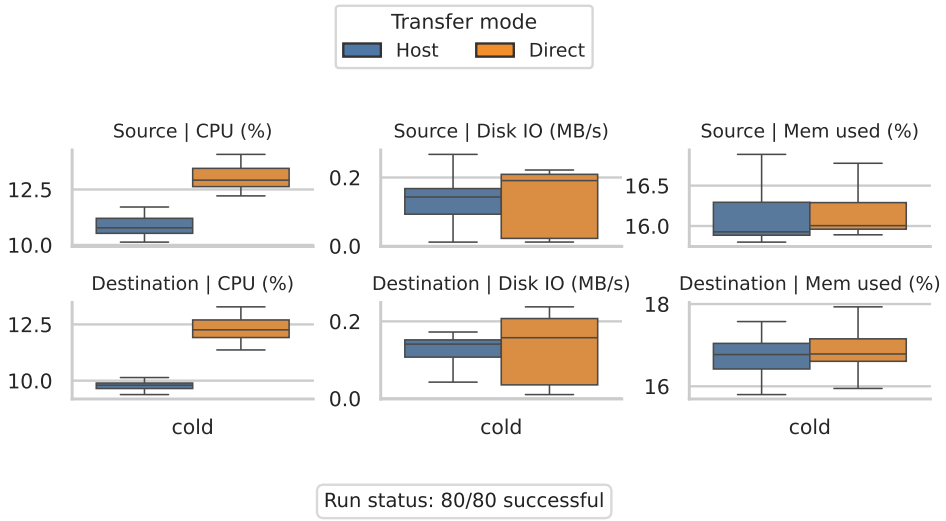


Figure 4.15.: Wasm per-node resource usage under the 5G profile.

4.6. Discussion

Taken together, the results show that migration feasibility depends on the interaction between workload state, link conditions, and the success criterion. Counter remains a fixed-overhead baseline even on poor links. The TCP client shows that preserving network state can reduce reliability before memory size becomes the main problem. Game of Life exposes the effect of a larger changing heap: pre-copy reduces its final unavailable window, while post-copy can restore quickly but still fail to produce observable progress. A latency distribution alone would hide that distinction.

For Game of Life, pre-copy transfers much of the memory state while the application is still running. Compared with cold migration, this reduces direct-mode downtime by approximately 22% under LTE and 82% under TEE Avg. The reduction comes with longer preparation, more disk activity, and a longer total migration. Post-copy makes the opposite trade-off by minimizing the initial archive while retaining a dependency on the source page server. The compact Wasm state and its completed TEE Worst runs show the potential of application-aware state capture, but they do not establish a same-workload speedup over native migration.

Transfer topology adds a separate operational constraint. Direct mode avoids the second copy leg and should be preferred when the endpoints can communicate. Host/relay mode remains useful when topology or policy requires a gateway, but its extra transfer cost becomes substantial on slow or lossy links. The setup-excluded plots isolate migration work from connection preparation; the raw totals in Appendix B.1 show that setup and handshakes can still become large under disadvantaged links.

There is therefore no fixed ranking of the mechanisms. Cold migration has the cleanest completion boundary but places the full archive transfer inside downtime. Pre-copy is effective when reducing that window justifies a longer migration and the workload does not dirty memory too quickly. Post-copy is attractive when the lazy-page path can sustain application progress after restore. Application-aware checkpointing can reduce the state sent through the network, at the cost of specialization and runtime support.

4.7. Limitations

The results should be read as measurements of this benchmark framework and its chosen workloads, not as universal migration constants. The workload mix is intentional: Counter isolates fixed overhead, the TCP client adds networked state, Game of Life adds memory pressure, and the Wasm workload represents application-level checkpointing. This makes it possible to compare migration approaches, but it does not make every workload a direct replacement for every other one.

The native measurements also stop below the container-orchestration layer. A full container or pod migration would add runtime metadata, filesystem state, namespaces, volumes, image layers, scheduling, and service-discovery behavior. The measured values are therefore useful lower-bound measurements for checkpoint/restore behavior, not complete Kubernetes or Podman migration numbers.

The transfer implementation also matters. The benchmark uses an SCP-based path because it is simple and reproducible, but under the TEE Worst profile it becomes part of the failure domain. The Game of Life result under that profile should therefore be read as a limit of the tested migration path under the configured link. It is not a statement that the process could never be migrated with another transport.

Finally, long experiments can accumulate VM state, stale processes, or Multipass restart problems. The framework mitigates this with reset scripts, run identifiers, chunking, and optional VM restarts. For TEE Worst, fewer tests were run than in the other profiles because each attempt took much longer and repeated transfer failures also made the full 40-runs impractical.

5. Conclusion

This thesis evaluated live migration strategies at the edge under a controlled set of network profiles and workloads. The experiments compared native process migration with CRIU and application-aware Wasm migration across WiFi 6, 5G, LTE, Starlink, and three tactical edge profiles. The goal was not only to measure whether a migration command can run. It was also to check whether a migrated service makes observable progress after restore. For that reason, the evaluation combines latency distributions with workload-level success criteria.

The clearest result is that network conditions become the dominant factor once the migrated state is large enough. For small workloads, such as Counter and the TCP client, fixed orchestration and transfer setup costs remain visible. Their downtime grows with the network profile, but the state is small enough that migration often remains feasible even under severe conditions. Game of Life changes the picture because its larger heap makes memory movement visible. Under Starlink and the TEE profiles, cold migration spends most of its unavailable window transferring the archive. Pre-copy can reduce that downtime by moving a large part of the state before the final freeze. For Game of Life in direct mode, the median downtime reduction grows from about 22% on LTE to about 82% on TEE Avg. The trade-off is also visible: under TEE Avg, direct-mode pre-copy increases setup-excluded migration time by about 61% compared with cold migration.

Post-copy has the opposite shape. When it works, it can start with a very small initial archive and short measured downtime. The restored process, however, remains dependent on lazy page delivery. This dependency is acceptable for small workloads and fast links, but it becomes fragile for Game of Life under LTE, Starlink, and tactical edge profiles. In these cases, the process may restore but fail to make prompt observable progress because missing pages are not delivered quickly or reliably enough. This does not make post-copy generally unsuitable; it shows that post-copy needs a reliable page-fault path and an application that can tolerate the remaining page-fetch cost after restore.

The Wasm migration results show a different design point. The Wasm prototype checkpoints application-level runtime state for the `3mm_with_cr.wasm` workload instead of capturing a Linux process tree. This is not a same-workload comparison with native CRIU, but it is still informative. The Wasm path remains successful through TEE Worst,

demonstrating the potential of specialized state capture when the runtime knows what must be saved. The cost is generality: this approach depends on module transformation and runtime support, while CRIU can be applied externally to many Linux processes.

The transfer topology result is partly expected, but still important to measure. Direct transfer is usually preferable when source and destination can communicate directly. Host/relay transfer is useful when a gateway path is operationally required, but it serializes two copy legs and becomes more expensive as the link becomes slower or lossier. The extra hop itself is not surprising. The important point is that edge deployments may impose that hop through topology, policy, or reachability constraints, and the cost can become large enough to change the migration decision.

Overall, the experiments support a workload-dependent view of live migration at the edge. There is no universally best migration strategy. Cold migration is simple and robust, but it places the full state-transfer cost inside downtime. Pre-copy is the strongest native option for memory-heavy workloads when the final unavailable window matters more than total migration duration, as long as the workload does not dirty memory faster than the pre-copy iterations can progress. A workload with a very high write rate, such as some training or data-processing jobs, can reduce this advantage because the final dump may still contain many changed pages. Post-copy can achieve very low initial downtime, but only when the lazy-page path is reliable enough for the restored application to keep making progress. Application-aware Wasm migration reduces unnecessary state movement, but does so by trading generality for specialization. In summary, live migration at the edge is not only about moving a process from one node to another. It is also about deciding whether the state is worth preserving, whether the network can carry it, and whether the service can still make progress after it arrives with that state intact.

5.1. Future Work

A first direction is container-level migration. The native CRIU experiments in this thesis migrate Linux processes directly. A production deployment would usually migrate containers or pods, for example through Podman, containerd, or Kubernetes-related mechanisms. That setting adds namespaces, runtime metadata, filesystem layers, volumes, service discovery, scheduling, and orchestration policy. Measuring the full path would show how much overhead appears above the checkpoint/restore core measured here.

A second direction is improving the transfer mechanism. The implementation uses SCP-based archive movement because it is simple and reproducible. Under TEE Worst, Game of Life shows that this path can fail before the migration strategy itself has

a chance to complete. Future work should evaluate resumable transfer, chunked archives, compression choices, transport protocols designed for lossy links, and tighter integration between pre-copy image generation and background transfer.

A third direction is adaptive migration policy. The results suggest that no strategy dominates. A practical edge system should choose cold, pre-copy, post-copy, or application-aware checkpointing based on workload state size, observed link quality, expected outage budget, and success risk. It should also decide whether migration is worth doing at all. For some services and profiles, restarting a fresh instance on the destination may be faster or more reliable than preserving the running state. Future work should therefore compare migration against redeployment before selecting the migration tool, the strategy, or the decision to migrate. The benchmark framework could be extended to learn or model those decisions from previous runs.

A fourth direction is broader workload and architecture coverage. The CSV schema already records source and destination architecture, but all final experiments used matching `x86_64` nodes. Future experiments should test heterogeneous architectures, different CPU and memory sizes, and workloads with different write rates and working-set shapes. This is especially relevant for Wasm: the application is compiled to a portable module and executed by a runtime, so the migration path may avoid some of the source/destination instruction-set coupling that native CRIU cannot avoid.

Finally, the Wasm path could be extended with more applications and more migration modes. The current prototype evaluates one application-aware migration path. Adding multiple checkpoint placements, incremental state transfer, or lazy state reconstruction would make it possible to compare Wasm-specific versions of cold, pre-copy, and post-copy ideas against the native CRIU strategies.

A. Appendix: Commands

This appendix documents the operational skeleton behind the experiments. Chapter 3 explains the design. This appendix records the commands, phases, and script entry points used to execute that design.

The examples use the standard node names from the repository. The source node is `edge-node-1`. The destination node is `edge-node-2`. The relay or server node is `edge-host-1`. Paths are shown as they appear in the repository unless they are remote VM paths.

A.1. Repository-Level Entry Points

The repository is organized around one top-level runner and four benchmark modules. The top-level runner applies network profiles, calls the module runners, and generates profile-specific plots. The module runners start workloads, run migrations, append CSV rows, and write run-id files.

Table A.1.: Main command entry points.

Entry point	Purpose
`run_all.py`	Runs configured benchmark suites under selected network profiles. It applies and clears traffic-control rules, supports chunking, restarts nodes, and triggers deferred plots.
`Container/scripts/orchestrators/repeat_benchmarks.py`	Repeats native CRIU counter migrations for cold, pre-copy, and post-copy strategies.
`Game-of-life-migration/scripts/orchestrators/repeat_benchmarks.py`	Repeats native CRIU Game of Life migrations.
`Network-live-migration/scripts/orchestrators/repeat_tcp_client_benchmarks.py`	Repeats native CRIU migrations of the TCP client workload. It also starts the TCP server and manages the VIP.
`WASM-migration/scripts/orchestrators/repeat_wasm_benchmarks.py`	Repeats the Wasm checkpoint/restore benchmark.

The suite registry is stored in `benchmarks.json`. The current registry expands to the

following command families.

```
python3 Container/scripts/orchestrators/repeat_benchmarks.py suite \
  --strategies cold,precopy,postcopy \
  --source edge-node-1 --dest edge-node-2 \
  --relay-node edge-host-1 \
  --host-runs 20 --direct-runs 20 \
  --iterations 3 \
  --snapshot-node-metrics \
  --profile-name "{profile_name}"

python3 Network-live-migration/scripts/orchestrators/
    repeat_tcp_client_benchmarks.py \
  --source edge-node-1 --dest edge-node-2 --server edge-host-1 \
  --relay-node edge-host-1 \
  --port 5000 --vip 10.138.215.250 \
  --strategies cold precopy postcopy \
  --iterations 3 \
  --host-runs 20 --direct-runs 20 \
  --snapshot-node-metrics \
  --profile-name "{profile_name}"

python3 Game-of-life-migration/scripts/orchestrators/repeat_benchmarks.py
     suite \
  --strategies cold,precopy,postcopy \
  --source edge-node-1 --dest edge-node-2 \
  --relay-node edge-host-1 \
  --host-runs 20 --direct-runs 20 \
  --iterations 3 \
  --snapshot-node-metrics \
  --profile-name "{profile_name}"

python3 WASM-migration/scripts/orchestrators/repeat_wasm_benchmarks.py
    suite \
  --strategies cold \
  --source edge-node-1 --dest edge-node-2 \
  --relay-node edge-host-1 \
  --host-runs 20 --direct-runs 20 \
  --snapshot-node-metrics \
```

```
  --profile-name "{profile_name}"
```

Listing A.1: Benchmark suite commands stored in benchmarks.json.

For long unattended runs, the experiments use chunking. A typical command is shown in Listing A.2. The chunk size limits how long CRIU, application, monitoring, and Multipass state can accumulate before the source and destination VMs are restarted.

```
python3 run_all.py \
  --profiles 1_WiFi_6,2_5G,3_LTE \
  --runs 40 \
  --run-chunk-size 10 \
  --restart-between-run-chunks \
  --restart-nodes edge-node-1,edge-node-2 \
  --continue-on-failure \
  --defer-suite-plots
```

Listing A.2: Top-level batch command for a 40-run profile sweep.

A.2. Node Provisioning and Setup

The VM setup is managed by Terraform and Multipass. The exact Terraform files are stored under `tools/terraform`. The usual host-side setup path is:

```
cd tools/terraform
terraform init
terraform apply -auto-approve
cd ../..

multipass list
bash tools/terraform/check_bootstrap.sh
```

Listing A.3: VM provisioning and readiness checks.

The readiness script is the source of truth before a benchmark run. It waits until each VM is reachable and until the bootstrap log reports that the node is fully provisioned. It also checks that key commands such as CRIU are available.

Node exporter is optional, but it is enabled in the main measurements. The scripts install the exporter, test it locally on each VM, and check clock synchronization.

```
bash Container/scripts/setup/install_node_exporter.sh \
  edge-node-1 edge-node-2 edge-host-1
```

```
bash Container/scripts/setup/check_node_exporter_metrics.sh \
  edge-node-1 edge-node-2 edge-host-1

bash Container/scripts/setup/check_time_sync.sh \
  edge-node-1 edge-node-2 edge-host-1
```

Listing A.4: Optional node exporter setup.

Before most tests, the corresponding module reset script is called. It kills remaining workload processes, removes CRIU temporary directories, removes stale post-copy page-server processes, and clears workload-specific PID files. The TCP reset path also removes the VIP from source and destination interfaces.

```
python3 Container/scripts/setup/reset_nodes.py \
  edge-node-1 edge-node-2

python3 Game-of-life-migration/scripts/setup/reset_nodes.py \
  edge-node-1 edge-node-2

python3 Network-live-migration/scripts/setup/reset_nodes.py \
  edge-node-1 edge-node-2 edge-host-1 10.138.215.250
```

Listing A.5: Representative reset commands.

A.3. Network Profile Application

Network profiles are stored in `network_profiles.json`. The runner applies each profile with Linux `tc`, `htb`, and `netem`. It first identifies each VM network interface and each VM IPv4 address. It then shapes only VM-to-VM traffic. Host-to-VM control traffic is kept mostly unshaped so that `multipass exec`, monitoring, and recovery commands remain usable.

The command skeleton used by `run_all.py` is:

```
# on each shaped VM interface
sudo tc qdisc del dev IFACE root 2>/dev/null || true
sudo tc qdisc add dev IFACE root handle 1: htb default 20
sudo tc class add dev IFACE parent 1: classid 1:1 htb rate ROOT_RATE
sudo tc class add dev IFACE parent 1:1 classid 1:20 htb rate ROOT_RATE
sudo tc class add dev IFACE parent 1:1 classid 1:10 htb rate BANDWIDTH
    ceil BANDWIDTH
```

```
sudo tc qdisc add dev IFACE parent 1:10 handle 10: netem delay LATENCY
    loss LOSS
sudo tc filter add dev IFACE protocol ip parent 1: prio 1 u32 match ip
    dst PEER_IP/32 flowid 1:10
```

Listing A.6: Traffic-control skeleton used by run_all.py.

Cleanup removes the root qdisc from each VM interface.

```
sudo tc qdisc del dev "$iface" root 2>/dev/null || true
```

Listing A.7: Traffic-control cleanup.

A.4. Workload Start Commands

Each native workload starts before every single migration run. This keeps measurements independent. The start scripts compile or reuse a remote binary, launch it in the background, write an output file, and store the PID in both a workload-specific file and a generic `/home/ubuntu/app.pid` file.

A.4.1. Counter Workload

The counter workload is a C process that writes incrementing values to `/home/ubuntu/counter.out`. The helper script compiles `Container/scripts/counter.c` if needed and starts `/tmp/counter`.

```
bash Container/scripts/workloads/start_counter_c.sh edge-node-1
```

Listing A.8: Counter workload start command.

The remote command shape is:

```
gcc -O2 -o /tmp/counter /home/ubuntu/counter.c
: > /home/ubuntu/counter.out
nohup /tmp/counter >> /home/ubuntu/counter.out 2>&1 &
echo $! > /home/ubuntu/counter.pid
cp /home/ubuntu/counter.pid /home/ubuntu/app.pid
```

Listing A.9: Remote counter process shape.

A.4.2. Game of Life Workload

The Game of Life workload is a C process. In the main measurements it uses a 2048 by 2048 grid, summary output, and a random initial pattern. This produces a larger heap-backed state than the counter workload.

```
GOL_WIDTH=2048 GOL_HEIGHT=2048 \
GOL_OUTPUT_MODE=summary GOL_PATTERN=random \
bash Game-of-life-migration/scripts/workloads/start_gol_c.sh edge-node-1
```

Listing A.10: Game of Life workload start command.

The remote process writes to `/home/ubuntu/gol.out`. The PID is stored in `/home/ubuntu/gol.pid` and `/home/ubuntu/app.pid`.

A.4.3. TCP Client Workload

The TCP workload has two processes. The server runs on `edge-host-1`. The client runs on `edge-node-1` and connects to the server. The client binds to a virtual IP. This is required because CRIU can restore an established TCP socket only if the local socket identity is preserved after restore.

```
TCP_PORT=5000
TCP_VIP=10.138.215.250

bash Network-live-migration/scripts/workloads/start_tcp_server.sh \
  edge-host-1 "$TCP_PORT"

TCP_VIP="$TCP_VIP" \
bash Network-live-migration/scripts/workloads/start_tcp_client.sh \
  edge-node-1 edge-host-1 "$TCP_PORT"
```

Listing A.11: TCP workload start commands.

The server endpoint is written to `/home/ubuntu/tcp_server_endpoint.txt`. The VIP is written to `/home/ubuntu/tcp_vip.txt`. The client PID is written to `/home/ubuntu/tcp_client.pid` and `/home/ubuntu/client.pid`.

A.5. Native CRIU Migration

The native CRIU workflows share a common structure. First, the orchestrator reads the source PID. Then it executes one of the migration strategies. After restore, it validates progress and appends a CSV row.

Table A.2.: CRIU flags used by the native process benchmarks.

Flag	Role in the experiments
`-t`, `--tree`	Selects the root process ID to dump or pre-dump.
`-D`, `--images-dir`	Selects the CRIU image directory.
`--prev-images-dir`	Points to a previous pre-dump image directory for iterative migration. The path is relative to the current image directory.
`--track-mem`	Enables memory-change tracking for iterative migration. It is used with pre-dump stages.
`--lazy-pages`	Enables post-copy migration. The initial image omits pages that can be fetched later.
`--address`, `--port`	Bind or connect to the post-copy page-server endpoint.
`--tcp-established`	Allows CRIU to dump and restore established TCP sockets. It is used by the TCP client workload.
`--shell-job`	Allows restoring a process that was launched as a shell-style background job.
`--restore-detached`	Restores the process in detached mode so the orchestrator command can return.
`--pidfile`	Writes the restored PID to a known file on the destination.
`--skip-file-rwx-check`	Relaxes CRIU file permission checks. It is used because workload binaries and output files are prepared by the orchestrator.

A.5.1. Cold Migration

Cold migration stops the process, creates a full image set, transfers it, unpacks it, and restores it on the destination. Downtime contains checkpoint, transfer, and restore.

```
# source: read PID and create a fresh image directory
PID=$(cat /home/ubuntu/counter.pid 2>/dev/null || cat /home/ubuntu/app.
    pid)
sudo rm -rf /tmp/CRIU-counter
sudo mkdir -p /tmp/CRIU-counter

# source: checkpoint the process
sudo criu dump \
  -t "$PID" \
  -D /tmp/CRIU-counter \
  -v4 -o dump.log \
```

```
  --shell-job \
  --skip-file-rwx-check

# source: package images
sudo tar -C /tmp -czf /tmp/CRIU-counter.tar.gz CRIU-counter
sudo cp /tmp/CRIU-counter.tar.gz /home/ubuntu/CRIU-counter.tar.gz
sudo chown ubuntu:ubuntu /home/ubuntu/CRIU-counter.tar.gz

# destination: unpack images
sudo rm -rf /tmp/CRIU-counter
sudo mkdir -p /tmp/CRIU-counter
sudo tar -C /tmp -xzf /home/ubuntu/CRIU-counter.tar.gz

# destination: restore process
sudo criu restore \
  -D /tmp/CRIU-counter \
  -v4 -o restore.log \
  --shell-job \
  --restore-detached \
  --pidfile /tmp/CRIU-counter/restored.pid \
  --skip-file-rwx-check
```

Listing A.12: Cold migration CRIU skeleton.

The Game of Life cold migration uses the same skeleton with `/tmp/CRIU-gol`, `/home/ubuntu/gol.pid`, and `/home/ubuntu/gol.out`. The TCP cold migration uses `/tmp/CRIU-tcp-client` and adds `--tcp-established`. It also moves the virtual IP before restore.

A.5.2. Pre-Copy Migration

Pre-copy migration runs one or more pre-dumps while the process continues to execute. Each pre-dump is archived and transferred to the destination before the final freeze. The final dump references the last pre-dump and captures the final delta. The final dump archive excludes the already-transferred pre-dump directories, but the destination still has the complete image chain because the pre-dump archives were unpacked earlier.

```
PID=$(cat /home/ubuntu/counter.pid 2>/dev/null || cat /home/ubuntu/app.
    pid)
sudo rm -rf /tmp/CRIU-counter
sudo mkdir -p /tmp/CRIU-counter
```

```
# source: iterative pre-dumps while the process is still running
sudo mkdir -p /tmp/CRIU-counter/iter-0
sudo criu pre-dump \
  -t "$PID" \
  -D /tmp/CRIU-counter/iter-0 \
  -v4 \
  --shell-job \
  --skip-file-rwx-check \
  --track-mem

sudo mkdir -p /tmp/CRIU-counter/iter-1
sudo criu pre-dump \
  -t "$PID" \
  -D /tmp/CRIU-counter/iter-1 \
  --prev-images-dir ../iter-0 \
  -v4 \
  --shell-job \
  --skip-file-rwx-check \
  --track-mem

# each iter-N directory is archived, transferred, and unpacked on
    destination
sudo tar -C /tmp/CRIU-counter -czf /home/ubuntu/CRIU-counter-iter-1.tar.
    gz iter-1

# source: final dump. This is the downtime checkpoint phase.
sudo criu dump \
  -t "$PID" \
  -D /tmp/CRIU-counter \
  -v4 -o dump.log \
  --prev-images-dir iter-1 \
  --shell-job \
  --skip-file-rwx-check

# source: archive only final dump files. Previous iter-* directories
    already exist on destination.
sudo tar -C /tmp --exclude='CRIU-counter/iter-*' \
  -czf /home/ubuntu/CRIU-counter-final.tar.gz CRIU-counter
```

```
# destination: unpack final dump over the existing pre-dump image chain
sudo mkdir -p /tmp/CRIU-counter
sudo tar -C /tmp -xzf /home/ubuntu/CRIU-counter-final.tar.gz

# destination: restore from the complete image chain
sudo criu restore \
  -D /tmp/CRIU-counter \
  -v4 -o restore.log \
  --shell-job \
  --restore-detached \
  --pidfile /tmp/CRIU-counter/restored.pid \
  --skip-file-rwx-check
```

Listing A.13: Pre-copy migration CRIU skeleton.

The benchmark records pre-dump time and streamed pre-copy transfer time separately. The plotted downtime uses only the final dump, the final archive transfer, and restore.

A.5.3. Post-Copy Migration

Post-copy migration creates a lazy image set and keeps a source-side CRIU process alive as a page server. The destination starts `criu lazy-pages`, restores with `--lazy-pages`, and fetches missing pages on demand. The archive transfer mode applies only to the initial image archive. Lazy page traffic uses the source-to-destination page-server path.

```
PID=$(cat /home/ubuntu/counter.pid 2>/dev/null || cat /home/ubuntu/app.
    pid)
SOURCE_IP=$(multipass exec edge-node-1 -- hostname -I | awk '{print $1}')
PORT=9999

# source: start lazy dump and page server
sudo rm -rf /tmp/CRIU-counter
sudo mkdir -p /tmp/CRIU-counter
sudo bash -lc '
  nohup criu dump -t '"$PID"' \
    -D /tmp/CRIU-counter \
    -v4 -o dump.log \
    --shell-job \
    --skip-file-rwx-check \
```

```
    --lazy-pages \
    --address 0.0.0.0 \
    --port '"$PORT"' \
    --leave-stopped \
    >/tmp/CRIU-counter/dump.stdout 2>&1 &
  echo $! > /tmp/CRIU-counter/dump.pid
'

# source: archive the initial image set
sudo tar -C /tmp -czf /tmp/CRIU-counter.tar.gz CRIU-counter
sudo cp /tmp/CRIU-counter.tar.gz /home/ubuntu/CRIU-counter.tar.gz
sudo chown ubuntu:ubuntu /home/ubuntu/CRIU-counter.tar.gz

# destination: unpack initial image set
sudo rm -rf /tmp/CRIU-counter
sudo mkdir -p /tmp/CRIU-counter
sudo tar -C /tmp -xzf /home/ubuntu/CRIU-counter.tar.gz

# destination: start lazy-pages helper
sudo bash -lc '
  cd /tmp/CRIU-counter
  nohup criu lazy-pages \
    -D /tmp/CRIU-counter \
    --page-server \
    --address '"$SOURCE_IP"' \
    --port '"$PORT"' \
    -v4 -o /tmp/CRIU-counter/lazy-pages.log \
    >/tmp/CRIU-counter/lazy-pages.stdout 2>&1 &
  echo $! > /tmp/CRIU-counter/lazy-pages.pid
'

# destination: restore with lazy pages enabled
sudo criu restore \
  -D /tmp/CRIU-counter \
  -v4 -o restore.log \
  --shell-job \
  --restore-detached \
  --pidfile /tmp/CRIU-counter/restored.pid \
  --skip-file-rwx-check \
```

```
  --lazy-pages
```

Listing A.14: Post-copy migration CRIU skeleton.

The orchestrator waits for the lazy-pages process, records its active time and log size, and kills both lazy-pages and the source page-server during cleanup.

A.6. Archive Transfer Modes

The migration strategy decides when images are produced. The transfer mode decides how the archive moves.

A.6.1. Direct Mode

In direct mode, the source VM copies the archive to the destination VM with SSH/SCP. The orchestrator first ensures SSH trust from source to destination. In the final scripts, the key installation step does not require a separate SSH probe by default. The transfer itself is the connectivity check: SCP is executed with retries and longer keepalive timeouts. A separate probe can still be enabled with `CRIU_SSH_TRUST_PROBE=1` for diagnosis.

```
# source: create \ac{SSH} key if missing
mkdir -p ~/.ssh && chmod 700 ~/.ssh
test -f ~/.ssh/id_ed25519 || \
  ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N "" -C "migration"

# destination: append source public key to authorized_keys
mkdir -p ~/.ssh && chmod 700 ~/.ssh
touch ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys

# source: copy archive to destination
scp -o BatchMode=yes \
    -o ConnectTimeout=45 \
    -o ConnectionAttempts=1 \
    -o ServerAliveInterval=30 \
    -o ServerAliveCountMax=4 \
    -o StrictHostKeyChecking=no \
    -o UserKnownHostsFile=/dev/null \
    /home/ubuntu/CRIU-counter.tar.gz \
    ubuntu@${DEST_IP}:/home/ubuntu/CRIU-counter.tar.gz
```

Listing A.15: Direct archive transfer skeleton.

A.6.2. Host or Relay Mode

Host mode has two implementations. Without a relay node, the physical host downloads the archive with `multipass transfer` and then uploads it to the destination. With `--relay-node edge-host-1`, the archive is staged through the relay VM with SCP.

```
multipass transfer edge-node-1:/home/ubuntu/CRIU-counter.tar.gz \
  /tmp/CRIU-counter.tar.gz

multipass transfer /tmp/CRIU-counter.tar.gz \
  edge-node-2:/home/ubuntu/CRIU-counter.tar.gz

rm -f /tmp/CRIU-counter.tar.gz
```

Listing A.16: Host-mediated transfer without relay VM.

```
# source -> relay
scp -o BatchMode=yes \
  -o ConnectTimeout=45 \
  -o ConnectionAttempts=1 \
  -o ServerAliveInterval=30 \
  -o ServerAliveCountMax=4 \
  -o StrictHostKeyChecking=no \
  -o UserKnownHostsFile=/dev/null \
  /home/ubuntu/CRIU-counter.tar.gz \
  ubuntu@${RELAY_IP}:/home/ubuntu/CRIU-counter.tar.gz

# relay -> destination
scp -o BatchMode=yes \
  -o ConnectTimeout=45 \
  -o ConnectionAttempts=1 \
  -o ServerAliveInterval=30 \
  -o ServerAliveCountMax=4 \
  -o StrictHostKeyChecking=no \
  -o UserKnownHostsFile=/dev/null \
  /home/ubuntu/CRIU-counter.tar.gz \
  ubuntu@${DEST_IP}:/home/ubuntu/CRIU-counter.tar.gz

# cleanup relay stage
```

```
rm -f /home/ubuntu/CRIU-counter.tar.gz
```

Listing A.17: Relay-mediated transfer through edge-host-1.

The transfer helper records setup time, first copy leg, second copy leg, cleanup time, and unpack time. Direct mode normally has one copy leg. Host or relay mode has two copy legs.

A.7. TCP-Specific CRIU Additions

The TCP benchmark follows the same cold, pre-copy, and post-copy structures. It adds two requirements. First, CRIU is called with `--tcp-established`. Second, the client virtual IP is moved from the source node to the destination node between dump and restore.

```
PID=$(cat /home/ubuntu/tcp_client.pid 2>/dev/null || cat /home/ubuntu/
    client.pid)
VIP=$(cat /home/ubuntu/tcp_vip.txt)
SERVER_ENDPOINT=$(cat /home/ubuntu/tcp_server_endpoint.txt)
SERVER_IP=${SERVER_ENDPOINT%:*}

sudo rm -rf /tmp/CRIU-tcp-client
sudo mkdir -p /tmp/CRIU-tcp-client
sudo criu dump \
  -t "$PID" \
  -D /tmp/CRIU-tcp-client \
  -v4 -o dump.log \
  --shell-job \
  --skip-file-rwx-check \
  --tcp-established

# move VIP from source to destination
sudo ip addr del "${VIP}/32" dev "$SRC_IFACE" 2>/dev/null || true
sudo ip addr add "${VIP}/32" dev "$DST_IFACE" 2>/dev/null || true
sudo arping -c 3 -A -I "$DST_IFACE" "$VIP" 2>/dev/null || true

# destination restore also uses --tcp-established
sudo criu restore \
  -D /tmp/CRIU-tcp-client \
  -v4 -o restore.log \
```

```
  --shell-job \
  --restore-detached \
  --pidfile /tmp/CRIU-tcp-client/restored.pid \
  --skip-file-rwx-check \
  --tcp-established
```

Listing A.18: TCP cold dump and VIP handoff skeleton.

After restore, the benchmark checks that the server still observes an established client connection and that the client continues producing output. If the VIP is not present on the destination, the restored socket has the wrong local identity and the benchmark fails.

A.8. Wasm Migration Command Skeleton

The Wasm module does not use CRIU. It uses the command interface from Edoardo Tinto's `wasm-migrate-commands`. The thesis benchmark does not reimplement that migration mechanism. It deploys the existing binaries, wraps them with repeatable orchestration, transfers the resulting state archive, and records metrics in the same style as the CRIU benchmarks.

The three upstream commands are:

```
./create_command comp.wasm ipc_file.txt main_memory.b checkpoint_memory.b
    \
  cgroup_name log_file.txt

./start_command ipc_file.txt

./migrate_command ipc_file.txt
```

Listing A.19: WebAssembly command interface.

The benchmark uses - as the cgroup name. This local change disables the original cgroup path requirement and allows the command to run on the Multipass nodes.

```
# host: deploy create_command, start_command, migrate_command, and module
multipass transfer wasm-migrate-commands/build/create_command \
  edge-node-1:/home/ubuntu/wasm-migration/bin/create_command
multipass transfer wasm-migrate-commands/build/start_command \
  edge-node-1:/home/ubuntu/wasm-migration/bin/start_command
multipass transfer wasm-migrate-commands/build/migrate_command \
  edge-node-1:/home/ubuntu/wasm-migration/bin/migrate_command
```

```
multipass transfer wasm-migrate-commands/wasm_test_computation/3
    mm_with_cr.wasm \
  edge-node-1:/home/ubuntu/wasm-migration/modules/3mm_with_cr.wasm

# source: create request server
create_command 3mm_with_cr.wasm source.ipc \
  source_main_memory.b source_checkpoint_memory.b \
  - source.log

# source: activate and then checkpoint
start_command source.ipc
migrate_command source.ipc

# source: package memory files
tar -czf wasm-state.tar.gz main_memory.b checkpoint_memory.b
```

Listing A.20: WebAssembly source-side checkpoint skeleton.

On the destination, the benchmark launches a new request server, unpacks the transferred memory files, places them where the destination server expects them, and sends `start_command`. Restore is considered complete when the destination log reports `request_server - restore memory completed`.

```
# destination: create destination request server
create_command 3mm_with_cr.wasm dest.ipc \
  dest_main_memory.b dest_checkpoint_memory.b \
  - dest.log

# destination: unpack and seed memory files
tar -xzf wasm-state.tar.gz
cp main_memory.b dest_main_memory.b
cp checkpoint_memory.b dest_checkpoint_memory.b

# destination: restore by activating the destination server
start_command dest.ipc

# benchmark waits for this log marker
# request_server - restore memory completed
```

Listing A.21: WebAssembly destination-side restore skeleton.

The Wasm metrics are mapped into the shared CSV schema as follows. `checkpoint_ms` is computed from injected source log markers between `checkpoint start` and `checkpoint completed`. `archive_create_ms` measures the Python wrapper around memory-file packaging. `transfer_ms` measures the archive transfer helper. `restore_ms` is a broad destination-side timer. It includes destination server launch, memory seeding, `start_command`, and the wait for the restore-complete marker.

A.9. Metrics, Artifacts, and Plots

Each module writes raw timing rows to a module-local CSV file. The common file name is `metrics/migration_metrics.csv`. Each repeat runner also writes a `*.run_ids.txt` file. Deferred plotting uses these run identifiers to filter append-only CSV files and avoid mixing older experiments into a new plot directory.

Table A.3.: Main artifact locations.

Path pattern	Contents
`<module>/metrics/migration_metrics.csv`	Raw migration rows for that module.
`<module>/metrics/node_exporter_metrics.csv`	Summaries derived from before/after node exporter snapshots.
`<module>/metrics/run_logs/`	Per-suite logs and run-id files.
`<module>/metrics/plots/<batch>/`	Generated figures and filtered CSV files for one batch/profile.
`run_all_artifacts/run_all__<timestamp>.log`	Top-level runner log for one profile invocation.
`run_all_artifacts/logs/`	Per-benchmark command logs created by `run_all.py`.

Plot generation is module-local. The script names are the same across the CRIU modules and the Wasm module.

```
python3 Container/scripts/visualization/generate_all_plots.py \
  --csv Container/metrics/migration_metrics.csv \
  --run-ids-file Container/metrics/run_logs/<run>.run_ids.txt \
  --profile-name 1_WiFi_6 \
  --out-dir Container/metrics/plots/<batch>
```

Listing A.22: Manual plot generation skeleton.

The generated plot set normally includes downtime comparison, phase breakdown, transfer analysis, transfer phase breakdown, and node exporter summaries when node snapshots exist. The current plot scripts write both `.png` and `.pdf` versions of each figure. Timing plots are generated from successful migration rows; the filtered input CSV and `successful\_migration\_metrics.csv` remain in the plot directory so failed runs can still be counted when reporting success rates.

A.10. Failure Recovery Commands

Long runs can fail because of stuck Multipass restarts, stale CRIU page servers, failed TCP VIP cleanup, or disk pressure from accumulated logs and plots. The following commands are used to inspect the state before restarting a batch.

```
multipass list
tail -f run_all_artifacts/<profile-or-driver>.out
tail -f run_all_artifacts/run_all__<timestamp>.log
```

Listing A.23: Operational checks during long runs.

If a VM is left in a bad Multipass state, the least destructive recovery is to restart the Multipass service, start stopped nodes, and resume with a fresh benchmark command. Existing CSV and plot artifacts are not deleted by these commands.

```
sudo snap restart multipass
multipass list
multipass start edge-node-1 edge-node-2 edge-host-1
multipass list
```

Listing A.24: Multipass recovery skeleton.

Module diagnosis scripts can be run after a failed migration. They check node reachability, CRIU availability, PID files, image directories, restored process state, and recent CRIU logs.

```
bash Container/scripts/helpers/diagnose_migration.sh edge-node-1 edge-
    node-2
bash Game-of-life-migration/scripts/helpers/diagnose_migration.sh edge-
    node-1 edge-node-2
bash Network-live-migration/scripts/helpers/diagnose_migration.sh edge-
    node-1 edge-node-2
```

Listing A.25: Diagnosis helpers.

B. Additional Evaluation Plots

This appendix collects the generated evaluation plots that are not all shown in Chapter 4. The main chapter selects representative figures for the argument; this appendix keeps the remaining plots available for inspection. Failed runs are removed from timing distributions, while every plot annotation records the corresponding success count.

The complete local plot inventory contains all profiles for Counter, TCP Client, and Wasm. Game of Life is available through TEE Avg. The TEE Worst Game of Life run did not produce a complete plot set because the transfers repeatedly failed under the configured network impairment.

B.1. Timing Extremes

Tables in this section summarize the fastest and slowest successful runs for each workload, profile, and migration method. Values are reported in milliseconds and failed runs are excluded from the timing cells. The success column reports successful runs over attempted runs, summed across host and direct transfer modes. Downtime tables use the same setup-excluded downtime definition used by the plots, so `C`, `T`, and `R` are checkpoint or final dump, transfer, and restore inside the plotted downtime window. Migration-time tables select the fastest and slowest runs by raw elapsed total, because that is the wall-clock duration observed by the benchmark driver; they also show the setup-excluded total for comparison. They also report `O`, the remaining setup-excluded time outside checkpoint, transfer, and restore, because migration time can include preparation, pre-dumps, validation waits, and other non-downtime work. For pre-copy, `P` reports pre-dump time, which happens before the final downtime window. Wasm checkpoint values are shown in microseconds because they are below the millisecond precision used for the other phases.

B.1.1. Downtime Extremes

Table B.1.: Counter downtime extremes by profile and method. Timing cells stack transfer mode, setup-excluded downtime in ms, and separate C, T, and R phases.

Profile	Method	Fastest	Slowest	Succ.
WiFi 6	Cold	direct total: 2 352 C: 528 T: 1 495 R: 329	host total: 4 216 C: 593 T: 3 317 R: 306	80/80
WiFi 6	Pre-copy	direct total: 2 296 C: 529 T: 1 461 R: 306 P: 16 055	host total: 4 361 C: 896 T: 3 126 R: 339 P: 27 597	79/80
WiFi 6	Post-copy	direct total: 2 788 C: 1 031 T: 1 398 R: 359	host total: 4 550 C: 1 386 T: 2 836 R: 328	80/80
5G	Cold	direct total: 2 552 C: 599 T: 1 650 R: 303	host total: 4 531 C: 606 T: 3 561 R: 364	80/80
5G	Pre-copy	direct total: 2 535 C: 555 T: 1 664 R: 316 P: 17 626	host total: 4 836 C: 876 T: 3 659 R: 301 P: 29 488	80/80
5G	Post-copy	direct total: 3 024 C: 1 043 T: 1 603 R: 378	host total: 5 110 C: 1 217 T: 3 541 R: 352	80/80
LTE	Cold	direct total: 3 784 C: 580 T: 2 934 R: 270	host total: 7 317 C: 646 T: 6 372 R: 299	79/80
LTE	Pre-copy	direct total: 3 749 C: 559 T: 2 885 R: 305 P: 22 524	host total: 7 252 C: 725 T: 6 204 R: 323 P: 40 302	80/80
LTE	Post-copy	direct total: 4 348 C: 985 T: 2 870 R: 493	host total: 8 248 C: 1 040 T: 6 687 R: 521	80/80

Profile	Method	Fastest	Slowest	Succ.
Starlink	Cold	direct total: 4 032 C: 592 T: 3 164 R: 276	host total: 9 068 C: 720 T: 7 936 R: 412	80/80
Starlink	Pre-copy	direct total: 3 932 C: 541 T: 3 084 R: 307 P: 25 555	host total: 9 141 C: 569 T: 8 223 R: 349 P: 43 445	80/80
Starlink	Post-copy	direct total: 4 552 C: 983 T: 3 115 R: 454	host total: 8 969 C: 1 122 T: 7 344 R: 503	80/80
TEE Best	Cold	direct total: 10 551 C: 636 T: 9 603 R: 312	host total: 33 701 C: 692 T: 32 633 R: 376	80/80
TEE Best	Pre-copy	direct total: 10 488 C: 556 T: 9 595 R: 337 P: 66 607	host total: 32 585 C: 957 T: 31 314 R: 314 P: 123 391	79/80
TEE Best	Post-copy	direct total: 12 251 C: 936 T: 10 135 R: 1 180	host total: 32 490 C: 1 006 T: 29 458 R: 2 026	80/80
TEE Avg	Cold	direct total: 20 075 C: 592 T: 19 185 R: 298	host total: 85 361 C: 584 T: 84 478 R: 299	80/80
TEE Avg	Pre-copy	direct total: 22 111 C: 576 T: 21 243 R: 292 P: 152 591	host total: 93 704 C: 572 T: 92 805 R: 327 P: 290 330	78/80
TEE Avg	Post-copy	direct total: 22 885 C: 1 066 T: 19 899 R: 1 920	host total: 84 689 C: 1 059 T: 77 135 R: 6 495	80/80
TEE Worst	Cold	direct total: 155 148 C: 628 T: 154 216 R: 304	host total: 1 204 100 C: 644 T: 1 203 139 R: 317	59/60

Profile	Method	Fastest	Slowest	Succ.
TEE Worst	Pre-copy	direct total: 152 647 C: 915 T: 151 420 R: 312 P: 662 394	host total: 1 097 795 C: 912 T: 1 096 566 R: 317 P: 1 674 253	59/60
TEE Worst	Post-copy	direct total: 155 762 C: 1 016 T: 141 376 R: 13 370	host total: 952 595 C: 970 T: 925 796 R: 25 829	60/60

Table B.2.: TCP Client downtime extremes by profile and method. Timing cells stack transfer mode, setup-excluded downtime in ms, and separate C, T, and R phases.

Profile	Method	Fastest	Slowest	Succ.
WiFi 6	Cold	direct total: 2 110 C: 341 T: 1 452 R: 317	host total: 3 809 C: 545 T: 2 782 R: 482	80/80
WiFi 6	Pre-copy	direct total: 2 119 C: 301 T: 1 518 R: 300 P: 16 107	host total: 4 094 C: 358 T: 3 277 R: 459 P: 27 889	79/80
WiFi 6	Post-copy	direct total: 2 504 C: 729 T: 1 415 R: 360	host total: 4 632 C: 863 T: 3 425 R: 344	78/80
5G	Cold	direct total: 2 323 C: 333 T: 1 678 R: 312	host total: 4 555 C: 386 T: 3 869 R: 300	80/80
5G	Pre-copy	direct total: 2 211 C: 296 T: 1 587 R: 328 P: 17 545	host total: 4 336 C: 711 T: 3 309 R: 316 P: 29 887	79/80
5G	Post-copy	direct total: 2 755 C: 755 T: 1 624 R: 376	host total: 4 943 C: 768 T: 3 774 R: 401	80/80

Profile	Method	Fastest	Slowest	Succ.
LTE	Cold	direct total: 3 523 C: 316 T: 2 891 R: 316	host total: 6 880 C: 360 T: 6 214 R: 306	80/80
LTE	Pre-copy	direct total: 3 543 C: 296 T: 2 944 R: 303 P: 22 715	host total: 7 057 C: 699 T: 6 017 R: 341 P: 40 917	80/80
LTE	Post-copy	direct total: 4 141 C: 771 T: 2 874 R: 496	host total: 7 490 C: 701 T: 6 294 R: 495	80/80
Starlink	Cold	direct total: 3 763 C: 358 T: 3 098 R: 307	host total: 8 430 C: 346 T: 7 747 R: 337	80/80
Starlink	Pre-copy	direct total: 3 632 C: 280 T: 3 048 R: 304 P: 24 348	host total: 7 865 C: 632 T: 6 891 R: 342 P: 42 683	76/80
Starlink	Post-copy	direct total: 4 290 C: 700 T: 3 060 R: 530	host total: 8 559 C: 752 T: 7 290 R: 517	80/80
TEE Best	Cold	direct total: 10 983 C: 322 T: 10 353 R: 308	host total: 29 198 C: 349 T: 28 543 R: 306	48/80
TEE Best	Pre-copy	direct total: 10 194 C: 305 T: 9 572 R: 317 P: 59 020	host total: 29 801 C: 299 T: 29 138 R: 364 P: 122 238	73/80
TEE Best	Post-copy	direct total: 11 560 C: 752 T: 9 662 R: 1 146	host total: 36 561 C: 733 T: 34 695 R: 1 133	66/80
TEE Avg	Cold	direct total: 20 322 C: 330 T: 19 643 R: 349	host total: 80 216 C: 432 T: 79 371 R: 413	74/80

Profile	Method	Fastest	Slowest	Succ.
TEE Avg	Pre-copy	direct total: 21 288 C: 329 T: 20 633 R: 326 P: 160 342	host total: 89 500 C: 643 T: 88 541 R: 316 P: 274 041	58/80
TEE Avg	Post-copy	direct total: 20 856 C: 715 T: 18 176 R: 1 965	host total: 105 370 C: 730 T: 101 051 R: 3 589	68/80
TEE Worst	Cold	direct total: 166 984 C: 340 T: 166 317 R: 327	host total: 1 031 267 C: 348 T: 1 030 581 R: 338	52/60
TEE Worst	Pre-copy	direct total: 173 937 C: 625 T: 172 982 R: 330 P: 775 429	host total: 969 373 C: 664 T: 968 397 R: 312 P: 1 203 034	34/60
TEE Worst	Post-copy	direct total: 176 398 C: 782 T: 163 250 R: 12 366	host total: 790 281 C: 745 T: 763 638 R: 25 898	24/60

Table B.3.: Game of Life downtime extremes by profile and method. Timing cells stack transfer mode, setup-excluded downtime in ms, and separate C, T, and R phases.

Profile	Method	Fastest	Slowest	Succ.
WiFi 6	Cold	direct total: 3 689 C: 340 T: 3 005 R: 344	host total: 5 850 C: 437 T: 5 074 R: 339	80/80
WiFi 6	Pre-copy	direct total: 3 239 C: 320 T: 2 595 R: 324 P: 20 468	host total: 4 882 C: 551 T: 4 003 R: 328 P: 31 787	80/80
WiFi 6	Post-copy	direct total: 2 784 C: 999 T: 1 444 R: 341	host total: 6 339 C: 999 T: 5 021 R: 319	80/80

Profile	Method	Fastest	Slowest	Succ.
5G	Cold	direct total: 4 265 C: 358 T: 3 575 R: 332	host total: 7 512 C: 353 T: 6 837 R: 322	80/80
5G	Pre-copy	direct total: 3 565 C: 315 T: 2 925 R: 325 P: 21 947	host total: 6 843 C: 397 T: 6 113 R: 333 P: 34 609	79/80
5G	Post-copy	direct total: 3 039 C: 934 T: 1 725 R: 380	host total: 5 075 C: 1 077 T: 3 642 R: 356	80/80
LTE	Cold	direct total: 7 568 C: 346 T: 6 906 R: 316	host total: 34 275 C: 358 T: 33 574 R: 343	80/80
LTE	Pre-copy	direct total: 5 776 C: 320 T: 5 110 R: 346 P: 43 897	host total: 17 720 C: 302 T: 17 102 R: 316 P: 55 313	79/80
LTE	Post-copy	–	–	0/80
Starlink	Cold	direct total: 7 370 C: 336 T: 6 667 R: 367	host total: 122 322 C: 368 T: 121 623 R: 331	80/80
Starlink	Pre-copy	direct total: 5 590 C: 340 T: 4 924 R: 326 P: 113 840	host total: 46 116 C: 657 T: 45 138 R: 321 P: 160 119	79/80
Starlink	Post-copy	–	–	0/80
TEE Best	Cold	direct total: 368 100 C: 341 T: 367 433 R: 326	host total: 1 075 766 C: 389 T: 1 075 034 R: 343	79/80
TEE Best	Pre-copy	direct total: 63 399 C: 661 T: 62 390 R: 348 P: 760 232	host total: 243 462 C: 640 T: 242 498 R: 324 P: 1 280 339	80/80

Profile	Method	Fastest	Slowest	Succ.
TEE Best	Post-copy	host total: 1 000 300 C: 980 T: 998 935 R: 385	host total: 1 000 300 C: 980 T: 998 935 R: 385	1/80
TEE Avg	Cold	direct total: 1 060 617 C: 340 T: 1 059 938 R: 339	host total: 2 638 855 C: 306 T: 2 638 197 R: 352	79/80
TEE Avg	Pre-copy	direct total: 166 449 C: 711 T: 165 426 R: 312 P: 1 776 580	host total: 490 114 C: 677 T: 489 088 R: 349 P: 3 483 861	79/80
TEE Avg	Post-copy	–	–	0/80
TEE Worst	Cold	–	–	–
TEE Worst	Pre-copy	–	–	–
TEE Worst	Post-copy	–	–	–

Table B.4.: Wasm downtime extremes by profile and method. Timing cells stack transfer mode, setup-excluded downtime in ms, C in microseconds, and T/R in ms.

Profile	Method	Fastest	Slowest	Succ.
WiFi 6	Wasm	direct total: 3 304 C: 127 μs T: 1 395 R: 1 909	host total: 5 097 C: 193 μs T: 3 093 R: 2 004	80/80
5G	Wasm	direct total: 3 522 C: 166 μs T: 1 545 R: 1 977	host total: 5 475 C: 133 μs T: 3 473 R: 2 002	80/80
LTE	Wasm	direct total: 4 669 C: 124 μs T: 2 769 R: 1 900	host total: 8 087 C: 129 μs T: 6 197 R: 1 890	80/80
Starlink	Wasm	direct total: 4 897 C: 141 μs T: 3 013 R: 1 884	host total: 9 331 C: 123 μs T: 7 387 R: 1 944	80/80

Profile	Method	Fastest	Slowest	Succ.
TEE Best	Wasm	direct total: 11 115 C: 180 μs T: 9 213 R: 1 902	host total: 32 158 C: 128 μs T: 30 239 R: 1 919	80/80
TEE Avg	Wasm	direct total: 20 011 C: 138 μs T: 18 088 R: 1 923	host total: 69 835 C: 127 μs T: 67 785 R: 2 050	79/80
TEE Worst	Wasm	direct total: 152 287 C: 145 μs T: 149 920 R: 2 367	host total: 936 904 C: 138 μs T: 934 620 R: 2 284	60/60

B.1.2. Migration Time Extremes

Table B.5.: Counter migration time extremes by profile and method. Timing cells stack transfer mode, raw total (the selection metric), setup-excluded total, and separate C, T, R, and O phases.

Profile	Method	Fastest	Slowest	Succ.
WiFi 6	Cold	direct raw: 12 923 excl.: 10 888 C: 598 T: 1 549 R: 303 O: 8 438	host raw: 16 328 excl.: 10 978 C: 643 T: 2 805 R: 463 O: 7 067	80/80
WiFi 6	Pre-copy	direct raw: 28 373 excl.: 26 223 C: 611 T: 1 536 R: 327 O: 23 749 P: 15 799	host raw: 43 342 excl.: 38 302 C: 550 T: 2 744 R: 306 O: 34 702 P: 28 546	79/80
WiFi 6	Post-copy	direct raw: 16 382 excl.: 14 180 C: 1 136 T: 1 535 R: 315 O: 11 194	host raw: 19 692 excl.: 14 266 C: 1 131 T: 2 744 R: 337 O: 10 054	80/80

Profile	Method	Fastest	Slowest	Succ.
5G	Cold	direct raw: 13 235 excl.: 10 966 C: 614 T: 1 774 R: 310 O: 8 268	host raw: 16 629 excl.: 11 410 C: 736 T: 3 260 R: 442 O: 6 972	80/80
5G	Pre-copy	direct raw: 29 767 excl.: 27 501 C: 578 T: 1 754 R: 298 O: 24 871 P: 16 650	host raw: 45 989 excl.: 40 577 C: 958 T: 3 088 R: 301 O: 36 230 P: 30 169	80/80
5G	Post-copy	direct raw: 16 744 excl.: 14 403 C: 1 052 T: 1 738 R: 358 O: 11 255	host raw: 21 242 excl.: 15 999 C: 1 034 T: 3 279 R: 373 O: 11 313	80/80
LTE	Cold	direct raw: 15 803 excl.: 12 811 C: 572 T: 2 989 R: 324 O: 8 926	host raw: 20 997 excl.: 14 099 C: 756 T: 5 744 R: 505 O: 7 094	79/80
LTE	Pre-copy	direct raw: 37 650 excl.: 34 638 C: 561 T: 2 924 R: 291 O: 30 862 P: 22 119	host raw: 61 135 excl.: 54 801 C: 659 T: 5 707 R: 310 O: 48 125 P: 41 800	80/80
LTE	Post-copy	direct raw: 20 803 excl.: 17 921 C: 1 111 T: 2 943 R: 493 O: 13 374	host raw: 25 959 excl.: 19 387 C: 1 040 T: 6 687 R: 521 O: 11 139	80/80
Starlink	Cold	direct raw: 16 263 excl.: 13 205 C: 543 T: 3 228 R: 311 O: 9 123	host raw: 22 720 excl.: 16 099 C: 720 T: 7 936 R: 412 O: 7 031	80/80

Profile	Method	Fastest	Slowest	Succ.
		direct raw: 40 100 excl.: 36 874 C: 578 T: 3 190 R: 343 O: 32 763	host raw: 65 944 excl.: 58 850 C: 607 T: 6 285 R: 291 O: 51 667	
Starlink	Pre-copy	P: 23 905	P: 45 473	80/80
		direct raw: 21 436 excl.: 18 339 C: 984 T: 3 131 R: 547	host raw: 27 279 excl.: 19 100 C: 1 194 T: 6 340 R: 516	
Starlink	Post-copy	O: 13 677	O: 11 050	80/80
		direct raw: 32 170 excl.: 23 992 C: 613 T: 10 021 R: 300	host raw: 56 590 excl.: 39 612 C: 624 T: 31 851 R: 300	
TEE Best	Cold	O: 13 058	O: 6 837	80/80
		direct raw: 91 414 excl.: 84 625 C: 536 T: 9 694 R: 310 O: 74 085	host raw: 187 488 excl.: 169 383 C: 963 T: 28 148 R: 292 O: 139 980	
TEE Best	Pre-copy	P: 60 538	P: 133 535	79/80
		direct raw: 42 465 excl.: 35 661 C: 1 043 T: 11 435 R: 1 183	host raw: 68 976 excl.: 52 244 C: 1 066 T: 30 275 R: 1 085	
TEE Best	Post-copy	O: 22 000	O: 19 818	80/80
		direct raw: 58 575 excl.: 45 539 C: 625 T: 22 010 R: 317	host raw: 126 370 excl.: 91 532 C: 586 T: 84 342 R: 300	
TEE Avg	Cold	O: 22 587	O: 6 304	80/80
		direct raw: 171 799 excl.: 156 279 C: 574 T: 22 402 R: 312 O: 132 991	host raw: 437 184 excl.: 391 500 C: 899 T: 50 349 R: 288 O: 339 964	
TEE Avg	Pre-copy	P: 113 796	P: 333 938	78/80

Profile	Method	Fastest	Slowest	Succ.
TEE Avg	Post-copy	direct raw: 74 708 excl.: 60 237 C: 1 027 T: 21 632 R: 1 916 O: 35 662	host raw: 149 601 excl.: 84 157 C: 993 T: 52 906 R: 1 897 O: 28 361	80/80
TEE Worst	Cold	direct raw: 164 020 excl.: 162 626 C: 628 T: 154 216 R: 304 O: 7 478	host raw: 1 751 495 excl.: 1 210 458 C: 644 T: 1 203 139 R: 317 O: 6 358	59/60
TEE Worst	Pre-copy	direct raw: 706 179 excl.: 704 734 C: 906 T: 177 220 R: 316 O: 526 292 P: 518 937	host raw: 2 781 279 excl.: 2 778 233 C: 912 T: 1 096 566 R: 317 O: 1 680 438 P: 1 674 253	59/60
TEE Worst	Post-copy	direct raw: 186 528 excl.: 185 144 C: 1 016 T: 141 376 R: 13 370 O: 29 382	host raw: 984 215 excl.: 981 125 C: 970 T: 925 796 R: 25 829 O: 28 530	60/60

Table B.6.: TCP Client migration time extremes by profile and method. Timing cells stack transfer mode, raw total (the selection metric), setup-excluded total, and separate C, T, R, and O phases.

Profile	Method	Fastest	Slowest	Succ.
WiFi 6	Cold	direct raw: 18 481 excl.: 16 298 C: 349 T: 1 494 R: 323 O: 14 132	host raw: 21 783 excl.: 16 947 C: 545 T: 2 782 R: 482 O: 13 138	80/80
WiFi 6	Pre-copy	direct raw: 34 648 excl.: 32 609 C: 317 T: 1 624 R: 321 O: 30 347 P: 15 689	host raw: 49 814 excl.: 45 047 C: 354 T: 2 623 R: 432 O: 41 638 P: 27 975	79/80

Profile	Method	Fastest	Slowest	Succ.
		direct raw: 20 870 excl.: 18 681 C: 749 T: 1 438 R: 330	host raw: 24 201 excl.: 19 302 C: 863 T: 3 425 R: 344	
WiFi 6	Post-copy	O: 16 164	O: 14 670	78/80
		direct raw: 18 907 excl.: 16 696 C: 335 T: 1 899 R: 311	host raw: 22 465 excl.: 16 795 C: 372 T: 3 190 R: 310	
5G	Cold	O: 14 151	O: 12 923	80/80
		direct raw: 35 609 excl.: 33 503 C: 297 T: 1 720 R: 305 O: 31 181	host raw: 52 048 excl.: 46 661 C: 376 T: 3 286 R: 403 O: 42 596	
5G	Pre-copy	P: 16 707	P: 29 774	79/80
		direct raw: 21 208 excl.: 18 914 C: 742 T: 1 798 R: 369	host raw: 24 574 excl.: 19 330 C: 742 T: 3 187 R: 379	
5G	Post-copy	O: 16 005	O: 15 022	80/80
		direct raw: 21 345 excl.: 18 388 C: 328 T: 2 954 R: 312	host raw: 26 215 excl.: 19 441 C: 410 T: 5 680 R: 416	
LTE	Cold	O: 14 794	O: 12 935	80/80
		direct raw: 44 476 excl.: 41 386 C: 286 T: 2 963 R: 305 O: 37 832	host raw: 67 619 excl.: 61 254 C: 311 T: 5 705 R: 296 O: 54 942	
LTE	Pre-copy	P: 22 225	P: 42 267	80/80
		direct raw: 23 857 excl.: 20 879 C: 700 T: 3 002 R: 574	host raw: 28 933 excl.: 22 304 C: 787 T: 5 956 R: 505	
LTE	Post-copy	O: 16 603	O: 15 056	80/80

Profile	Method	Fastest	Slowest	Succ.
Starlink	Cold	direct raw: 22 141 excl.: 19 035 C: 339 T: 3 178 R: 320 O: 15 198	host raw: 28 297 excl.: 20 552 C: 462 T: 6 716 R: 439 O: 12 935	80/80
Starlink	Pre-copy	direct raw: 46 370 excl.: 43 278 C: 294 T: 3 188 R: 330 O: 39 466 P: 23 949	host raw: 72 710 excl.: 65 683 C: 309 T: 6 785 R: 316 O: 58 273 P: 45 338	76/80
Starlink	Post-copy	direct raw: 24 543 excl.: 21 444 C: 764 T: 3 108 R: 505 O: 17 067	host raw: 30 380 excl.: 23 204 C: 831 T: 6 872 R: 550 O: 14 951	80/80
TEE Best	Cold	direct raw: 37 902 excl.: 30 404 C: 328 T: 10 857 R: 334 O: 18 885	host raw: 56 999 excl.: 38 777 C: 373 T: 25 731 R: 320 O: 12 353	48/80
TEE Best	Pre-copy	direct raw: 94 917 excl.: 87 386 C: 294 T: 9 661 R: 295 O: 77 136 P: 56 635	host raw: 189 100 excl.: 172 405 C: 279 T: 24 625 R: 343 O: 147 158 P: 134 243	73/80
TEE Best	Post-copy	direct raw: 39 930 excl.: 33 082 C: 757 T: 10 061 R: 1 150 O: 21 114	host raw: 69 689 excl.: 51 376 C: 733 T: 34 695 R: 1 133 O: 14 815	66/80
TEE Avg	Cold	direct raw: 63 057 excl.: 51 397 C: 352 T: 23 719 R: 318 O: 27 008	host raw: 127 269 excl.: 76 599 C: 422 T: 62 160 R: 420 O: 13 597	74/80

Profile	Method	Fastest	Slowest	Succ.
TEE Avg	Pre-copy	direct raw: 187 163 excl.: 172 296 C: 613 T: 25 029 R: 315 O: 146 339 P: 121 633	host raw: 421 612 excl.: 387 426 C: 631 T: 57 312 R: 323 O: 329 160 P: 316 133	58/80
TEE Avg	Post-copy	direct raw: 66 754 excl.: 53 155 C: 742 T: 24 455 R: 1 947 O: 26 011	host raw: 152 428 excl.: 120 064 C: 730 T: 101 051 R: 3 589 O: 14 694	68/80
TEE Worst	Cold	direct raw: 183 286 excl.: 181 831 C: 340 T: 166 317 R: 327 O: 14 847	host raw: 1 047 619 excl.: 1 044 545 C: 348 T: 1 030 581 R: 338 O: 13 278	52/60
TEE Worst	Pre-copy	direct raw: 787 512 excl.: 786 074 C: 656 T: 207 932 R: 314 O: 577 172 P: 561 668	host raw: 2 962 169 excl.: 2 958 912 C: 651 T: 756 998 R: 336 O: 2 200 927 P: 2 187 234	34/60
TEE Worst	Post-copy	direct raw: 194 489 excl.: 193 101 C: 782 T: 163 250 R: 12 366 O: 16 703	host raw: 808 703 excl.: 805 563 C: 745 T: 763 638 R: 25 898 O: 15 282	24/60

Table B.7.: Game of Life migration time extremes by profile and method. Timing cells stack transfer mode, raw total (the selection metric), setup-excluded total, and separate C, T, R, and O phases.

Profile	Method	Fastest	Slowest	Succ.
WiFi 6	Cold	direct raw: 16 023 excl.: 13 861 C: 370 T: 3 267 R: 324 O: 9 900	host raw: 19 684 excl.: 14 499 C: 482 T: 4 833 R: 520 O: 8 664	80/80

Profile	Method	Fastest	Slowest	Succ.
		direct raw: 35 743 excl.: 33 639 C: 319 T: 2 654 R: 399 O: 30 267	host raw: 50 569 excl.: 45 869 C: 401 T: 3 998 R: 420 O: 41 050	
WiFi 6	Pre-copy	P: 20 380	P: 32 812	80/80
		direct raw: 21 596 excl.: 19 413 C: 1 012 T: 1 611 R: 334	host raw: 25 818 excl.: 20 508 C: 1 023 T: 2 757 R: 407	
WiFi 6	Post-copy	O: 16 456	O: 16 321	80/80
		direct raw: 16 583 excl.: 14 322 C: 355 T: 3 685 R: 328	host raw: 20 860 excl.: 16 007 C: 353 T: 6 837 R: 322	
5G	Cold	O: 9 954	O: 8 495	80/80
		direct raw: 37 352 excl.: 34 990 C: 310 T: 3 004 R: 339 O: 31 337	host raw: 56 785 excl.: 51 559 C: 370 T: 4 522 R: 325 O: 46 342	
5G	Pre-copy	P: 21 441	P: 38 323	79/80
		direct raw: 25 866 excl.: 23 475 C: 976 T: 1 722 R: 364	host raw: 31 480 excl.: 25 864 C: 986 T: 3 213 R: 417	
5G	Post-copy	O: 20 413	O: 21 248	80/80
		direct raw: 21 173 excl.: 18 209 C: 349 T: 6 928 R: 326	host raw: 48 719 excl.: 42 405 C: 358 T: 33 574 R: 343	
LTE	Cold	O: 10 606	O: 8 130	80/80
		direct raw: 52 098 excl.: 49 039 C: 361 T: 5 553 R: 375 O: 42 750	host raw: 96 737 excl.: 88 575 C: 319 T: 8 976 R: 393 O: 78 887	
LTE	Pre-copy	P: 31 863	P: 70 719	79/80
LTE	Post-copy	–	–	0/80

Profile	Method	Fastest	Slowest	Succ.
Starlink	Cold	direct raw: 21 340 excl.: 18 178 C: 336 T: 6 667 R: 367 O: 10 808	host raw: 137 626 excl.: 130 359 C: 368 T: 121 623 R: 331 O: 8 037	80/80
Starlink	Pre-copy	direct raw: 95 784 excl.: 90 889 C: 312 T: 14 785 R: 343 O: 75 449 P: 64 583	host raw: 253 238 excl.: 246 334 C: 631 T: 39 479 R: 336 O: 205 888 P: 197 784	79/80
Starlink	Post-copy	–	–	0/80
TEE Best	Cold	direct raw: 389 559 excl.: 382 605 C: 341 T: 367 433 R: 326 O: 14 505	host raw: 1 099 854 excl.: 1 083 743 C: 389 T: 1 075 034 R: 343 O: 7 977	79/80
TEE Best	Pre-copy	direct raw: 786 254 excl.: 779 295 C: 683 T: 95 804 R: 317 O: 682 491 P: 666 452	host raw: 1 774 396 excl.: 1 757 133 C: 640 T: 199 096 R: 336 O: 1 557 061 P: 1 548 919	80/80
TEE Best	Post-copy	host raw: 1 033 972 excl.: 1 017 859 C: 980 T: 998 935 R: 385 O: 17 559	host raw: 1 033 972 excl.: 1 017 859 C: 980 T: 998 935 R: 385 O: 17 559	1/80
TEE Avg	Cold	direct raw: 1 098 774 excl.: 1 080 025 C: 350 T: 1 060 032 R: 394 O: 19 249	host raw: 2 681 765 excl.: 2 646 836 C: 306 T: 2 638 197 R: 352 O: 7 981	79/80
TEE Avg	Pre-copy	direct raw: 1 887 917 excl.: 1 876 225 C: 642 T: 223 247 R: 346 O: 1 651 990 P: 1 623 340	host raw: 4 029 618 excl.: 3 989 146 C: 699 T: 445 133 R: 357 O: 3 542 957 P: 3 534 917	79/80
TEE Avg	Post-copy	–	–	0/80

Profile	Method	Fastest	Slowest	Succ.
TEE Worst	Cold	–	–	–
TEE Worst	Pre-copy	–	–	–
TEE Worst	Post-copy	–	–	–

Table B.8.: Wasm migration time extremes by profile and method. Timing cells stack transfer mode, raw total (the selection metric), setup-excluded total, C in microseconds, and T/R/O in ms.

Profile	Method	Fastest	Slowest	Succ.
WiFi 6	Wasm	direct raw: 10 798 excl.: 8 620 C: 221 µs T: 1 346 R: 1 986 O: 5 288	host raw: 15 856 excl.: 11 050 C: 156 µs T: 2 759 R: 2 129 O: 6 162	80/80
5G	Wasm	direct raw: 10 892 excl.: 8 704 C: 161 µs T: 1 600 R: 1 927 O: 5 177	host raw: 16 280 excl.: 11 062 C: 136 µs T: 3 132 R: 2 326 O: 5 604	80/80
LTE	Wasm	direct raw: 12 959 excl.: 10 021 C: 124 µs T: 2 730 R: 2 010 O: 5 281	host raw: 20 577 excl.: 13 526 C: 129 µs T: 6 197 R: 1 890 O: 5 439	80/80
Starlink	Wasm	direct raw: 13 247 excl.: 10 163 C: 154 µs T: 3 016 R: 1 939 O: 5 208	host raw: 22 291 excl.: 14 527 C: 180 µs T: 7 014 R: 2 004 O: 5 509	80/80
TEE Best	Wasm	direct raw: 23 792 excl.: 16 960 C: 125 µs T: 9 479 R: 2 036 O: 5 445	host raw: 52 596 excl.: 37 415 C: 128 µs T: 30 239 R: 1 919 O: 5 257	80/80

Profile	Method	Fastest	Slowest	Succ.
TEE Avg	Wasm	direct raw: 39 321 excl.: 26 063 C: 126 μs T: 18 715 R: 1 932 O: 5 416	host raw: 110 463 excl.: 63 273 C: 120 μs T: 55 377 R: 2 343 O: 5 553	79/80
TEE Worst	Wasm	direct raw: 255 080 excl.: 173 508 C: 127 μs T: 165 715 R: 2 227 O: 5 566	host raw: 1 275 489 excl.: 942 508 C: 138 μs T: 934 620 R: 2 284 O: 5 604	60/60

B.2. CDF Plots

The CDF plots are generated in three comparison groups: all profiles together, the access-network profiles, and the tactical-edge profiles. The access-network group contains WiFi 6, 5G, and LTE; Starlink is shown in the all-profile group and again with the tactical-edge profiles. Each group is split by metric, transfer mode, and workload. All CDF x-axes are linear milliseconds, and the tick labels show raw millisecond values.

B.2.1. All Profiles

B.2.1.1. Host/Relay Mode: Downtime CDFs

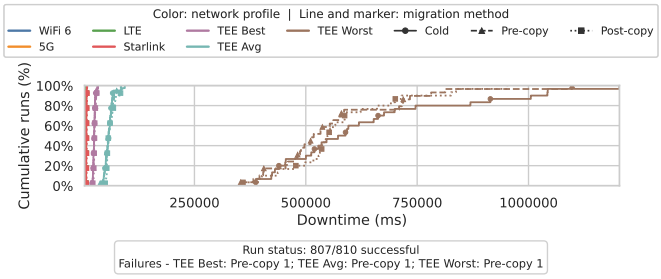


Figure B.1.: Counter downtime CDF for host/relay transfer mode in the all profiles comparison.

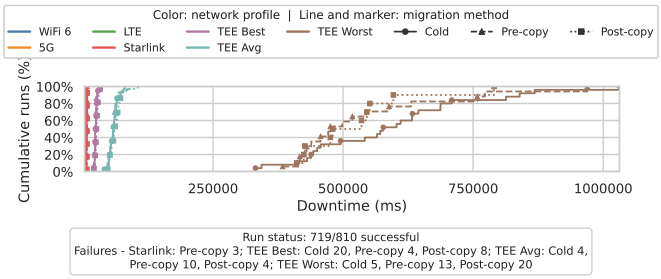


Figure B.2.: TCP Client downtime CDF for host/relay transfer mode in the all profiles comparison.

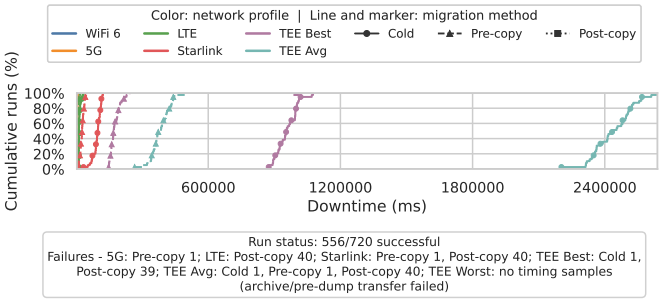


Figure B.3.: Game of Life downtime CDF for host/relay transfer mode in the all profiles comparison.

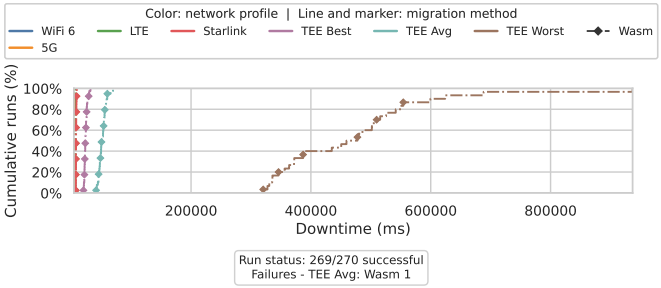


Figure B.4.: Wasm downtime CDF for host/relay transfer mode in the all profiles comparison.

B.2.1.2. Direct Mode: Downtime CDFs

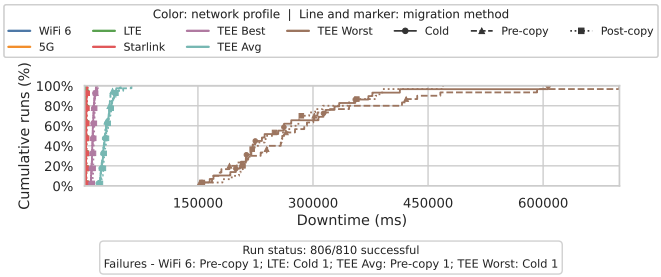


Figure B.5.: Counter downtime CDF for direct transfer mode in the all profiles comparison.

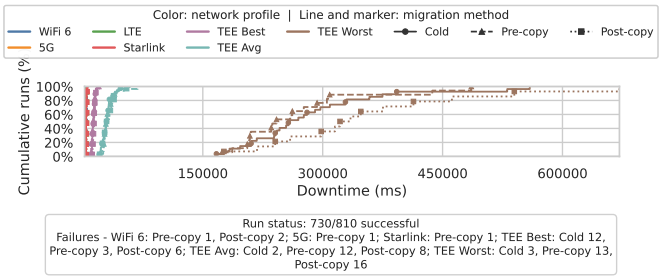


Figure B.6.: TCP Client downtime CDF for direct transfer mode in the all profiles comparison.

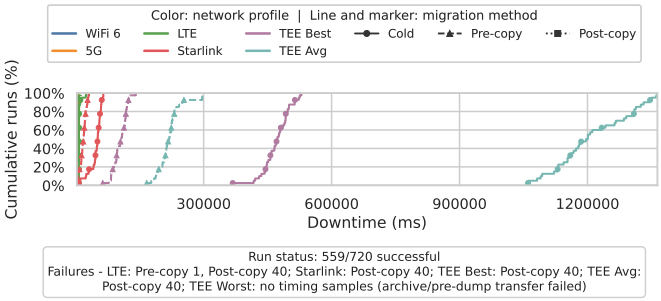


Figure B.7.: Game of Life downtime CDF for direct transfer mode in the all profiles comparison.

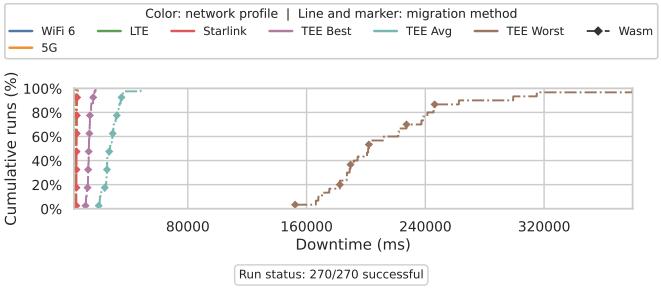


Figure B.8.: Wasm downtime CDF for direct transfer mode in the all profiles comparison.

B.2.2. Access Networks

B.2.2.1. Host/Relay Mode: Downtime CDFs

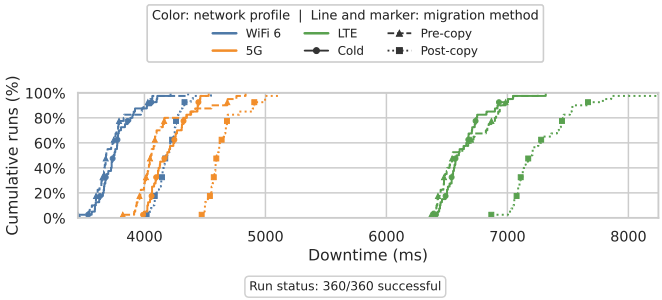


Figure B.9.: Counter downtime CDF for host/relay transfer mode in the access networks comparison.

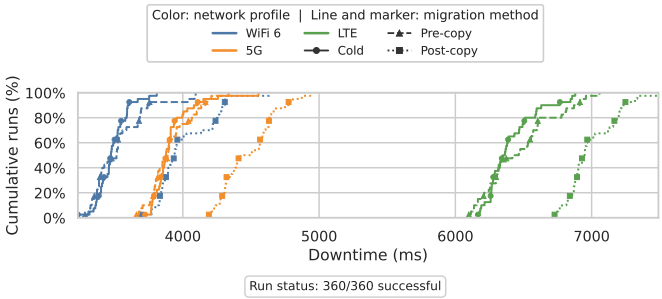


Figure B.10.: TCP Client downtime CDF for host/relay transfer mode in the access networks comparison.

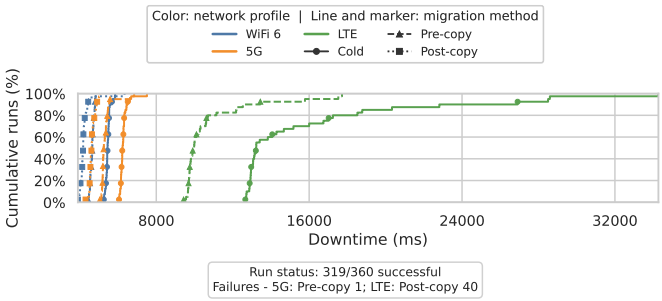


Figure B.11.: Game of Life downtime CDF for host/relay transfer mode in the access networks comparison.

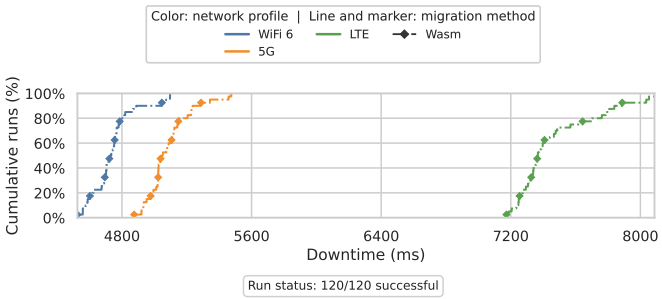


Figure B.12.: Wasm downtime CDF for host/relay transfer mode in the access networks comparison.

B.2.2.2. Direct Mode: Downtime CDFs

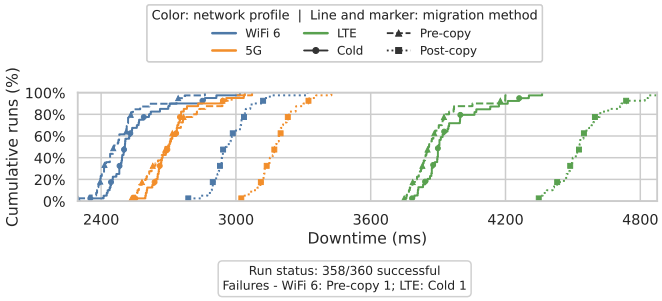


Figure B.13.: Counter downtime CDF for direct transfer mode in the access networks comparison.

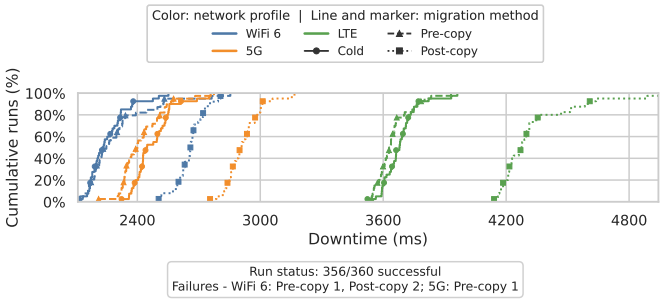


Figure B.14.: TCP Client downtime CDF for direct transfer mode in the access networks comparison.

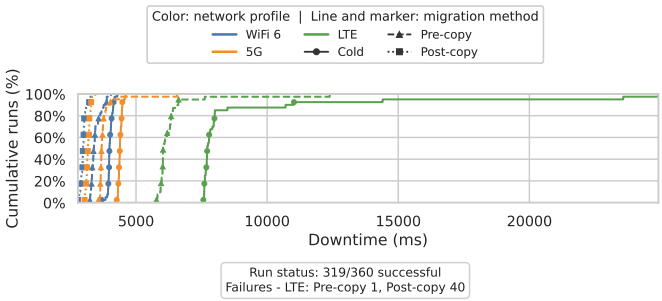


Figure B.15.: Game of Life downtime CDF for direct transfer mode in the access networks comparison.

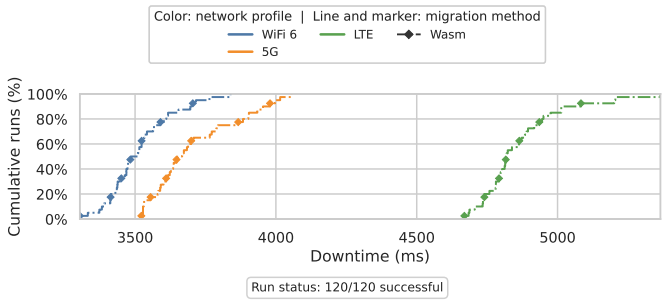


Figure B.16.: Wasm downtime CDF for direct transfer mode in the access networks comparison.

B.2.3. Tactical Edge

B.2.3.1. Host/Relay Mode: Downtime CDFs

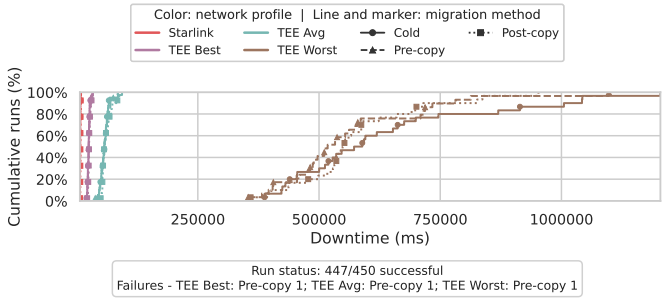


Figure B.17.: Counter downtime CDF for host/relay transfer mode in the tactical edge comparison.

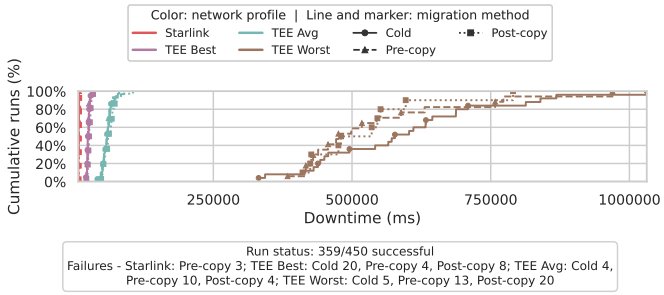


Figure B.18.: TCP Client downtime CDF for host/relay transfer mode in the tactical edge comparison.

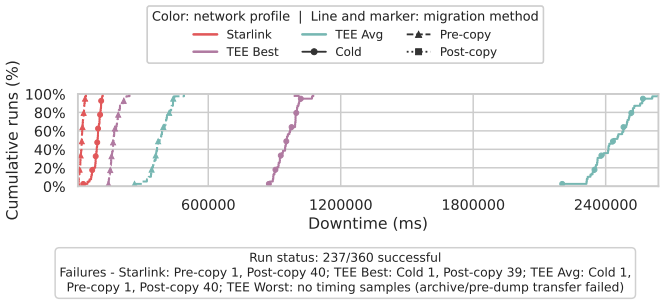


Figure B.19.: Game of Life downtime CDF for host/relay transfer mode in the tactical edge comparison.

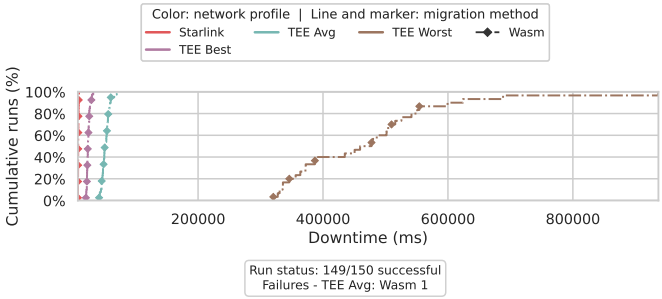


Figure B.20.: Wasm downtime CDF for host/relay transfer mode in the tactical edge comparison.

B.2.3.2. Direct Mode: Downtime CDFs

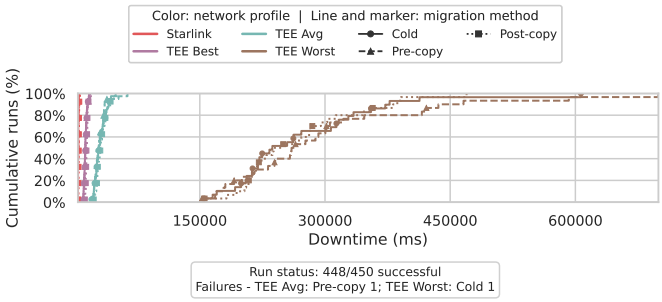


Figure B.21.: Counter downtime CDF for direct transfer mode in the tactical edge comparison.

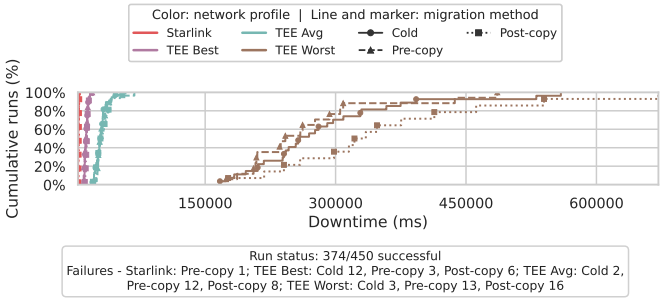


Figure B.22.: TCP Client downtime CDF for direct transfer mode in the tactical edge comparison.

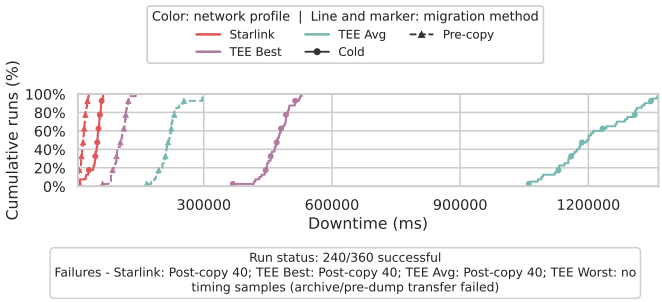


Figure B.23.: Game of Life downtime CDF for direct transfer mode in the tactical edge comparison.

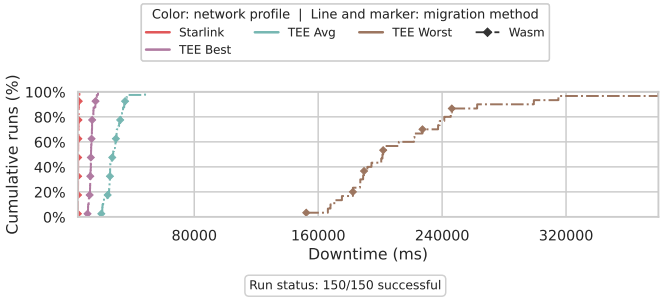


Figure B.24.: Wasm downtime CDF for direct transfer mode in the tactical edge comparison.

B.3. Counter Plots

These figures show the smallest native CRIU workload. Each available profile includes downtime, phase, transfer, transfer-breakdown, and per-node resource plots.

B.3.1. WiFi 6

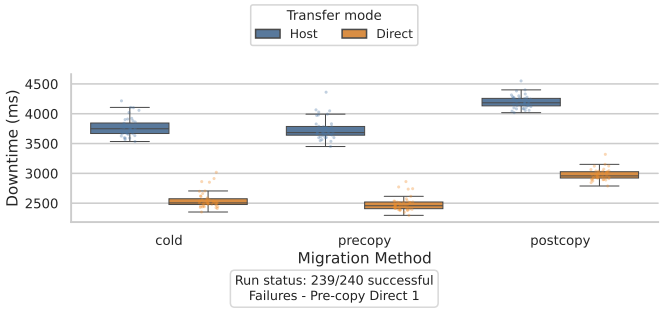


Figure B.25.: Counter downtime comparison under WiFi 6.

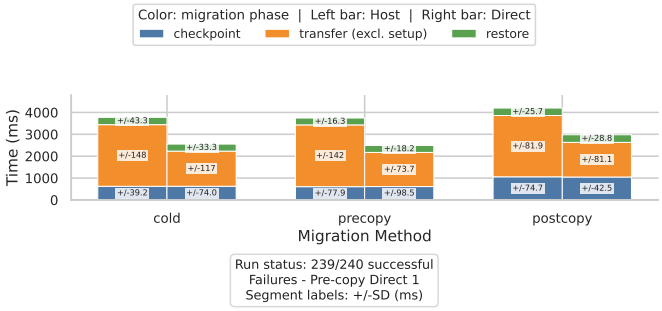


Figure B.26.: Counter phase breakdown under WiFi 6.

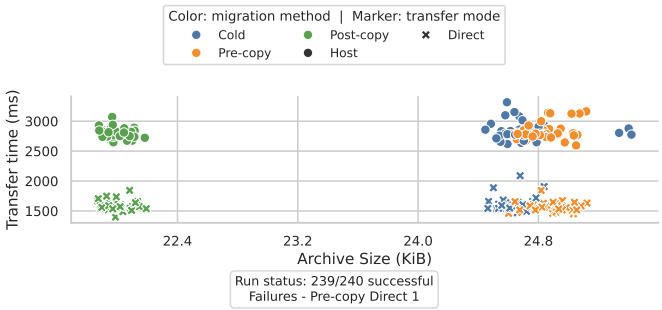


Figure B.27.: Counter transfer size and transfer time under WiFi 6.

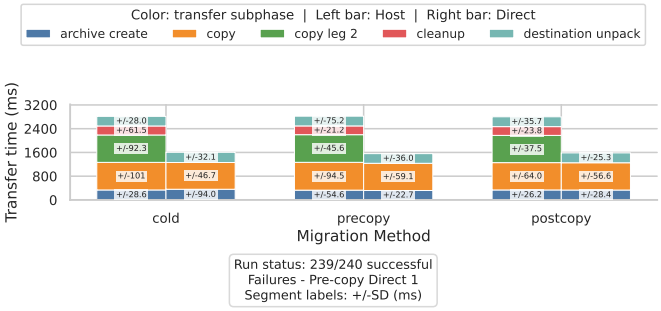


Figure B.28.: Counter transfer phase breakdown under WiFi 6.

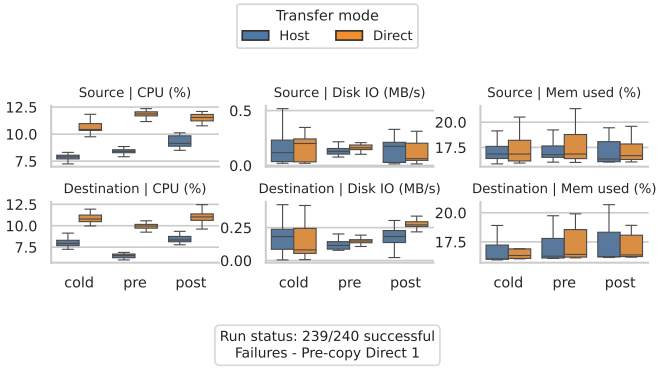


Figure B.29.: Counter per-node resource usage under WiFi 6.

B.3.2. 5G

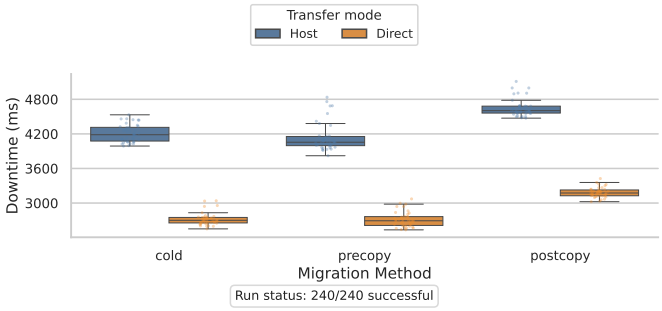


Figure B.30.: Counter downtime comparison under 5G.

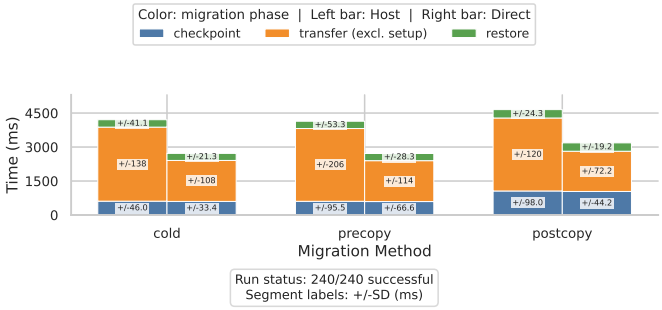


Figure B.31.: Counter phase breakdown under 5G.

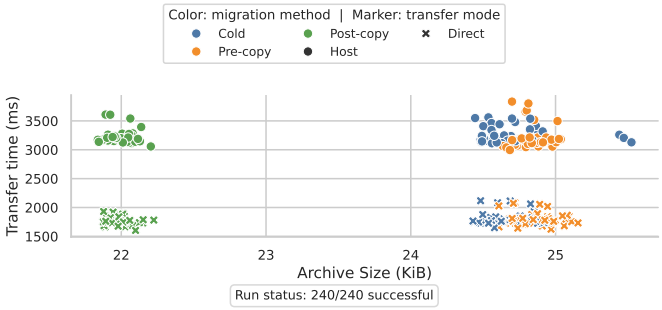


Figure B.32.: Counter transfer size and transfer time under 5G.

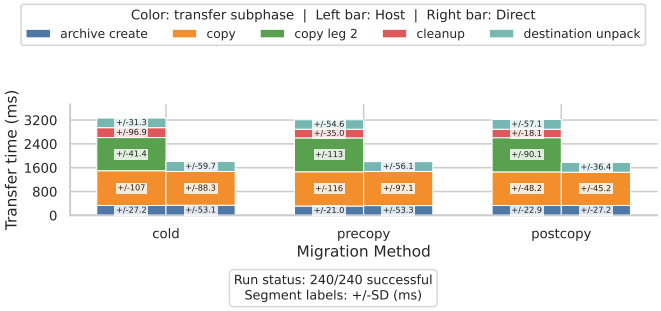


Figure B.33.: Counter transfer phase breakdown under 5G.

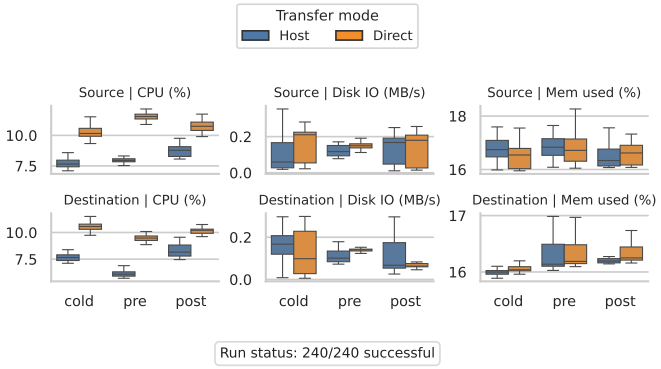


Figure B.34.: Counter per-node resource usage under 5G.

B.3.3. LTE

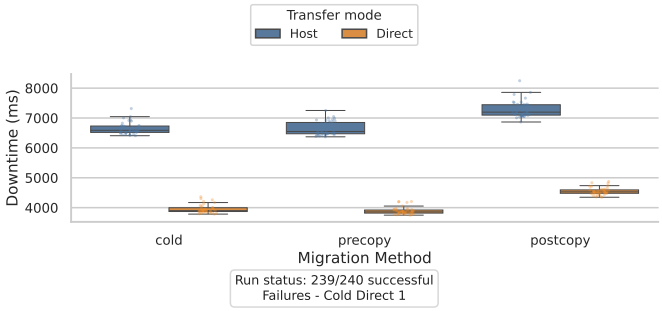


Figure B.35.: Counter downtime comparison under LTE.

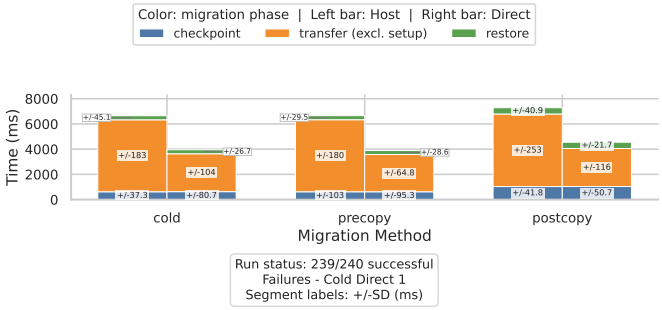


Figure B.36.: Counter phase breakdown under LTE.

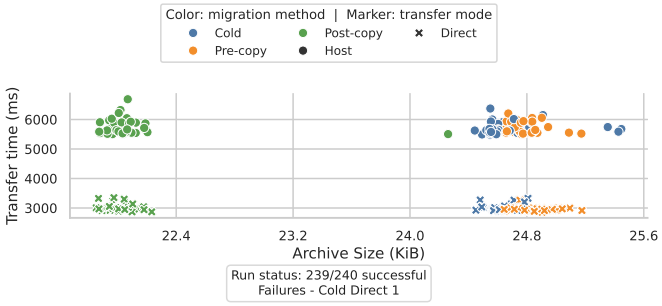


Figure B.37.: Counter transfer size and transfer time under LTE.

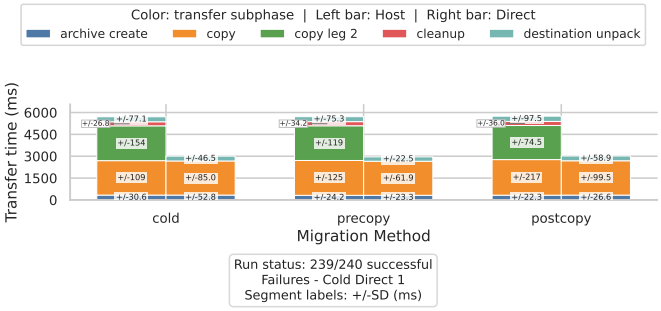


Figure B.38.: Counter transfer phase breakdown under LTE.

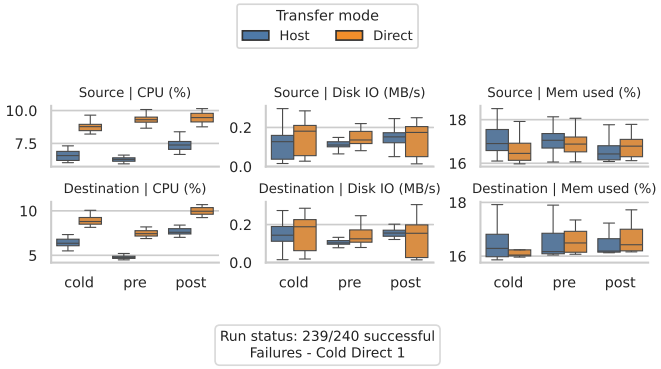


Figure B.39.: Counter per-node resource usage under LTE.

B.3.4. Starlink

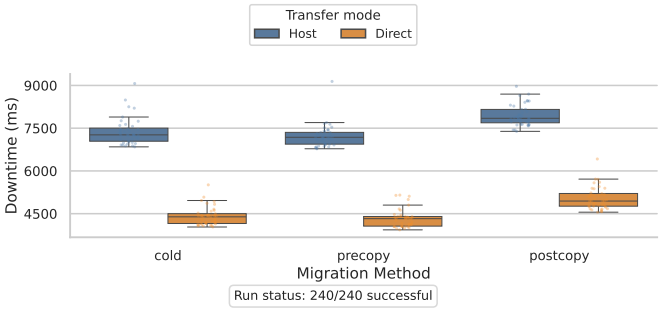


Figure B.40.: Counter downtime comparison under Starlink.

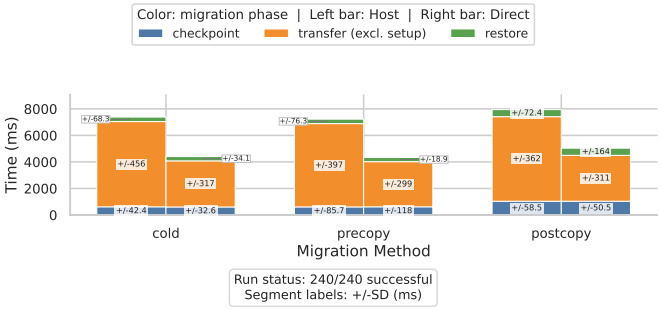


Figure B.41.: Counter phase breakdown under Starlink.

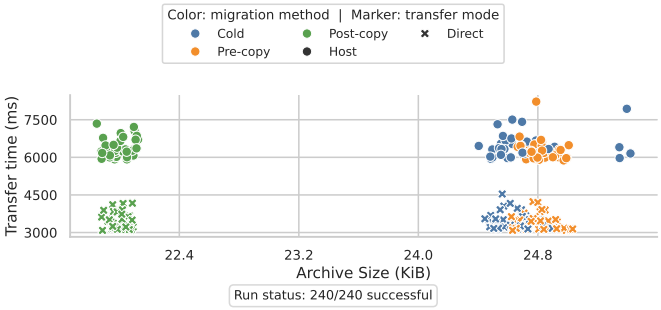


Figure B.42.: Counter transfer size and transfer time under Starlink.

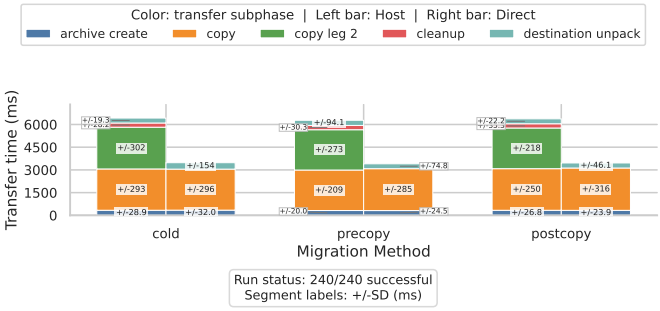


Figure B.43.: Counter transfer phase breakdown under Starlink.

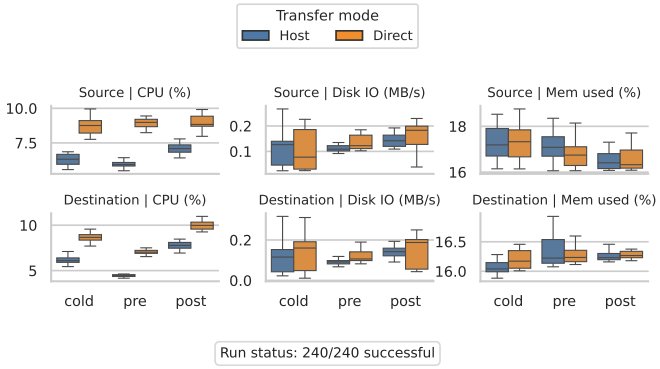


Figure B.44.: Counter per-node resource usage under Starlink.

B.3.5. TEE Best

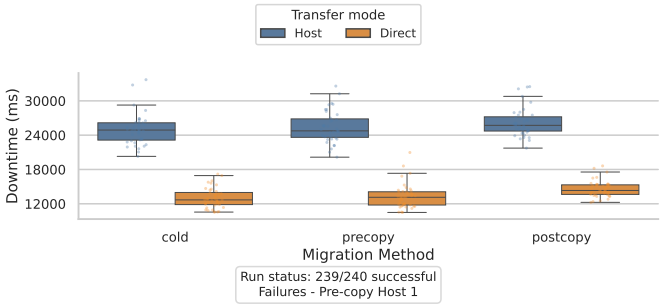


Figure B.45.: Counter downtime comparison under TEE Best.

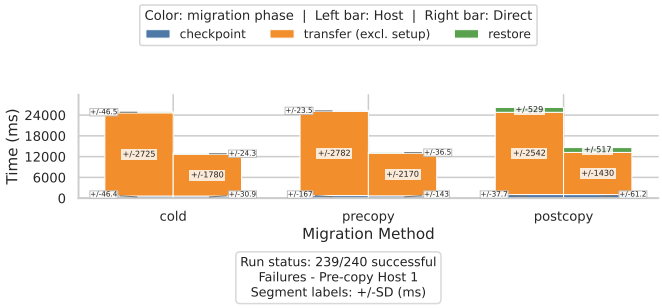


Figure B.46.: Counter phase breakdown under TEE Best.

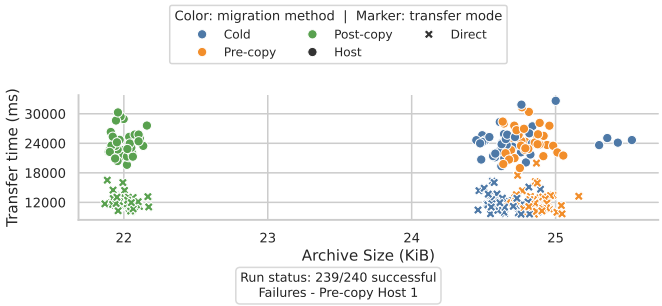


Figure B.47.: Counter transfer size and transfer time under TEE Best.

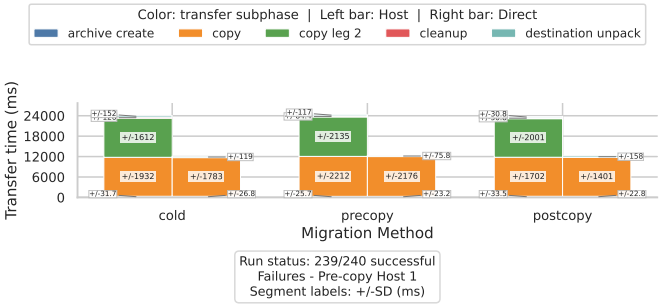


Figure B.48.: Counter transfer phase breakdown under TEE Best.

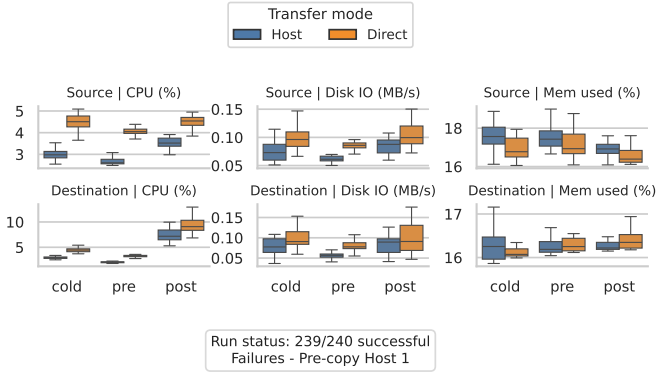


Figure B.49.: Counter per-node resource usage under TEE Best.

B.3.6. TEE Avg

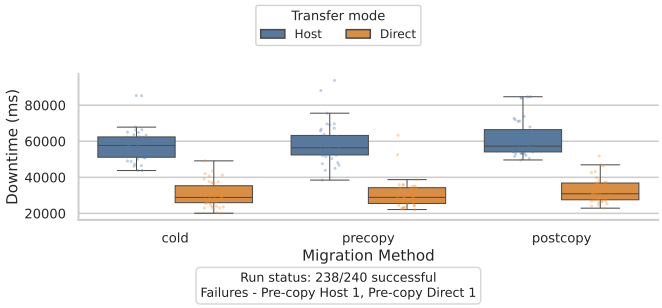


Figure B.50.: Counter downtime comparison under TEE Avg.

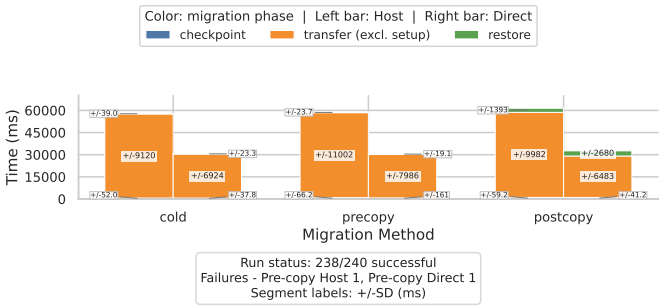


Figure B.51.: Counter phase breakdown under TEE Avg.

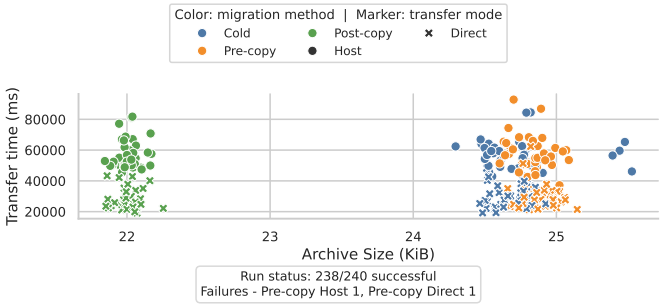


Figure B.52.: Counter transfer size and transfer time under TEE Avg.

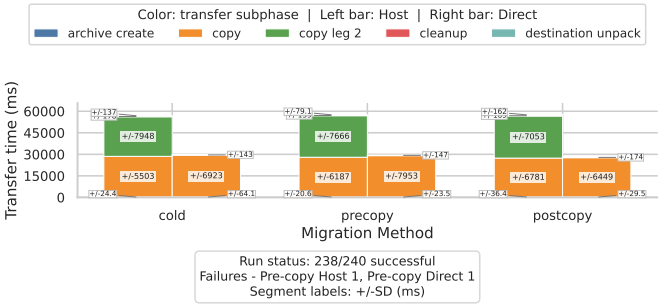


Figure B.53.: Counter transfer phase breakdown under TEE Avg.

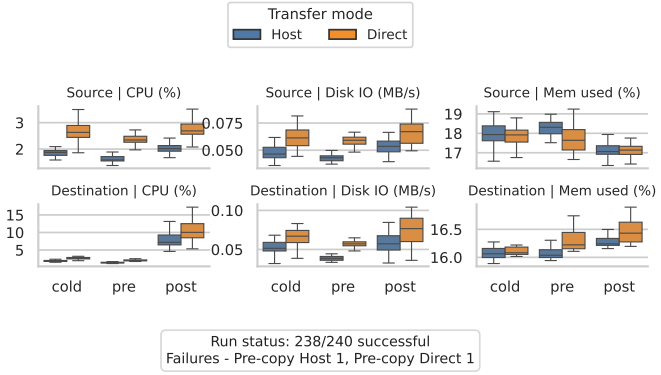


Figure B.54.: Counter per-node resource usage under TEE Avg.

B.3.7. TEE Worst

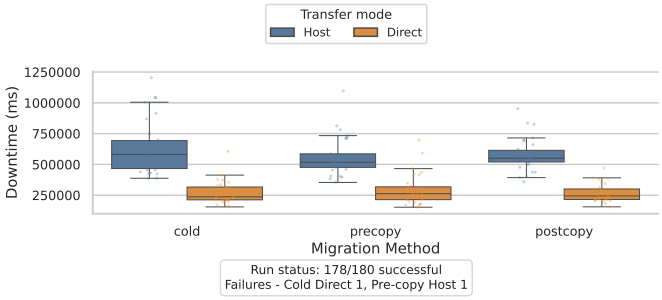


Figure B.55.: Counter downtime comparison under TEE Worst.

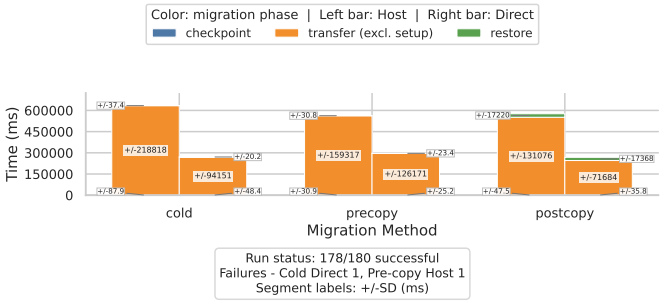


Figure B.56.: Counter phase breakdown under TEE Worst.

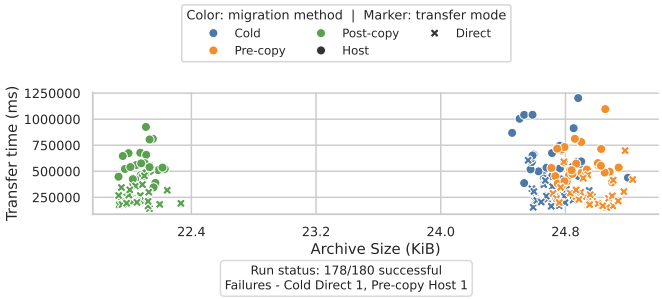


Figure B.57.: Counter transfer size and transfer time under TEE Worst.

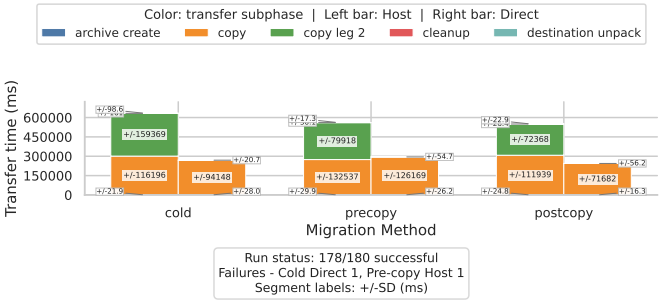


Figure B.58.: Counter transfer phase breakdown under TEE Worst.

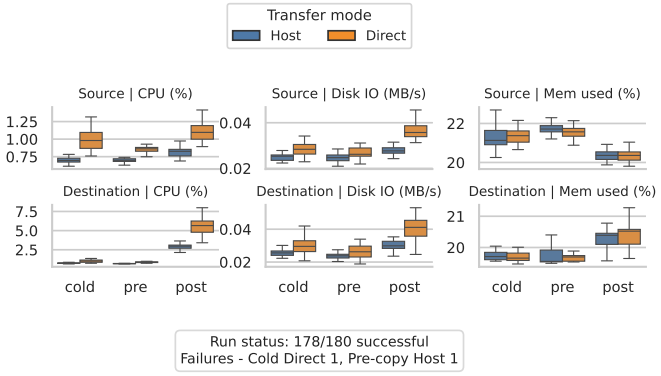


Figure B.59.: Counter per-node resource usage under TEE Worst.

B.4. TCP Client Plots

These figures show the native CRIU workload that preserves an established TCP connection. Each available profile includes downtime, phase, transfer, transfer-breakdown, and per-node resource plots.

B.4.1. WiFi 6

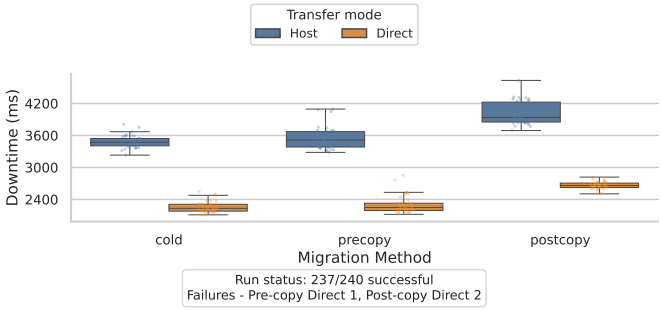


Figure B.60.: TCP Client downtime comparison under WiFi 6.

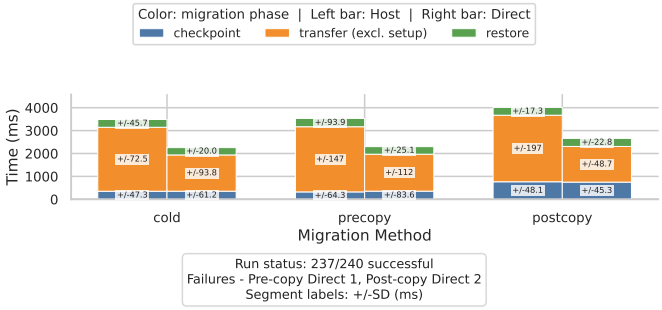


Figure B.61.: TCP Client phase breakdown under WiFi 6.

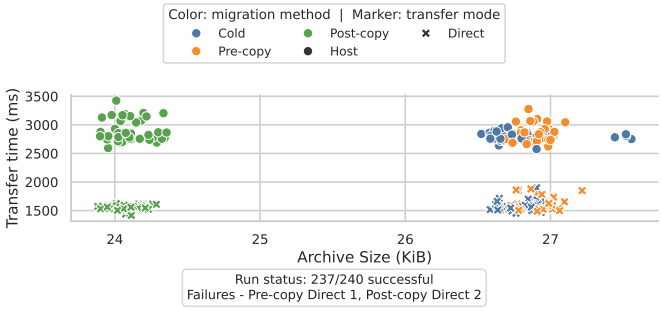


Figure B.62.: TCP Client transfer size and transfer time under WiFi 6.

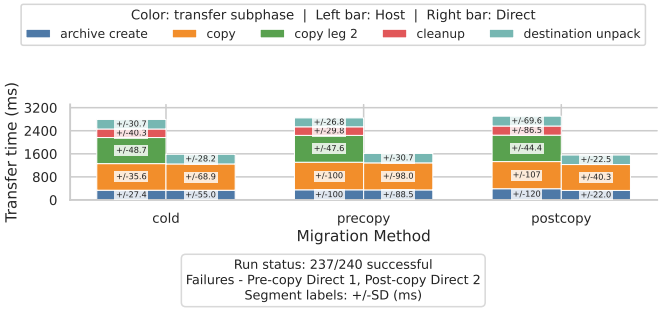


Figure B.63.: TCP Client transfer phase breakdown under WiFi 6.

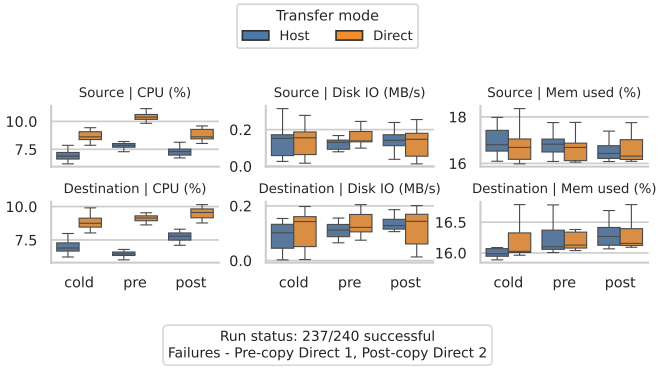


Figure B.64.: TCP Client per-node resource usage under WiFi 6.

B.4.2. 5G

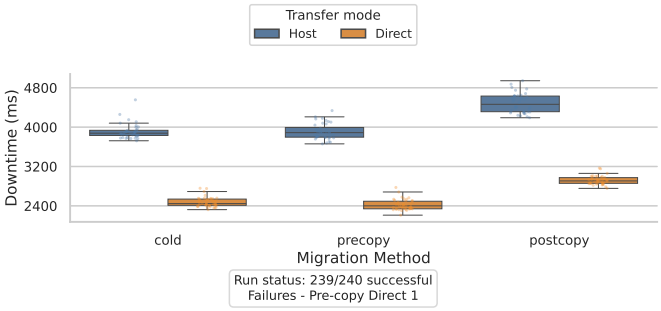


Figure B.65.: TCP Client downtime comparison under 5G.

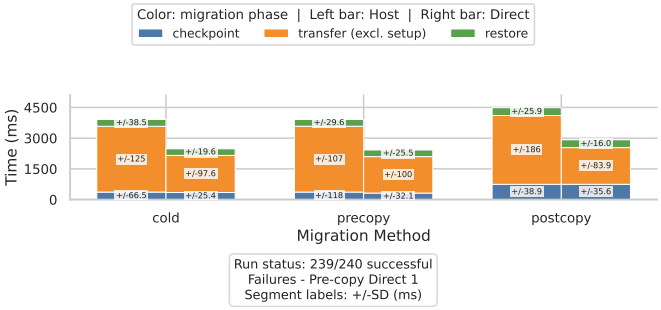


Figure B.66.: TCP Client phase breakdown under 5G.

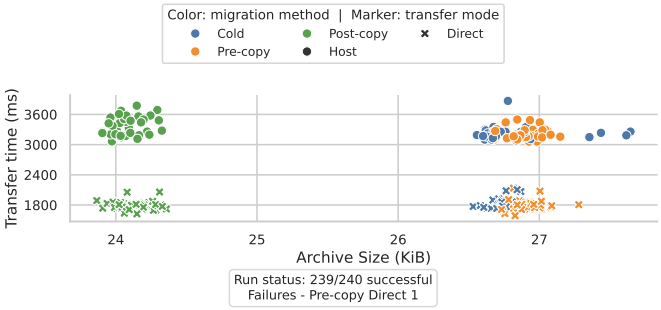


Figure B.67.: TCP Client transfer size and transfer time under 5G.

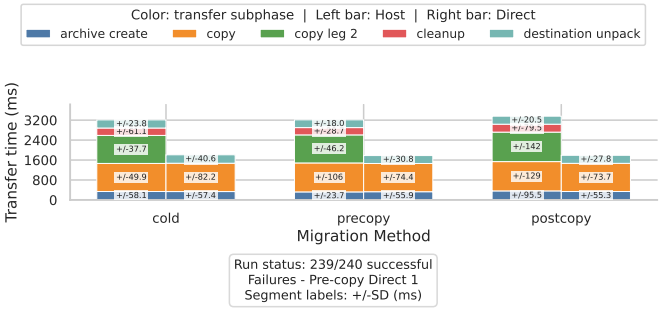


Figure B.68.: TCP Client transfer phase breakdown under 5G.

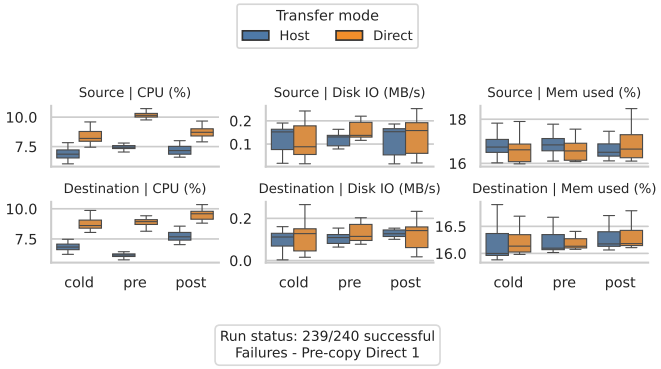


Figure B.69.: TCP Client per-node resource usage under 5G.

B.4.3. LTE

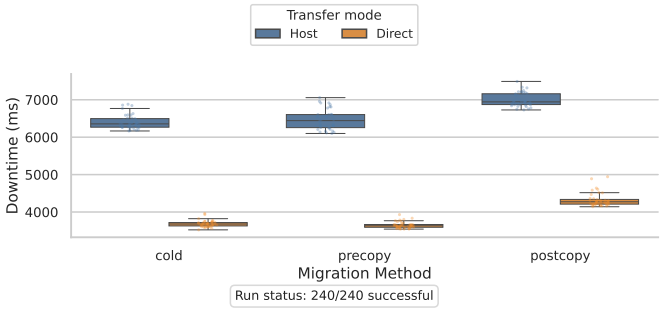


Figure B.70.: TCP Client downtime comparison under LTE.

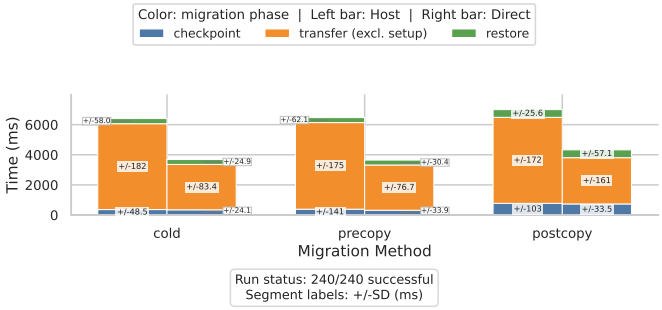


Figure B.71.: TCP Client phase breakdown under LTE.

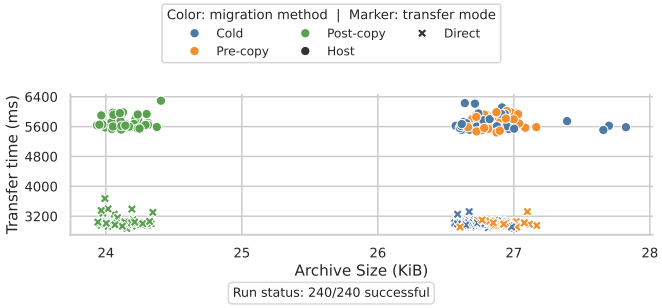


Figure B.72.: TCP Client transfer size and transfer time under LTE.

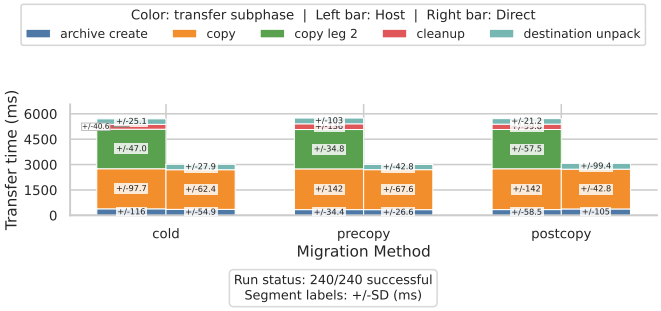


Figure B.73.: TCP Client transfer phase breakdown under LTE.

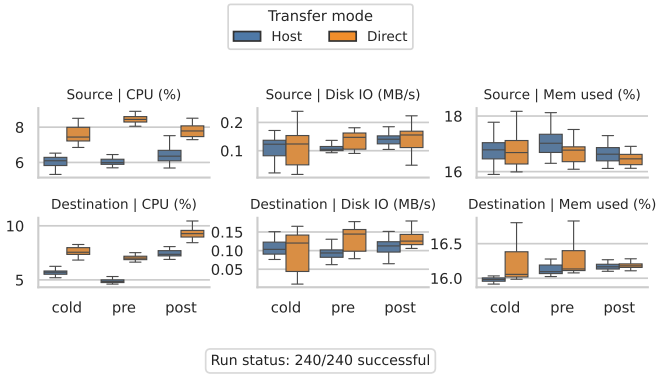


Figure B.74.: TCP Client per-node resource usage under LTE.

B.4.4. Starlink

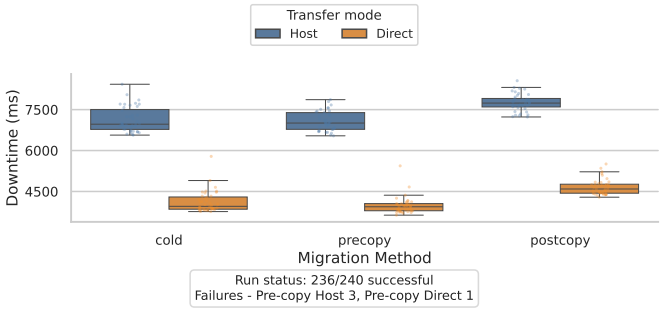


Figure B.75.: TCP Client downtime comparison under Starlink.

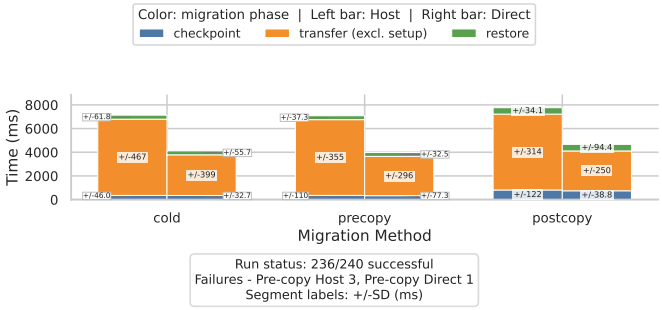


Figure B.76.: TCP Client phase breakdown under Starlink.

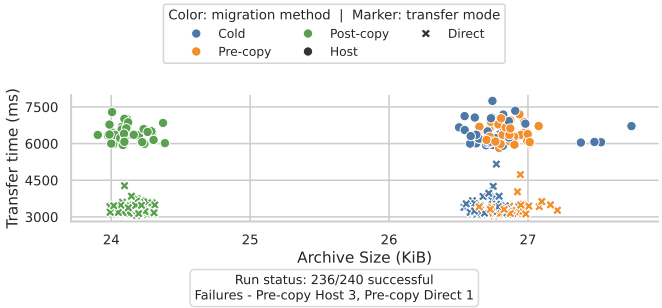


Figure B.77.: TCP Client transfer size and transfer time under Starlink.

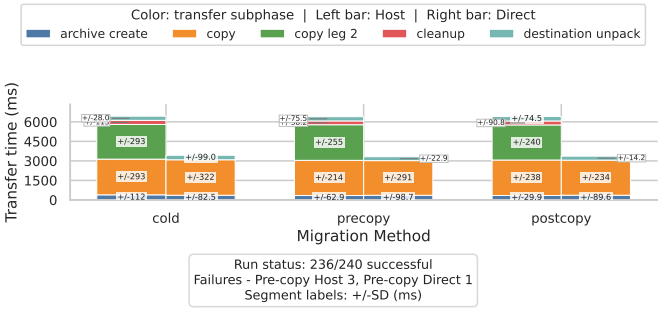


Figure B.78.: TCP Client transfer phase breakdown under Starlink.

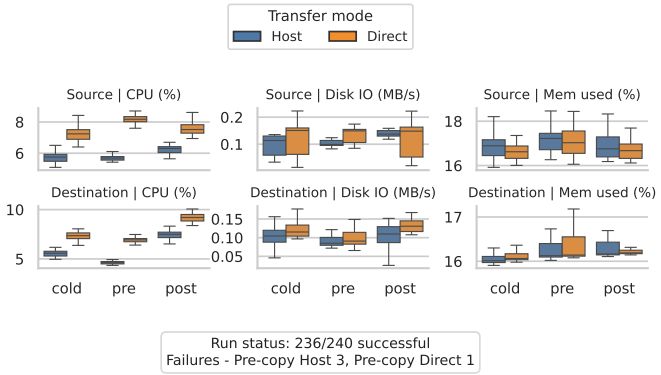


Figure B.79.: TCP Client per-node resource usage under Starlink.

B.4.5. TEE Best

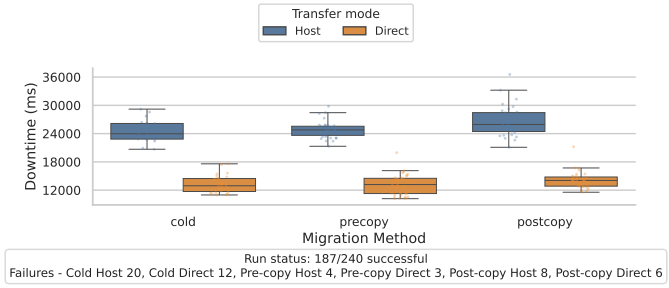


Figure B.80.: TCP Client downtime comparison under TEE Best.

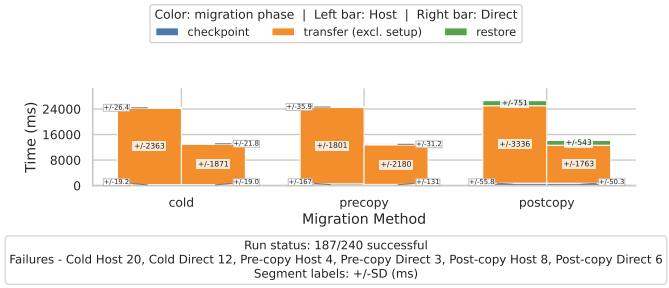


Figure B.81.: TCP Client phase breakdown under TEE Best.

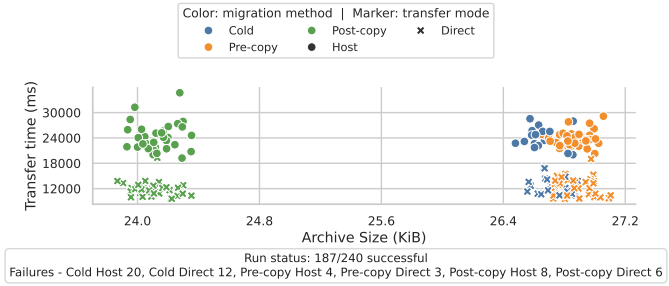


Figure B.82.: TCP Client transfer size and transfer time under TEE Best.

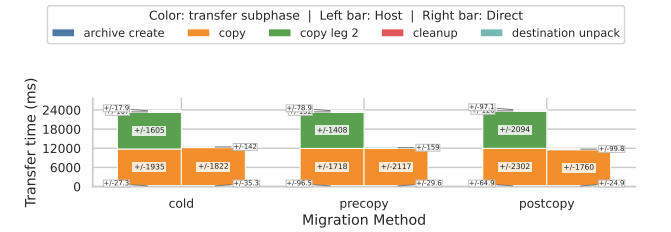


Figure B.83.: TCP Client transfer phase breakdown under TEE Best.

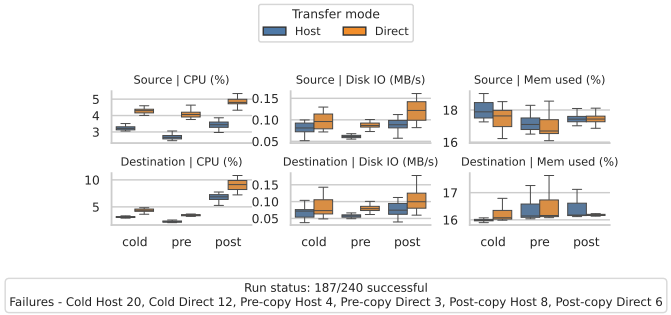


Figure B.84.: TCP Client per-node resource usage under TEE Best.

B.4.6. TEE Avg

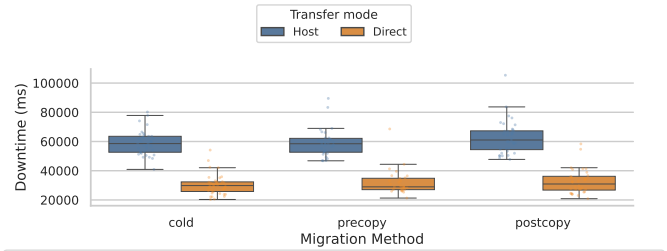


Figure B.85.: TCP Client downtime comparison under TEE Avg.

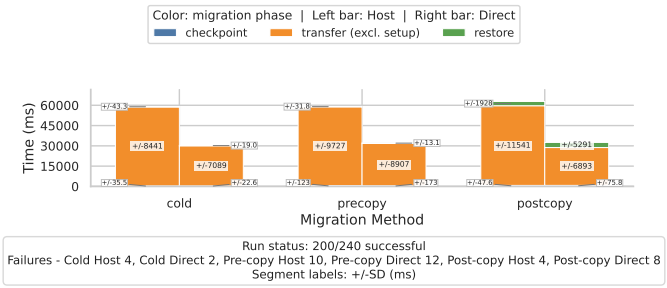


Figure B.86.: TCP Client phase breakdown under TEE Avg.

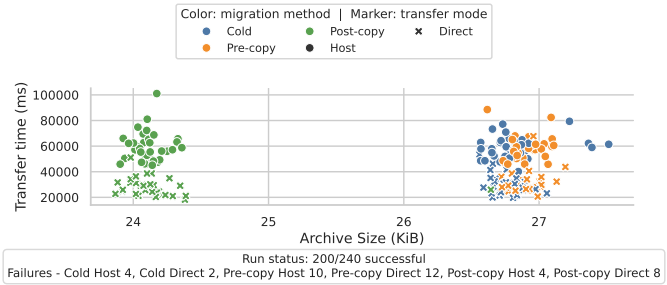


Figure B.87.: TCP Client transfer size and transfer time under TEE Avg.

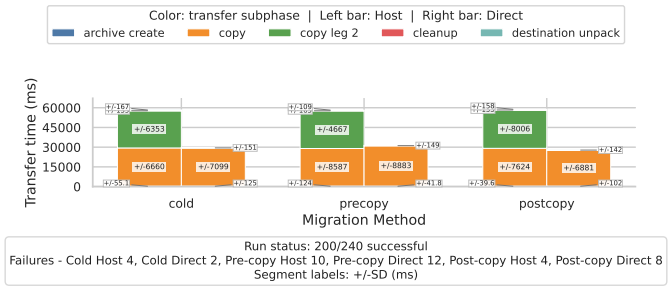


Figure B.88.: TCP Client transfer phase breakdown under TEE Avg.

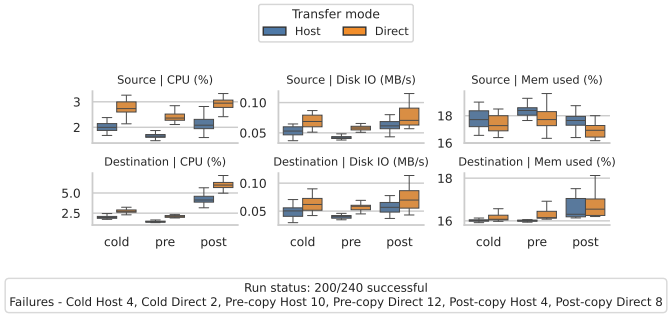


Figure B.89.: TCP Client per-node resource usage under TEE Avg.

B.4.7. TEE Worst

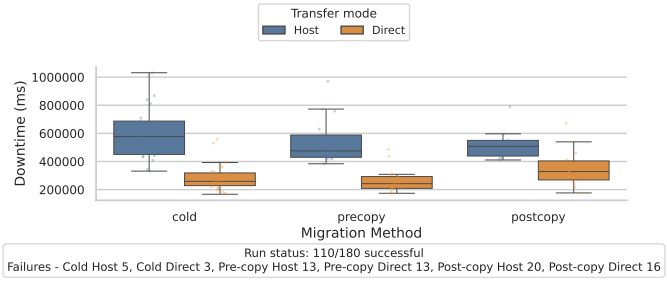


Figure B.90.: TCP Client downtime comparison under TEE Worst.

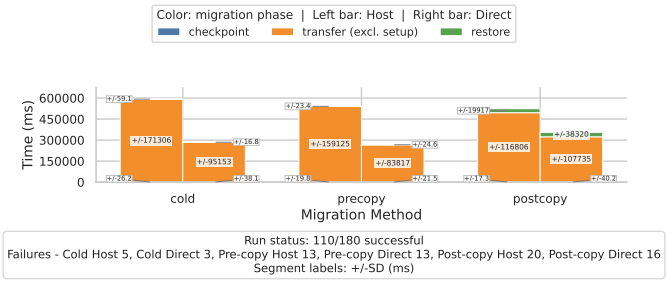


Figure B.91.: TCP Client phase breakdown under TEE Worst.

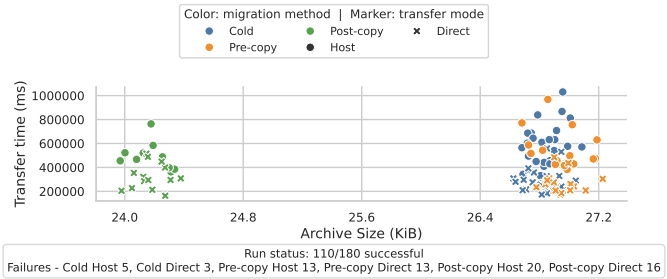


Figure B.92.: TCP Client transfer size and transfer time under TEE Worst.

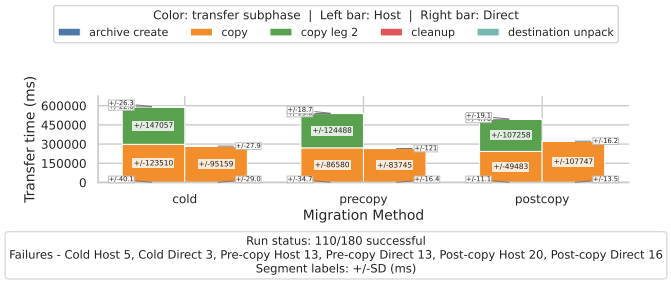


Figure B.93.: TCP Client transfer phase breakdown under TEE Worst.

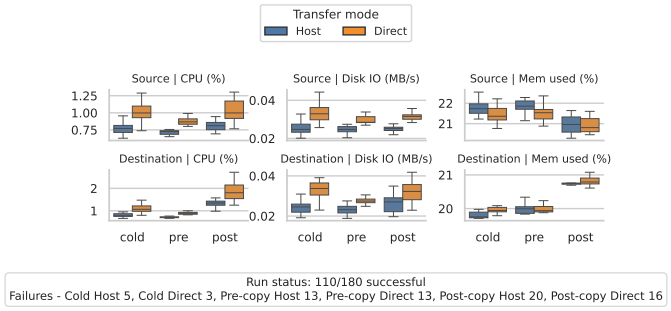


Figure B.94.: TCP Client per-node resource usage under TEE Worst.

B.5. Game of Life Plots

These figures show the largest native CRIU workload. Each available profile includes downtime, phase, transfer, transfer-breakdown, and per-node resource plots.

B.5.1. WiFi 6

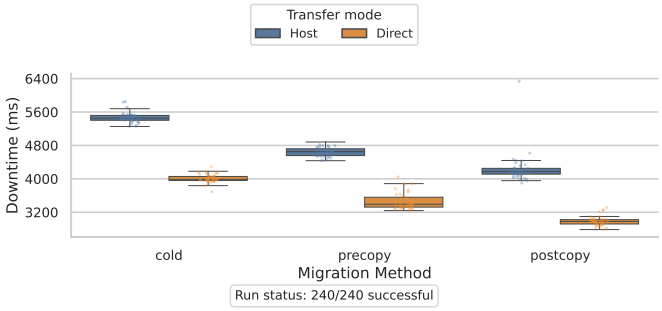


Figure B.95.: Game of Life downtime comparison under WiFi 6.

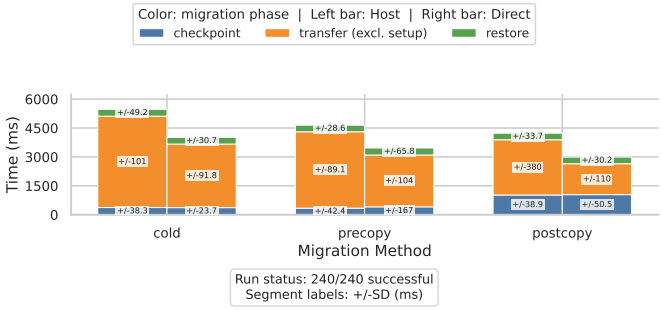


Figure B.96.: Game of Life phase breakdown under WiFi 6.

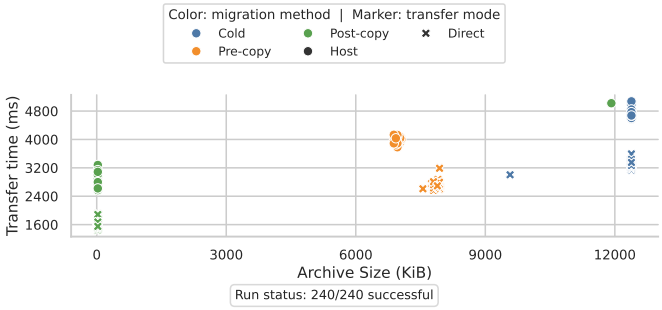


Figure B.97.: Game of Life transfer size and transfer time under WiFi 6.

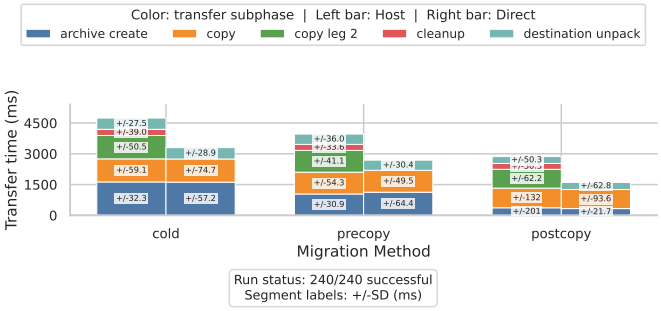


Figure B.98.: Game of Life transfer phase breakdown under WiFi 6.

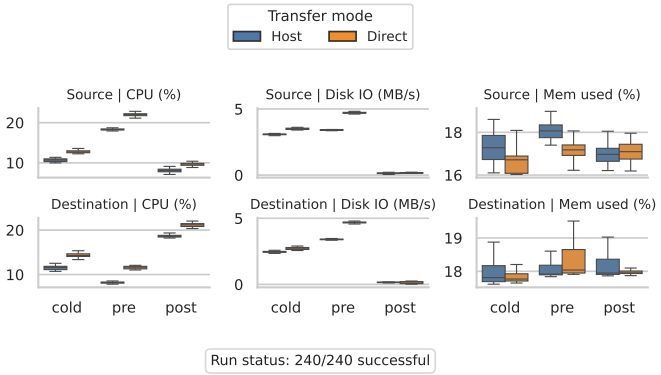


Figure B.99.: Game of Life per-node resource usage under WiFi 6.

B.5.2. 5G

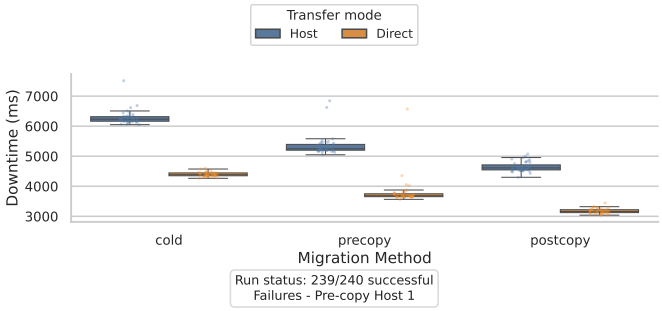


Figure B.100.: Game of Life downtime comparison under 5G.

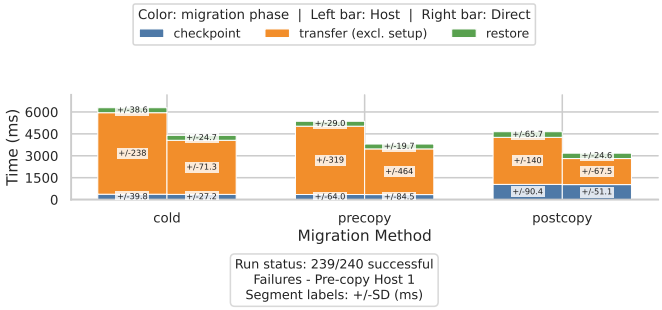


Figure B.101.: Game of Life phase breakdown under 5G.

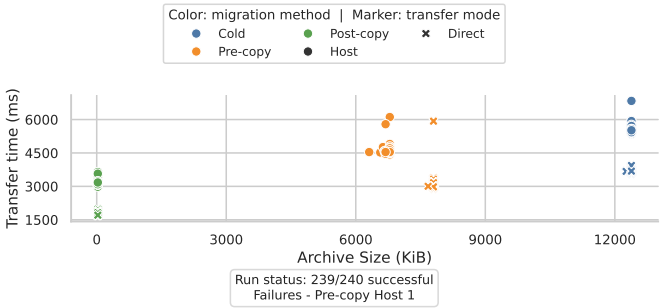


Figure B.102.: Game of Life transfer size and transfer time under 5G.

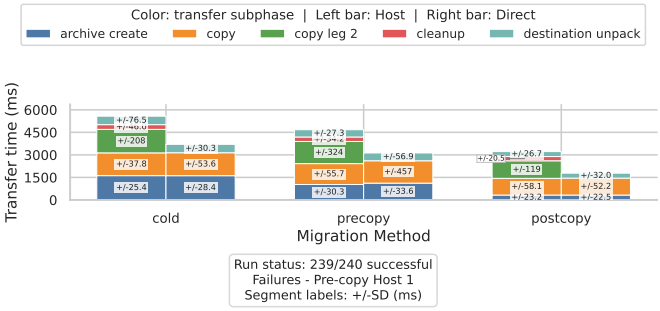


Figure B.103.: Game of Life transfer phase breakdown under 5G.

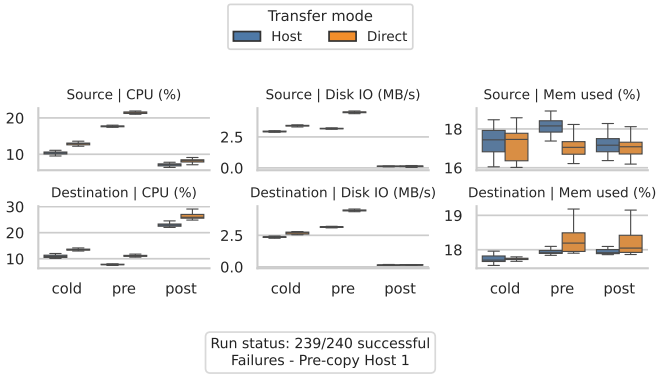


Figure B.104.: Game of Life per-node resource usage under 5G.

B.5.3. LTE

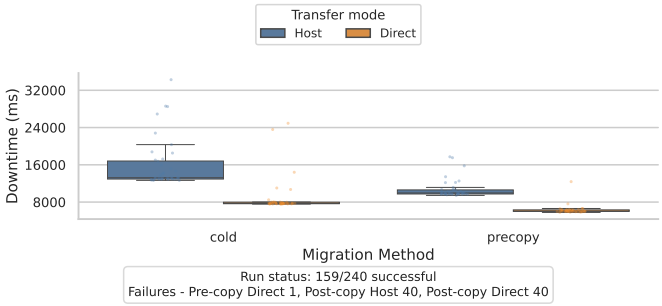


Figure B.105.: Game of Life downtime comparison under LTE.

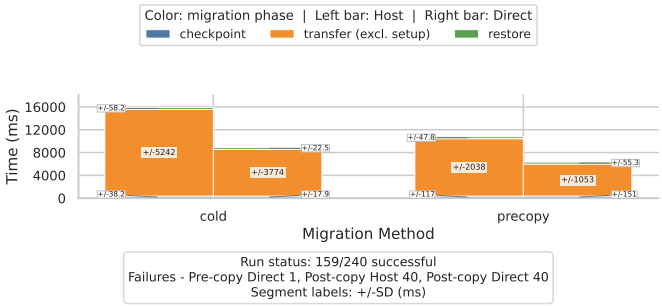


Figure B.106.: Game of Life phase breakdown under LTE.

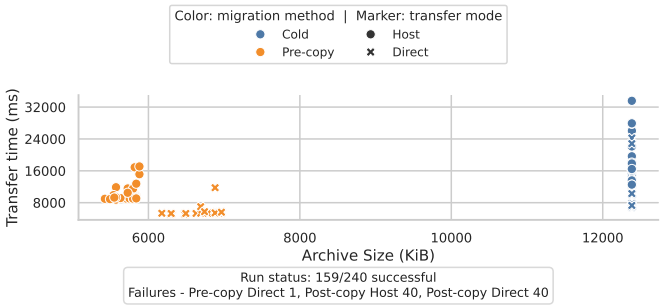


Figure B.107.: Game of Life transfer size and transfer time under LTE.

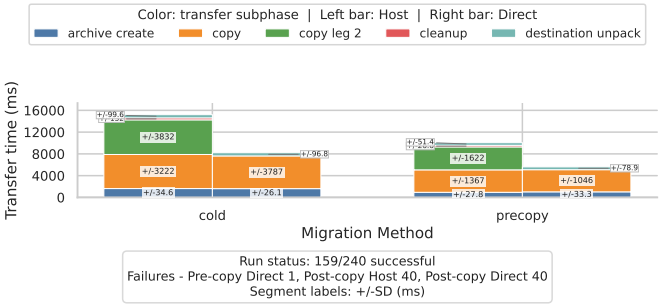


Figure B.108.: Game of Life transfer phase breakdown under LTE.

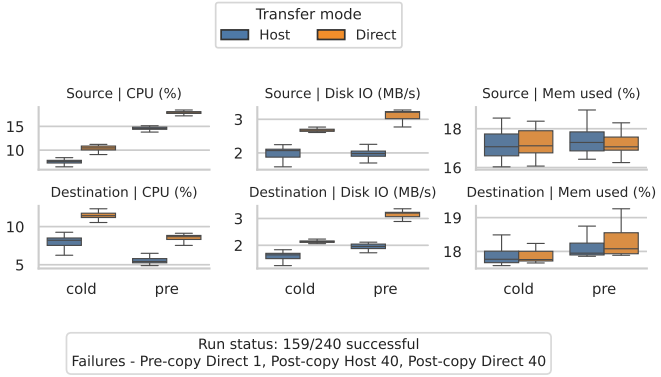


Figure B.109.: Game of Life per-node resource usage under LTE.

B.5.4. Starlink

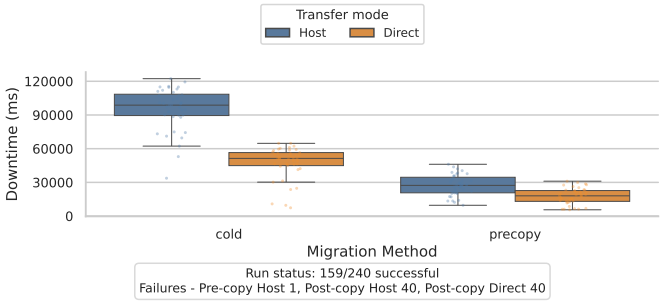


Figure B.110.: Game of Life downtime comparison under Starlink.

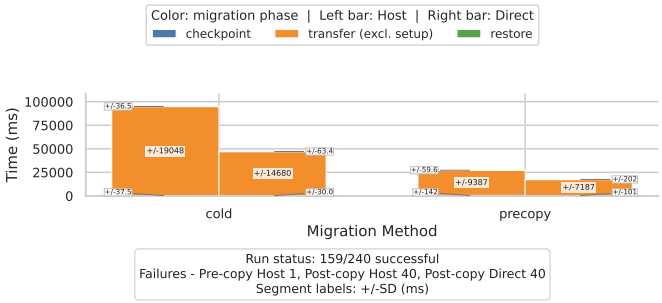


Figure B.111.: Game of Life phase breakdown under Starlink.

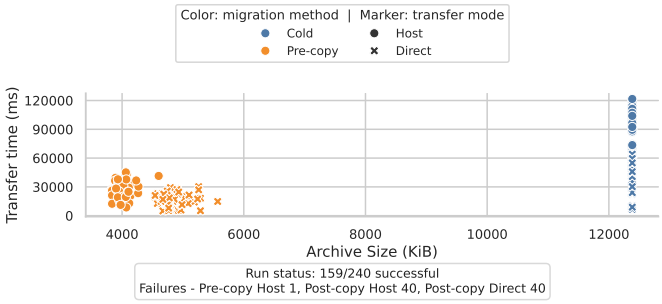


Figure B.112.: Game of Life transfer size and transfer time under Starlink.

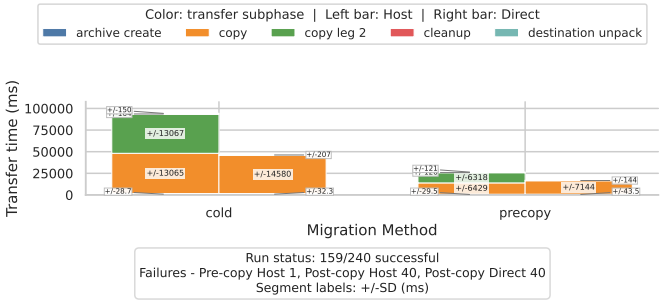


Figure B.113.: Game of Life transfer phase breakdown under Starlink.

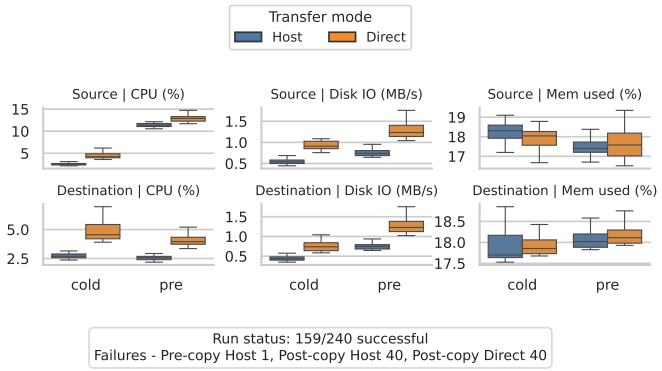


Figure B.114.: Game of Life per-node resource usage under Starlink.

B.5.5. TEE Best

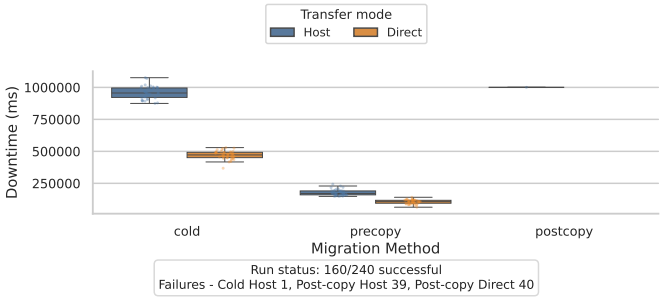


Figure B.115.: Game of Life downtime comparison under TEE Best.

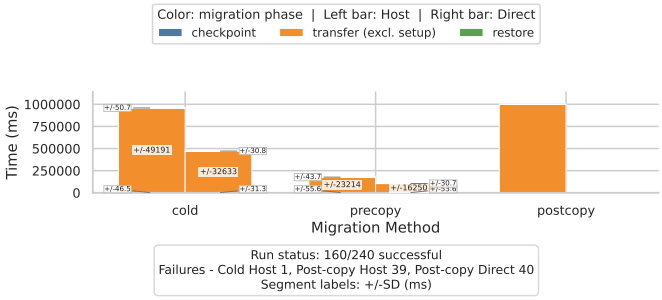


Figure B.116.: Game of Life phase breakdown under TEE Best.

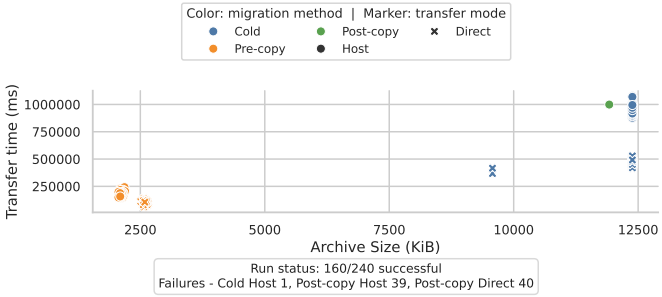


Figure B.117.: Game of Life transfer size and transfer time under TEE Best.

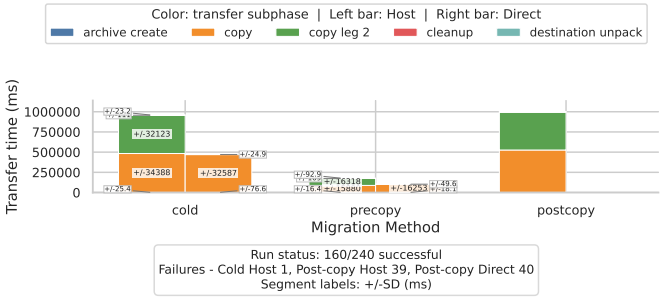


Figure B.118.: Game of Life transfer phase breakdown under TEE Best.

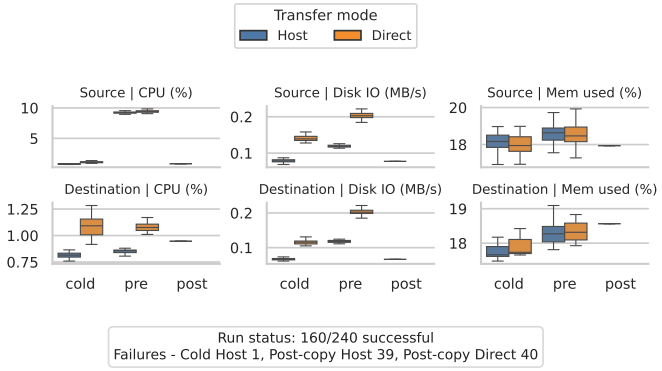


Figure B.119.: Game of Life per-node resource usage under TEE Best.

B.5.6. TEE Avg

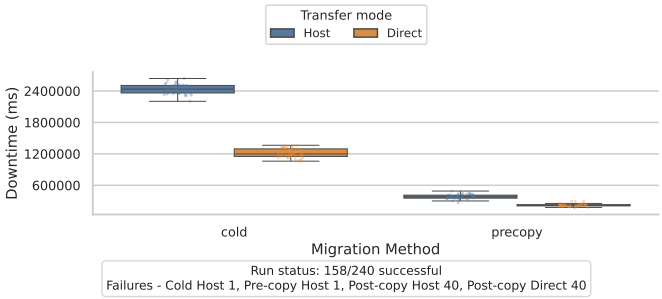


Figure B.120.: Game of Life downtime comparison under TEE Avg.

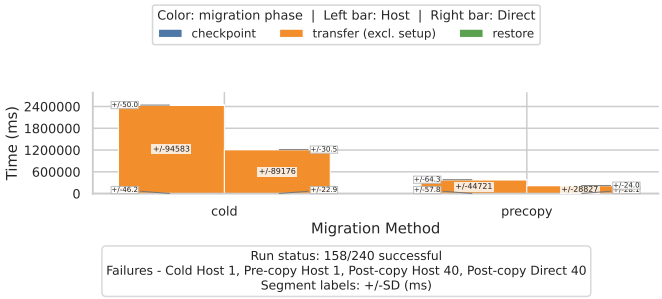


Figure B.121.: Game of Life phase breakdown under TEE Avg.

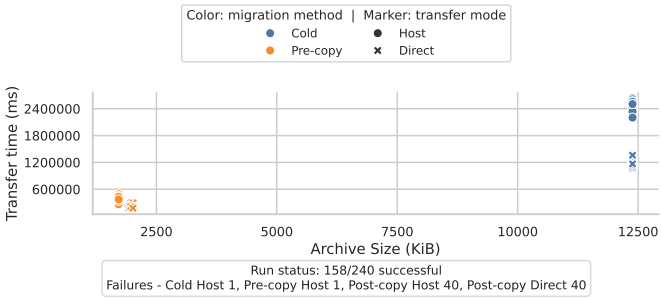


Figure B.122.: Game of Life transfer size and transfer time under TEE Avg.

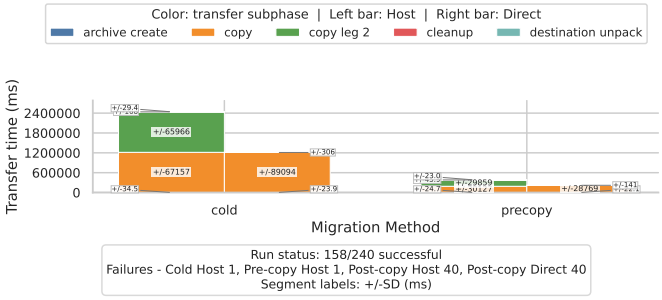


Figure B.123.: Game of Life transfer phase breakdown under TEE Avg.

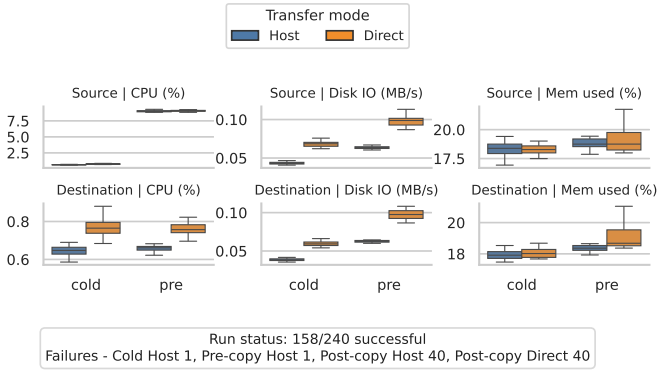


Figure B.124.: Game of Life per-node resource usage under TEE Avg.

B.6. Wasm Plots

These figures show the application-aware Wasm migration workload. Each available profile includes downtime, phase, transfer, transfer-breakdown, and per-node resource plots.

B.6.1. WiFi 6

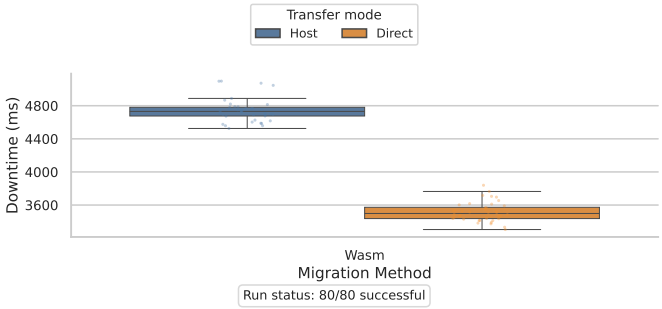


Figure B.125.: Wasm downtime comparison under WiFi 6.

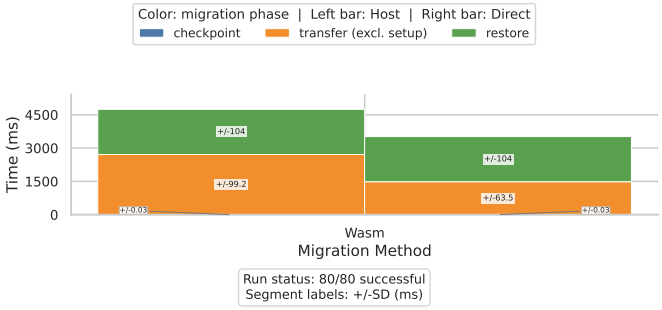


Figure B.126.: Wasm phase breakdown under WiFi 6.

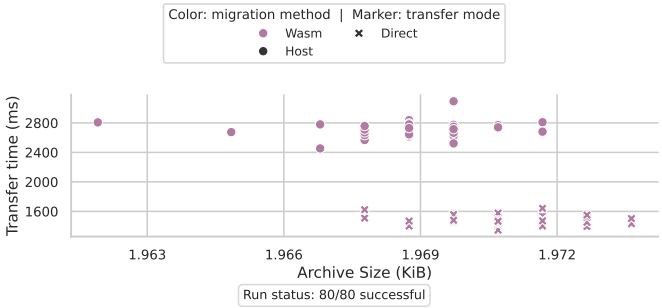


Figure B.127.: Wasm transfer size and transfer time under WiFi 6.

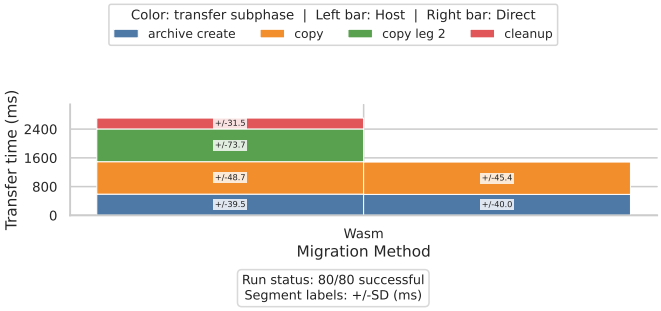


Figure B.128.: Wasm transfer phase breakdown under WiFi 6.

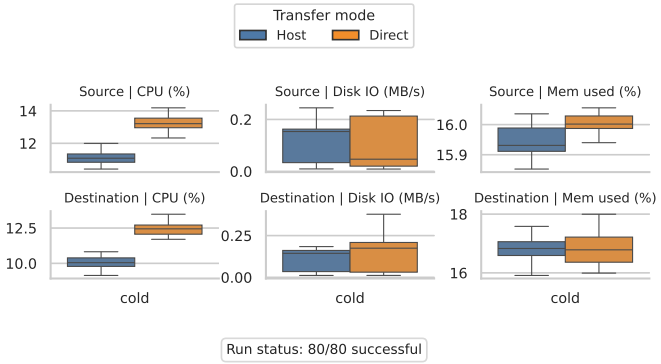


Figure B.129.: Wasm per-node resource usage under WiFi 6.

B.6.2. 5G

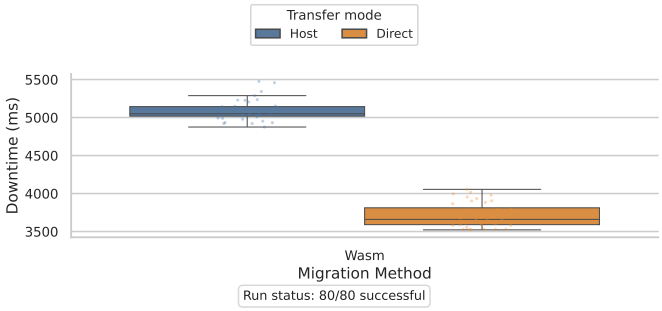


Figure B.130.: Wasm downtime comparison under 5G.

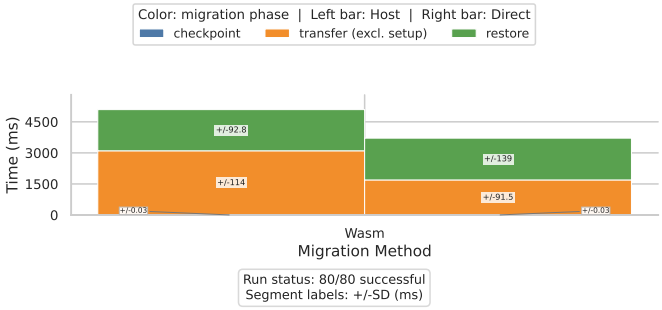


Figure B.131.: Wasm phase breakdown under 5G.

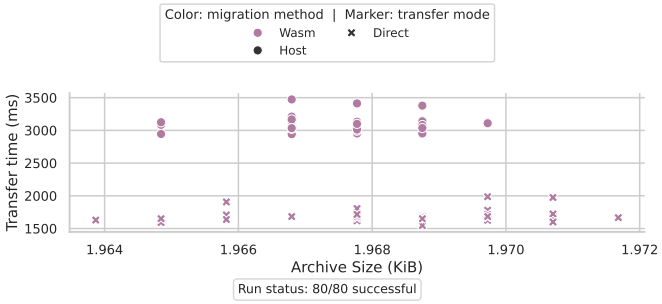


Figure B.132.: Wasm transfer size and transfer time under 5G.

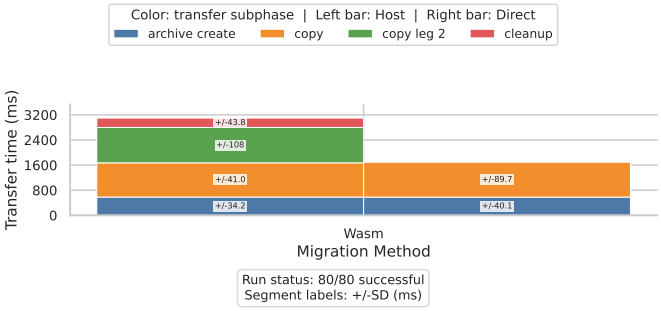


Figure B.133.: Wasm transfer phase breakdown under 5G.

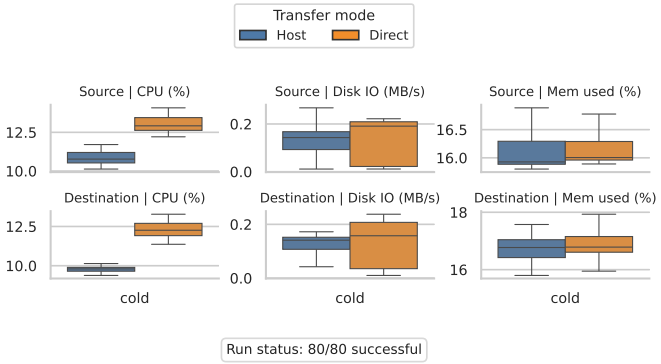


Figure B.134.: Wasm per-node resource usage under 5G.

B.6.3. LTE

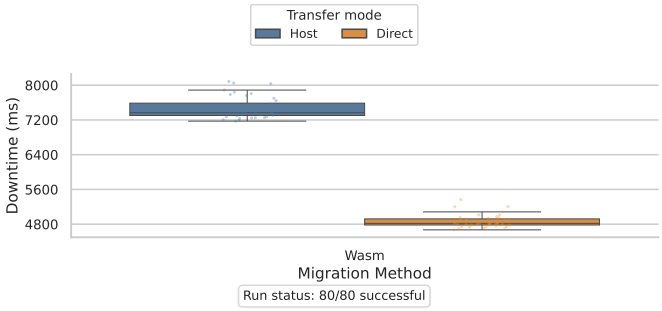


Figure B.135.: Wasm downtime comparison under LTE.

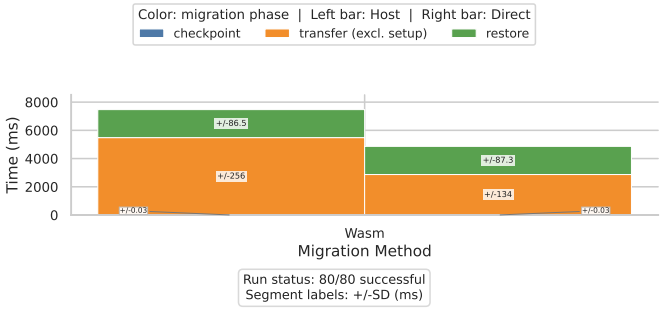


Figure B.136.: Wasm phase breakdown under LTE.

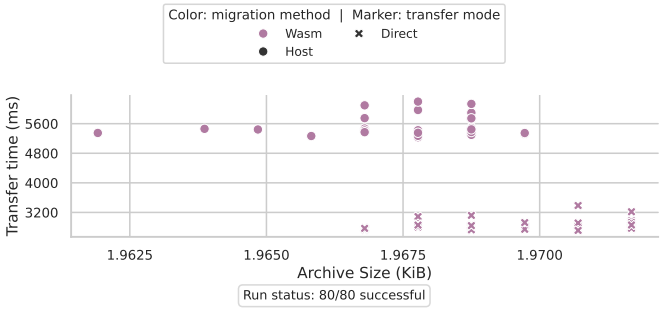


Figure B.137.: Wasm transfer size and transfer time under LTE.

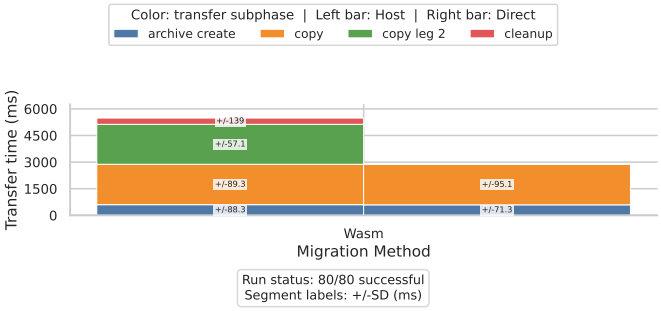


Figure B.138.: Wasm transfer phase breakdown under LTE.

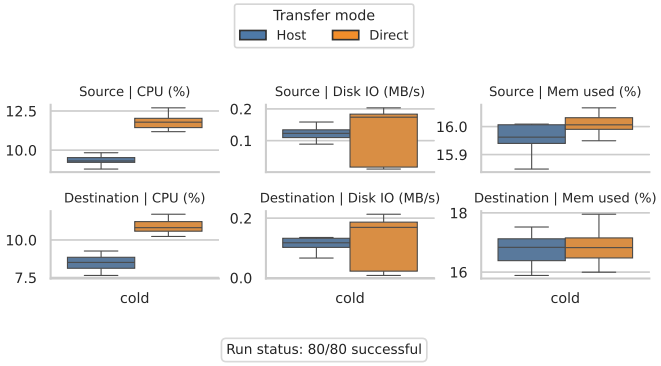


Figure B.139.: Wasm per-node resource usage under LTE.

B.6.4. Starlink

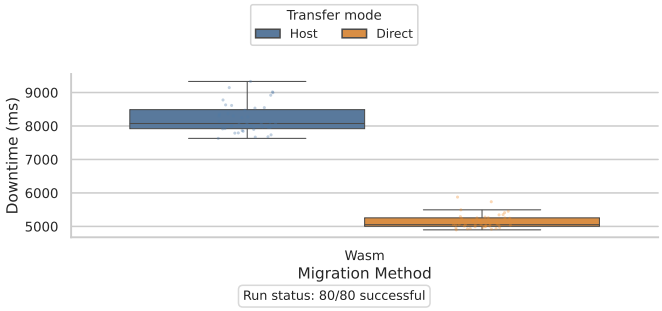


Figure B.140.: Wasm downtime comparison under Starlink.

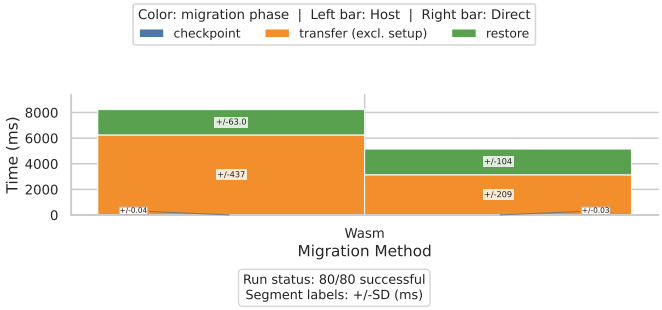


Figure B.141.: Wasm phase breakdown under Starlink.

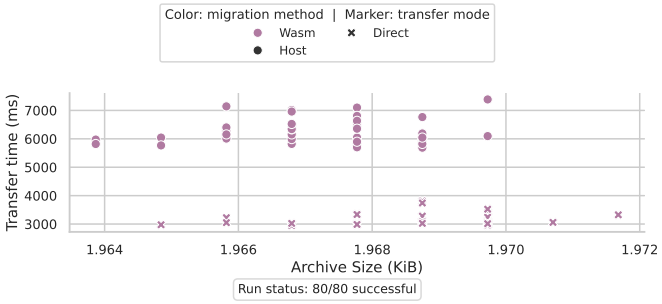


Figure B.142.: Wasm transfer size and transfer time under Starlink.

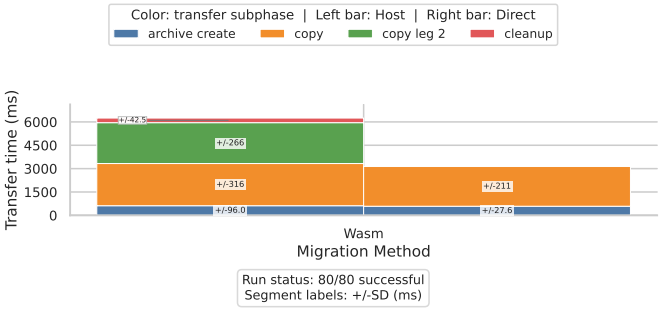


Figure B.143.: Wasm transfer phase breakdown under Starlink.

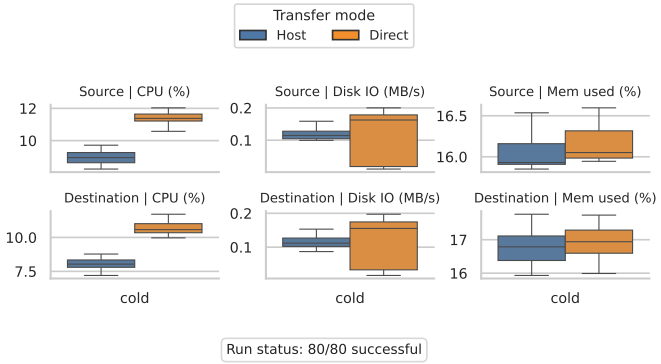


Figure B.144.: Wasm per-node resource usage under Starlink.

B.6.5. TEE Best

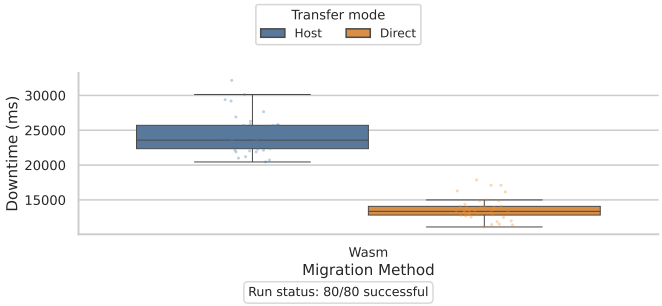


Figure B.145.: Wasm downtime comparison under TEE Best.

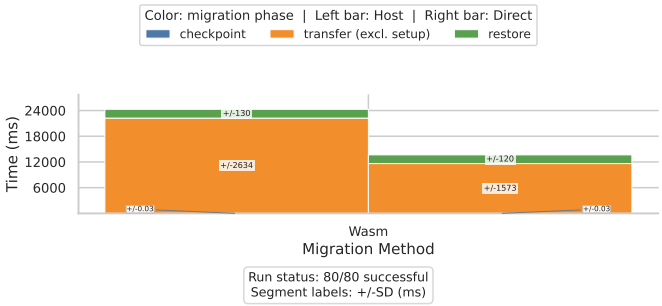


Figure B.146.: Wasm phase breakdown under TEE Best.

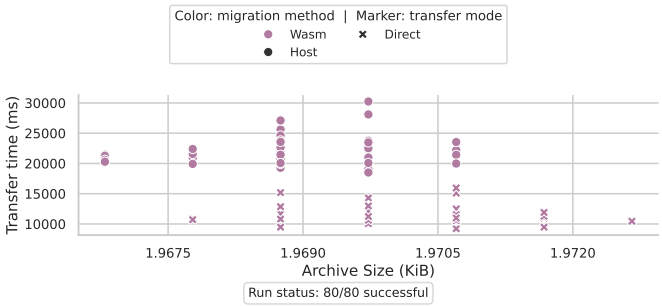


Figure B.147.: Wasm transfer size and transfer time under TEE Best.

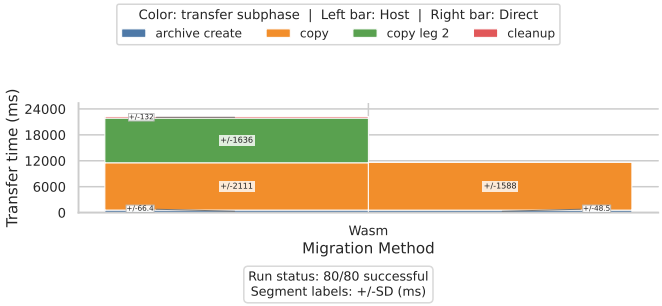


Figure B.148.: Wasm transfer phase breakdown under TEE Best.

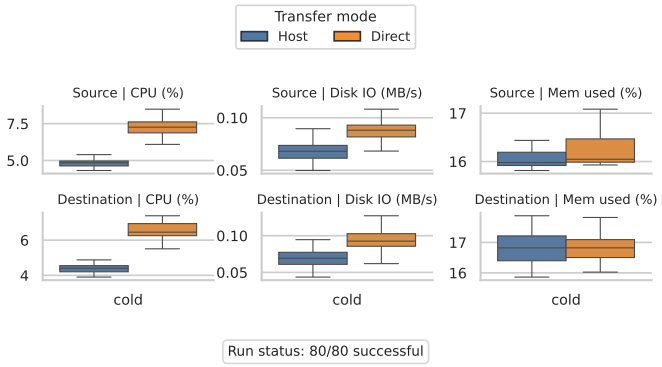


Figure B.149.: Wasm per-node resource usage under TEE Best.

B.6.6. TEE Avg

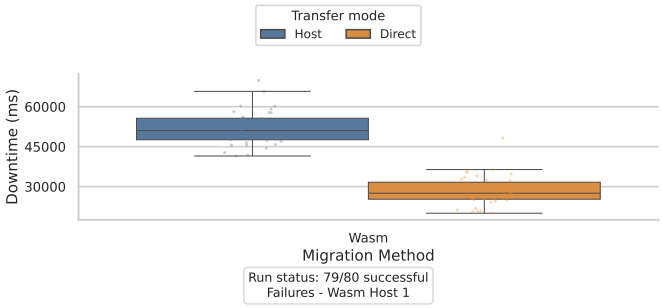


Figure B.150.: Wasm downtime comparison under TEE Avg.

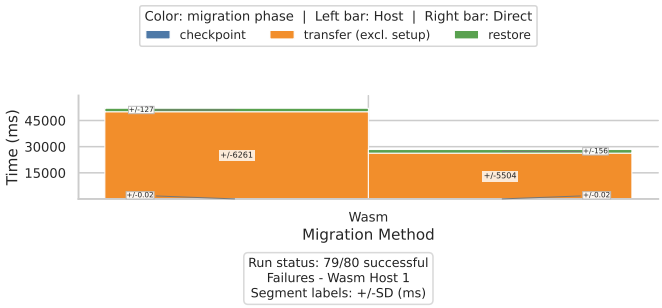


Figure B.151.: Wasm phase breakdown under TEE Avg.

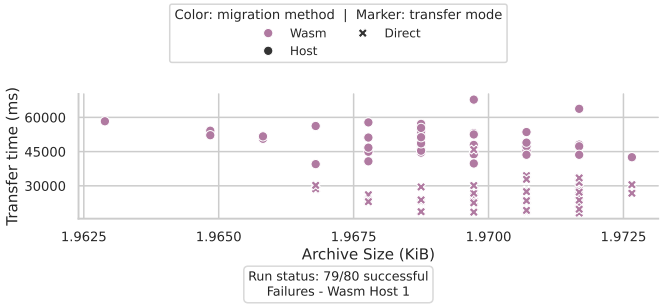


Figure B.152.: Wasm transfer size and transfer time under TEE Avg.

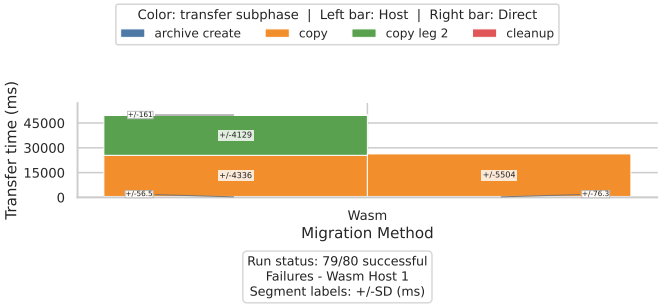


Figure B.153.: Wasm transfer phase breakdown under TEE Avg.

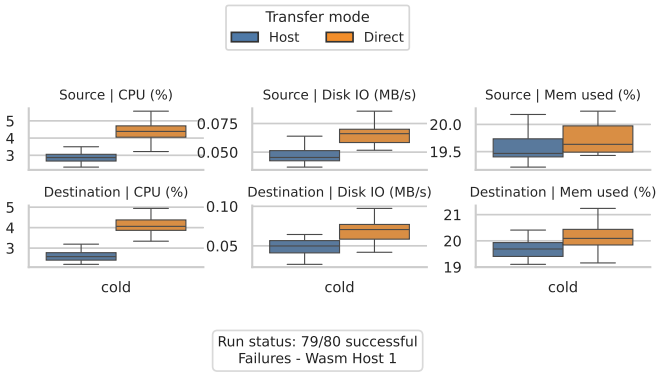


Figure B.154.: Wasm per-node resource usage under TEE Avg.

B.6.7. TEE Worst

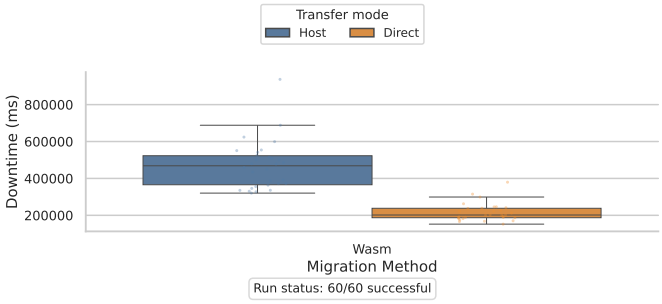


Figure B.155.: Wasm downtime comparison under TEE Worst.

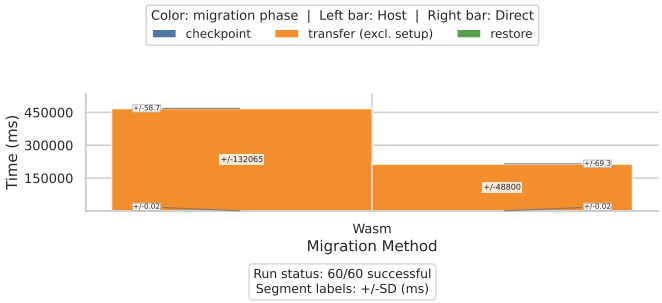


Figure B.156.: Wasm phase breakdown under TEE Worst.

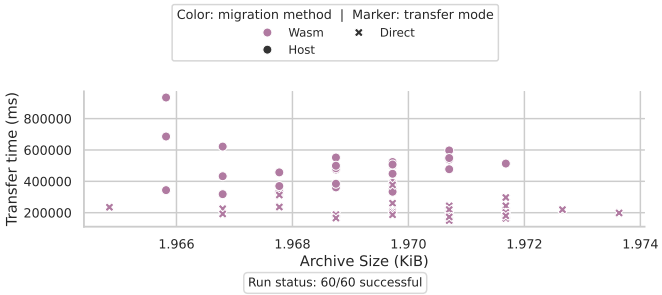


Figure B.157.: Wasm transfer size and transfer time under TEE Worst.

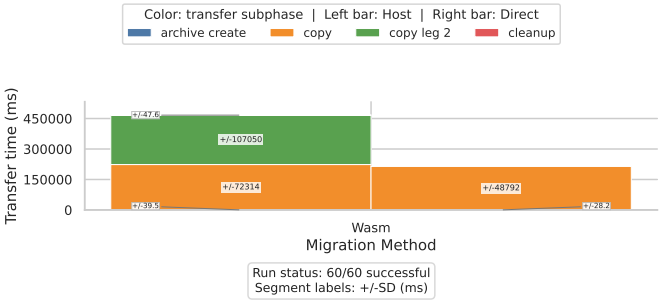


Figure B.158.: Wasm transfer phase breakdown under TEE Worst.

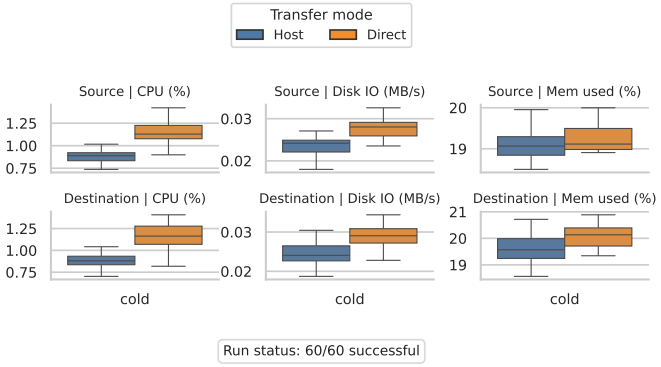


Figure B.159.: Wasm per-node resource usage under TEE Worst.

Abbreviations

CRIU Checkpoint/Restore In Userspace

TEE Tactical Edge Environment

VM Virtual Machine

Wasm WebAssembly

SSH Secure Shell

SCP Secure Copy Protocol

TCP Transmission Control Protocol

VIP Virtual IP address

PID Process Identifier

JSON JavaScript Object Notation

CDF Cumulative Distribution Function

CSV Comma-Separated Values

I/O Input/Output

LTE Long-Term Evolution

List of Figures

List of Tables

Bibliography

[1] W. Datema, H. Bastiaansen, A. Amato, M. Fogli, T. Kudla, J. van der Geest, P. Sanchez, N. Suri, and S. Watson. "Adaptive Computing in Federated Tactical Clouds." In: *Proceedings of the IST-208 Research Symposium on the Tactical Edge*. STO Meeting Proceedings STO-MP-IST-208. NATO Science and Technology Organization. 2024.

[2] W. Soussi, G. Gür, and B. Stiller. "Democratizing Container Live Migration for Enhanced Future Networks: A Survey." In: *ACM Computing Surveys* 57.4 (Dec. 2024), pp. 1–37. DOI: 10.1145/3704436.

[3] E. Tinto, L. Marchiori, and T. Vardanega. "Live Migration of Compiled Wasm Modules across the Compute Continuum." In: *Journal of Systems Architecture* 168 (2025), p. 103532. DOI: 10.1016/j.sysarc.2025.103532.

[4] HashiCorp. *Terraform*. Project website. URL: https://www.terraform.io/ (visited on 06/07/2026).

[5] Canonical. *Multipass*. Project website. URL: https://canonical.com/multipass (visited on 06/07/2026).

[6] Python Software Foundation. *Python*. Project website. URL: https://www.python.org/ (visited on 06/07/2026).

[7] A. Amato, H. Bastiaansen, W. Datema, M. Fogli, R. Fronteddu, R. Galliera, J. van der Geest, T. Kudla, P. Sanchez, N. Suri, and S. Watson. "A Performance Cost/Benefit Analysis of Adaptive Computing in the Tactical Edge." In: *2024 International Conference on Military Communication and Information Systems (ICMCIS)*. IEEE, 2024. DOI: 10.1109/ICMCIS61231.2024.10541027.

[8] CRIU Project. *Usage scenarios*. URL: https://criu.org/Usage_scenarios (visited on 05/26/2026).

[9] CRIU Project. *Live migration*. URL: https://criu.org/Live_migration (visited on 05/06/2026).

[10] CRIU Project. *Iterative migration*. URL: https://criu.org/Iterative_migration (visited on 05/06/2026).

[11] CRIU Project. *Lazy migration*. URL: https://criu.org/Lazy_migration (visited on 05/06/2026).

[12] CRIU Project. *Page server*. URL: https://criu.org/Page_server (visited on 05/06/2026).

[13] I. Raimondi. *Evaluation of Live Migration Strategies at the Edge*. Experiment code, orchestration scripts, plotting utilities, and generated artifacts. URL: https://github.com/HMF2475/Evaluation-of-Live-Migration-Strategies-at-the-Edge (visited on 06/04/2026).

[14] M. Gardner. "Mathematical Games: The Fantastic Combinations of John Conway's New Solitaire Game Life." In: *Scientific American* 223.4 (Oct. 1970), pp. 120–123.

[15] E. Tinto. *wasm-migrate-commands*. Upstream repository for the WebAssembly migration command prototype. URL: https://github.com/TintoEdoardo/wasm-migrate-commands (visited on 06/07/2026).

[16] I. Raimondi. *wasm-migrate-commands*. Fork used by the experiment repository. URL: https://github.com/HMF2475/wasm-migrate-commands (visited on 06/07/2026).

[17] Oakestra. *Oakestra*. Project website. URL: https://www.oakestra.io/ (visited on 06/04/2026).

[18] G. Bartolomeo, J. Cao, X. Su, and N. Mohan. "Characterizing Distributed Mobile Augmented Reality Applications at the Edge." In: *Companion of the 19th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT Companion '23)*. Used as the source for representative WiFi, 5G, and LTE network profile values. Association for Computing Machinery, 2023, pp. 1–10. DOI: 10.1145/3624354.3630584.

[19] Starlink. *Starlink Specifications*. Used as the source for representative Starlink network profile values. URL: https://starlink.com/legal/documents/DOC-1470-99699-90 (visited on 05/27/2026).

[20] T. Kudla, M. Fogli, S. Webb, G. Pingen, N. Suri, and H. Bastiaansen. "Quantifying the Performance of Cloud-Oriented Container Orchestrators on Emulated Tactical Networks." In: *IEEE Communications Magazine* 60.5 (2022), pp. 74–80. DOI: 10.1109/MCOM.003.00975.